

Contents

1	詳細設計 index(運営者システム)	6
1.1	0. 文書情報	6
1.2	1. 詳細設計方針	6
1.2.1	1.1 本書の位置付け	6
1.2.2	1.2 用語とスコープ	6
1.2.3	1.3 実装データ形式・命名	6
1.2.4	1.4 詳細化区分	6
1.2.5	1.5 最重要方針(再掲)	7
1.2.6	1.6 スコープ(運営者コンソール限定)	7
1.2.7	1.7 メインシステム正本範囲	7
1.2.8	1.8 MVP 初期値	8
1.3	2. システム全体構成	8
1.3.1	2.1 Worker トポロジー	8
1.3.2	2.2 Worker 責務マトリクス	9
1.3.3	2.3 ドメイン・URL 分離 (D-01)	9
1.3.4	2.4 環境構成	10
1.3.5	2.5 リクエストフロー	10
1.3.6	2.6 ディレクトリ構成例	11
1.4	3. 共通処理方針	12
1.4.1	3.1 メール通知	12
1.4.2	3.2 セキュリティ	12
1.4.3	3.3 構造化ログ	12
1.4.4	3.4 リリース戦略	12
1.5	4. 詳細設計ファイル一覧	12
1.6	5. 実装順序	13
1.7	6. 非機能・運用設計サマリ	13
1.8	7. リリース戦略	14
1.8.1	7.1 リリース運用	14
1.8.2	7.2 設計決定 D-01~D-20 詳細化マッピング	14
1.8.3	7.3 詳細設計引継ぎ事項 確定マッピング	14
1.9	8. 未確定事項・確認事項	15
2	DD01: 運営者認証・6 段認可 (運営者システム)	15
2.1	0. 文書情報	15
2.2	1. 対象範囲	15
2.3	2. 収録ロジック・対応章	16
2.4	3. 詳細設計本文	16
2.4.1	3.1.6 段認可判定モデル	16
2.4.2	3.2 ログインフロー	16
2.4.3	3.3 MFA(TOTP) 検証	17
2.4.4	3.4 セッション管理	17
2.4.5	3.5 再認証(クリティカル操作)	17
2.4.6	3.6 IP 許可リスト	17
2.4.7	3.7 運営者アカウント・ライフサイクル(FR-225)	17
2.4.8	3.8 ミドルウェア実装ガイドライン	18
2.4.9	3.9 HKDF info 値リスト(認証関連、★TH-12)	18
2.4.10	3.10 主要エラーコード	18
2.5	4. 関連設計	18
2.6	5. テスト観点	19
2.6.1	5.1 ユニットテスト	19
2.6.2	5.2 結合テスト (Miniflare)	19
2.6.3	5.3 E2E テスト (Playwright)	19
2.6.4	5.4 受入条件マッピング	19
2.7	6. 未確定事項・確認事項	19
3	DD02: 4-eyes 承認フロー (運営者システム)	19
3.1	0. 文書情報	19
3.2	1. 対象範囲	20
3.3	2. 収録ロジック・対応章	20
3.4	3. 詳細設計本文	20

3.4.1	3.1 4-eyes 対象 10 操作	20
3.4.2	3.2 状態遷移(<code>operator_approvals.state</code>)	21
3.4.3	3.3 <code>payload_hash</code> 検証	21
3.4.4	3.4 自己承認禁止	21
3.4.5	3.5 4-eyes 申請モジュール (CMP-E、★TH-5)	21
3.4.6	3.6 4-eyes 承認モジュール (CMP-F、★TH-5)	22
3.4.7	3.7 4-eyes API OpenAPI 抜粋	22
3.4.8	3.8 楽観ロック・同時実行制御	24
3.4.9	3.9 期限切れ自動バッチ	24
3.4.10	3.10 4-eyes フロー E2E テストシナリオ (付録 C.X)	25
3.5	4. 関連設計	25
3.6	5. テスト観点	25
3.6.1	5.1 ユニットテスト	25
3.6.2	5.2 結合テスト (Miniflare)	25
3.6.3	5.3 E2E テスト (Playwright)	25
3.6.4	5.4 負荷試験	26
3.6.5	5.5 受入条件マッピング	26
3.7	6. 未確定事項・確認事項	26

4 DD03: 監査ハッシュチェーン (運営者システム) 26

4.1	0. 文書情報	26
4.2	1. 対象範囲	26
4.3	2. 収録ロジック・対応章	26
4.4	3. 詳細設計本文	27
4.4.1	3.1 監査ハッシュチェーン構造	27
4.4.2	3.2 書込トランザクション (D-04)	27
4.4.3	3.3 日次ハッシュチェーン検証 (<code>AuditChainVerifierWorker</code> 、D-04)	27
4.4.4	3.4 不一致検出時の通知ポリシー	28
4.4.5	3.5 監査ログ検索 API	28
4.4.6	3.6 監査エクスポート (HMAC 署名、D-17)	29
4.4.7	3.7 3 区分保持 (D-08、付録 F)	29
4.4.8	3.8 <code>accounts_retired</code> 永久保持	29
4.4.9	3.9 構造化ログ (監査関連)	29
4.4.10	3.10 主要エラーコード	30
4.5	4. 関連設計	30
4.6	5. テスト観点	30
4.6.1	5.1 ユニットテスト	30
4.6.2	5.2 結合テスト (Miniflare)	30
4.6.3	5.3 E2E テスト (Playwright)	30
4.6.4	5.4 負荷試験	31
4.6.5	5.5 受入条件マッピング	31
4.7	6. 未確定事項・確認事項	31

5 DD04: Stripe Webhook 一次受信 (運営者システム) 31

5.1	0. 文書情報	31
5.2	1. 対象範囲	31
5.3	2. 収録ロジック・対応章	32
5.4	3. 詳細設計本文	32
5.4.1	3.1 受信エンドポイント (D-10)	32
5.4.2	3.2 受信処理フロー	32
5.4.3	3.3 状態遷移(<code>webhook_events.state</code>)	33
5.4.4	3.4 冪等性ストア	33
5.4.5	3.5 ペイロード正規化 + 差分検出 (D-06)	33
5.4.6	3.6 複数バージョン併存運用ルール	34
5.4.7	3.7 除外フィールド適用例	35
5.4.8	3.8 DLQ 自動指数バックオフ + 手動リプレイ	35
5.4.9	3.9 ペイロード差分検出後のレビュー (SCR-099)	36
5.4.10	3.10 Stripe API 呼出 (★TH-9)	36
5.4.11	3.11 連携 IF #10(本書 → メイン)	36
5.4.12	3.12 主要エラーコード	37
5.5	4. 関連設計	37
5.6	5. テスト観点	37

5.6.1	5.1 ユニットテスト	37
5.6.2	5.2 結合テスト (Miniflare + Stripe Test Mode)	37
5.6.3	5.3 E2E テスト (Playwright)	37
5.6.4	5.4 負荷試験	37
5.6.5	5.5 受入条件マッピング	38
5.7	6. 未確定事項・確認事項	38
6	DD05: 削除データ復元 (運営者システム)	38
6.1	0. 文書情報	38
6.2	1. 対象範囲	38
6.3	2. 収録ロジック・対応章	38
6.4	3. 詳細設計本文	39
6.4.1	3.1 削除データのライフサイクル (D-07)	39
6.4.2	3.2 復元時の副作用ロールバック (FR-204、D-07)	39
6.4.3	3.3 復元実行 API	39
6.4.4	3.4 復元実行フロー	40
6.4.5	3.5 4-eyes 承認ログ対象 (MVP)	40
6.4.6	3.6 検索・一覧 (SCR-090)	40
6.4.7	3.7 副作用 (a)~(g) ロールバック詳細	41
6.4.8	3.8 復元結果の記録	41
6.4.9	3.9 通知集約 (FR-211、D-19)	42
6.4.10	3.10 主要エラーコード	42
6.5	4. 関連設計	42
6.6	5. テスト観点	42
6.6.1	5.1 ユニットテスト	42
6.6.2	5.2 結合テスト (Miniflare)	42
6.6.3	5.3 E2E テスト (Playwright)	43
6.6.4	5.4 受入条件マッピング	43
6.7	6. 未確定事項・確認事項	43
7	DD06: お知らせ承認・配信 (運営者システム)	43
7.1	0. 文書情報	43
7.2	1. 対象範囲	43
7.3	2. 収録ロジック・対応章	43
7.4	3. 詳細設計本文	44
7.4.1	3.1 状態遷移 (<code>announcement_drafts.state</code>)	44
7.4.2	3.2 お知らせ作成 API	44
7.4.3	3.3 二段階 HTML サニタイズ (R-011)	45
7.4.4	3.4 予約配信スケジューラ (D-09)	45
7.4.5	3.5 連携 IF #7 JSON Schema(本書 → メイン)	46
7.4.6	3.6 訂正告知 (FR-189)	46
7.4.7	3.7 テスト送信 (<code>POST /announcements/{id}/test-send</code>)	46
7.4.8	3.8 通知配信 (D-20)	46
7.4.9	3.9 10 分集約窓 (D-19、運営者操作通知)	47
7.4.10	3.10 主要エラーコード	47
7.5	4. 関連設計	47
7.6	5. テスト観点	47
7.6.1	5.1 ユニットテスト	47
7.6.2	5.2 結合テスト (Miniflare)	47
7.6.3	5.3 E2E テスト (Playwright)	48
7.6.4	5.4 品質回帰	48
7.6.5	5.5 受入条件マッピング	48
7.7	6. 未確定事項・確認事項	48
8	DD07: KV・R2 オブジェクト (運営者システム)	48
8.1	0. 文書情報	48
8.2	1. 対象範囲	48
8.3	2. 収録ロジック・対応章	48
8.4	3. 詳細設計本文	49
8.4.1	3.1 KV キー命名規則	49
8.4.2	3.2 KV キー全表	49
8.4.3	3.3 R2 オブジェクトパス	50

8.4.4	3.4 KV キー早見表 (付録 J 同等)	51
8.4.5	3.5 監視 KPI 動的閾値の初期値 (<code>monitoring:thresholds:<kpi_id></code>)	51
8.4.6	3.6 サークットブレーカ KV (<code>cb:internal-api:<endpoint></code>)	52
8.4.7	3.7 計画停止 KV (<code>sre:planned-outage:<id></code>)	52
8.4.8	3.8 KV 運用ガイドライン	52
8.4.9	3.9 R2 運用ガイドライン	52
8.5	4. 関連設計	52
8.6	5. テスト観点	53
8.6.1	5.1 ユニットテスト	53
8.6.2	5.2 結合テスト (Miniflare)	53
8.6.3	5.3 E2E テスト	53
8.6.4	5.4 運用テスト	53
8.7	6. 未確定事項・確認事項	53
9	DD08: Cron 実装 (運営者システム)	53
9.1	0. 文書情報	53
9.2	1. 対象範囲	54
9.3	2. 収録ロジック・対応章	54
9.4	3. 詳細設計本文	54
9.4.1	3.1 Cron Triggers UTC cron 式表 (全 cron)	54
9.4.2	3.2 各 cron の擬似コード	55
9.4.3	3.3 Cron 失敗時の通知ポリシー	57
9.4.4	3.4 Cron UTC 一覧 (付録 L 同等)	57
9.4.5	3.5 wrangler.toml 設定例	57
9.4.6	3.6 cron 実装ガイドライン	58
9.5	4. 関連設計	58
9.6	5. テスト観点	58
9.6.1	5.1 ユニットテスト	58
9.6.2	5.2 結合テスト (Miniflare)	58
9.6.3	5.3 E2E テスト	58
9.6.4	5.4 受入条件マッピング	58
9.7	6. 未確定事項・確認事項	59
10	DD09: 監査 action コード (運営者システム)	59
10.1	0. 文書情報	59
10.2	1. 対象範囲	59
10.3	2. 収録ロジック・対応章	59
10.4	3. 詳細設計本文	59
10.4.1	3.1 命名規約	59
10.4.2	3.2 全 action コード一覧 (約 60 件)	59
10.4.3	3.3 action コード使用ルール	62
10.4.4	3.4 3 区分保持の物理対応 (<code>action_code × retention_class</code>)	62
10.4.5	3.5 <code>retention_class</code> 別の物理削除契機	63
10.4.6	3.6 action コード追加プロセス	63
10.4.7	3.7 監査記録の必須項目	63
10.5	4. 関連設計	63
10.6	5. テスト観点	63
10.6.1	5.1 ユニットテスト	63
10.6.2	5.2 結合テスト (Miniflare)	64
10.6.3	5.3 CI 整合性チェック	64
10.6.4	5.4 月次運用テスト	64
10.7	6. 未確定事項・確認事項	64
11	DD10: PII 偽陽性報告 (運営者システム)	64
11.1	0. 文書情報	64
11.2	1. 対象範囲	64
11.3	2. 収録ロジック・対応章	65
11.4	3. 詳細設計本文	65
11.4.1	3.1 PII 検出層 (参考)	65
11.4.2	3.2 状態遷移 (<code>pii_false_positive_reports.state</code>)	65
11.4.3	3.3 3 営業日判定タイマー (NFR-805)	65
11.4.4	3.4 PII 偽陽性報告作成 API	65

11.4.5	3.5 PII ルール改定 API(段階ロールアウト、D-13)	66
11.4.6	3.6 SCR-098 主要機能	66
11.4.7	3.7 ルール改定例	66
11.4.8	3.8 ロールアウト判定アルゴリズム (メイン側で参照)	67
11.4.9	3.9 PII マスキング (報告データ)	67
11.4.10	3.10 主要エラーコード	67
11.5	4. 関連設計	68
11.6	5. テスト観点	68
11.6.1	5.1 ユニットテスト	68
11.6.2	5.2 結合テスト (Miniflare)	68
11.6.3	5.3 E2E テスト (Playwright)	68
11.6.4	5.4 品質回帰	68
11.6.5	5.5 受入条件マッピング	68
11.7	6. 未確定事項・確認事項	68

12DD11: 状態遷移詳細 (運営者システム) 69

12.1	0. 文書情報	69
12.2	1. 対象範囲	69
12.3	2. 収録ロジック・対応章	69
12.4	3. 詳細設計本文	69
12.4.1	3.1 <code>webhook_events.state</code> (12 状態、AC-041)	69
12.4.2	3.2 <code>operator_approvals.state</code> (7 状態、FR-226)	70
12.4.3	3.3 <code>announcement_drafts.state</code> (8 状態、FR-188 / FR-189)	70
12.4.4	3.4 <code>pii_false_positive_reports.state</code> (6 状態、FR-064)	70
12.4.5	3.5 <code>webhook_payload_diffs.state</code> (4 状態、AC-041 異常系)	70
12.4.6	3.6 <code>accounts.contract_status</code> (参考、メイン主管)	71
12.4.7	3.7 状態遷移実装ガイドライン	71
12.4.8	3.8 状態遷移共通エラーコード	71
12.4.9	3.9 状態テーブル一覧	71
12.4.10	3.10 状態遷移サマリ	71
12.5	4. 関連設計	72
12.6	5. テスト観点	73
12.6.1	5.1 ユニットテスト	73
12.6.2	5.2 結合テスト (Miniflare)	73
12.6.3	5.3 E2E テスト (Playwright)	73
12.6.4	5.4 状態遷移マトリクス検証 CI	73
12.7	6. 未確定事項・確認事項	73

13DD12: 運営者用ログ・テスト (運営者システム) 73

13.1	0. 文書情報	73
13.2	1. 対象範囲	74
13.3	2. 収録ロジック・対応章	74
13.4	3. 詳細設計本文	74
13.4.1	3.1 構造化ログ JSON Schema(★TH-11)	74
13.4.2	3.2 機密項目マスキング	74
13.4.3	3.3 エラー ID 体系	75
13.4.4	3.4 RFC 7807 problem+json 形式	75
13.4.5	3.5 テスト種別 × 責務 (基本設計 §14.1 準拠)	75
13.4.6	3.6 SCR × FR × AC × テストレベル早見表	76
13.4.7	3.7 受入条件マッピング (本書側主管)	77
13.4.8	3.8 品質回帰データセット	77
13.4.9	3.9 負荷試験 (顧管側、メイン §17.5 と対称形)	77
13.4.10	3.10 トレース ID 採番	77
13.4.11	3.11 ログ集約 + 検索	78
13.4.12	3.12 テストインフラ	78
13.4.13	3.13 リリース判定チェックリスト (MVP)	78
13.5	4. 関連設計	78
13.6	5. テスト観点	78
13.6.1	5.1 構造化ログのスキーマ整合性	78
13.6.2	5.2 テストインフラ自体の検証	78
13.6.3	5.3 受入条件カバレッジ計算	79
13.7	6. 未確定事項・確認事項	79

1 詳細設計 index(運営者システム)

1.1 0. 文書情報

項目	内容
文書種別	詳細設計 index
対象システム	FAQ AI ウィジェット SaaS / 運営者システム (顧客管理システム / admin.open-faq.example.com)
版数	v1.0
作成日	2026-05-17
ステータス	承認済
クラウド前提	Cloudflare(Workers / D1 / KV / R2 / Queues / Workers AI / Secrets Store / Cron Triggers)
メール配信	Resend
課金プロバイダ	Stripe(Webhook 一次受信は運営者側)
上位文書	基本設計 / 要件定義
兄弟文書	メインシステム 詳細設計 index

1.2 1. 詳細設計方針

1.2.1 1.1 本書の位置付け

本書は基本設計 v3.0 を実装着手可能な粒度に詳細化した文書である。基本設計が「方式・全体像」を確定するのに対し、本書は「DDL・API スキーマ・KV キー命名・Cron UTC 式・HKDF info 値・action コード一覧・エラーログスキーマなど、実装担当者が即時着手できる具体値」を確定する。

表記、採番、文書同期、基本設計と詳細設計の境界などの文書保守ルールは [CLAUDE.md](#) に置く。

1.2.2 1.2 用語とスコープ

- 用語: 基本設計 付録 A の用語を参照する。
- 主語: 「運営者」「管理者ユーザー」「エンドユーザー」を厳密に区別。本書のスコープは運営者 (**service_operator**) が触れる機能のみ。
- 本書は **MVP 範囲のみ** を記載する。MVP で実現する実装仕様、初期値、運用制約だけを本文に置く。

1.2.3 1.3 実装データ形式・命名

項目	規約
ID 形式	ULID 26 文字 (運営者・チケット・申請・リプライ等) / Stripe ID は evt_*sub_*inv_*cn_* をそのまま PK
命名	テーブル: snake_case 複数形 / カラム: snake_case 単数形 / 状態列: state または status / KV キー: <feature>.<scope>.<id> コロン区切り英小文字 + ハイフン
日時	TIMESTAMP 型 ISO 8601 UTC(2026-05-12T10:00:00Z)。表示時に JST 変換
真偽値	INTEGER 0/1
JSON	TEXT 列に JSON 文字列で保存
エラー	RFC 7807 application/problem+json(type / title / status / code / detail / trace_id)
改行	ファイル末尾は LF 改行で終わる

1.2.4 1.4 詳細化区分

区分	本書で確定する内容
API	エンドポイントごとの OpenAPI スキーマ、JSON Schema、エラーコード一覧 (TH-1)
DDL	NULL 制約、外部キー、CHECK 制約、インデックスの正確な構文 (TH-2)
画面詳細	SCR-090~099 のコンポーネント、項目、API 呼出、4-eyes UI (TH-3, TH-5)
HTML サニタイザ	許可タグ・属性ホワイトリスト具体値 (TH-4)
監査 action コード	<resource>.<verb> 全コード一覧 + retention_class (TH-6)
KV キー一覧	個別キーの命名・TTL 値・サンプル値 (TH-7)

区分	本書で確定する内容
Webhook 除外フィールド	D-06 付録 H 列挙の Stripe API バージョン別拡充 (TH-8)
Stripe API	Subscription resume / Invoice / Credit Note の具体パラメータ (TH-9)
Cron 実装	UTC 表現の cron 式 (TH-10)
エラーログ詳細	構造化ログのスキーマ、必須フィールド、機密項目マスキング (TH-11)
HKDF info 値	正式リスト (TH-12)
その他	連携 IF #1~#12 通信スキーマ、Runbook 雛形、SLA 計測ロジック、状態遷移詳細

1.2.5 1.5 最重要方針 (再掲)

基本設計 §0 / 要件 §2 を継承:

1. 運営者は本サービス全契約横断の最小限の運用権限のみを持ち、利用者側業務 (FAQ 公開・チャット返信・退会受付など) を肩代わりしない。
2. 4-eyes 原則は MVP ハードゲート 3 操作 + 承認ログのみ 7 操作で実装する。
3. すべての運営者操作は監査ログ (ハッシュチェーン) に記録し、3 区分保持 (1y / 5y / 7y) で物理管理する。
4. 課金 Webhook の一次受信は本書側のみが行い、メイン側は受けない (D-10)。
5. 運営者プレーンと利用者プレーンはサブドメイン分離 (D-01)、Worker 分離 (D-02) で攻撃面を最小化する。

1.2.6 1.6 スcope (運営者コンソール限定)

本書はメイン §1.4 で「運営者主管」と区分されたものを詳細化する:

#	機能ブロック	主管 SCR	主管 FR / NFR
1	運営者認証 (MFA + IP 許可 + 再認証 5 分)	ログイン画面、全画面	FR-220, FR-221, FR-222, FR-007, NFR-311
2	4-eyes 申請承認 (MVP 3 ハードゲート + 7 承認ログ)	承認待ち一覧、各操作モデル	FR-226, § 6.2.1 (基本設計)
3	削除データ参照・復元 (副作用 (a)~(g) ロールバック)	SCR-090 / SCR-091	FR-200~FR-211, FR-222, FR-223
4	AI 推論パラメータ 3 階層上書き	SCR-092	FR-055, FR-061~FR-066, AC-034
5	契約別レート/予算上書き + サプレスリスト復帰	SCR-093	FR-121, FR-128, FR-224, NFR-503, NFR-504
6	お知らせ作成・配信 (運営者)	SCR-094	FR-149, FR-188, FR-189
7	監査ログ閲覧・エクスポート (HMAC 署名)	SCR-096	FR-229, FR-230, FR-232, NFR-306, NFR-602
8	課金 Webhook リプレイ・DLQ 操作	SCR-097	FR-302, NFR-808, NFR-809
9	PII 誤検出報告管理 (3 営業日判定)	SCR-098	FR-060, FR-064, NFR-805
10	Webhook ペイロード差分検出	SCR-099	FR-302 異常系, AC-041
11	月次請求確定 cron (月初 02:00 JST)	バックグラウンド	FR-303, AC-042
12	監査ハッシュチェーン日次検証	バックグラウンド	NFR-306, RB-017
13	運営者操作通知 (FR-211、10 分集約)	連携 IF #12	FR-211, D-19

1.2.7 1.7 メインシステム正本範囲

次はメインシステム詳細設計の正本対象であり、本書では連携境界として扱う:

- ・利用者認証・管理者ユーザー登録完了・パスワードリセット (/auth/*)
- ・公開ウィジェット (/widget/*)
- ・未解決質問・個別チャット (/inquiries/*, /chat/*)
- ・FAQ CRUD・FTS 検索 (/projects/*/faqs/*)
- ・管理者ユーザー inbox (主管はメイン、本書は連携 IF #12 で配信指示のみ)
- ・ウィジェット側レート制限の実適用 (主管はメイン、本書は連携 IF #5 で上書き値同期)
- ・メール送信 (主管はメイン、本書は運営者通知 + テスト送信のみ Resend 直叩き)
- ・サプレスリスト本体 (主管はメイン、本書は SCR-093 経由で個別アドレス復帰承認のみ)

メイン主管エンティティ (accounts, projects, faqs, question_logs, chat_rooms, chat_messages, inquiries, inbox_messages 等) は [メインテーブル設計](#) を正本参照し、本書では運営者側主管 20 エンティティのみ DDL を書き下す。

1.2.8 1.8 MVP 初期値

項目	本書での扱い
Workers AI 利用モデル	@cf/meta/llama-3.1-8b-instruct
データリージョン	Cloudflare apac
AI しきい値初期値	信頼度 0.60 / 関連度 0.50
PII 検出	第 1 層 (正規表現) + 第 3 層 (FAQ 整合性検査)
4-eyes	ハードゲート 3 操作 + 承認ログのみ 7 操作

1.3 2. システム全体構成

1.3.1 2.1 Worker トポロジー

本書側は [メインシステム](#) と別の [wrangler プロジェクト](#) (D-02) としてデプロイする。攻撃面の縮小、MFA/IP 許可ミドルウェアの分離、独立デプロイ・独立ロールバックを目的とする。

flowchart TB

```
subgraph Browser[運営者ブラウザ]
    SPA[Admin SPA<br/>admin.open-faq.example.com<br/>Cloudflare Pages]
end

subgraph External[外部サービス]
    Stripe[Stripe API / Webhook]
    Resend[Resend API / Webhook]
end

subgraph AdminPlane[運営者プレーン Worker 群]
    AdminConsole[AdminConsoleWorker<br/>SPA 配信 + 全運営 API<br/>/admin/api/v1/*]
    BillingWebhook[BillingWebhookWorker<br/>POST /webhooks/stripe<br/>POST /webhooks/resend]
    Scheduler[AnnouncementSchedulerWorker<br/>1 分 cron]
    ChainVerifier[AuditChainVerifierWorker<br/>日次 02:00 JST cron]
    MonthlyBilling[MonthlyBillingCronWorker<br/>月初 02:00 JST cron]
    RetentionPurge[RetentionPurgeWorker<br/>日次 03:00 JST cron]
    AuditArchive[R2AuditArchiveWorker<br/>年次 12/31 04:00 JST cron]
    DlqBackoff[DLQAutoBackoffWorker<br/>5 分 cron]
end

subgraph DataPlane[データプレーン]
    D1Admin[(D1: admin_db<br/>運営者主管 21 テーブル)]
    KV[(KV: admin_cache<br/>セッション / レート / AI パラメータ / フラグ)]
    R2[(R2: admin_archive<br/>DLQ 退避 / 監査エクスポート / アーカイブ)]
    Queues[(Queues: admin_queues<br/>webhook-dlq / notification-batch / restore-rollback)]
    AI[(Workers AI<br/>AI 推論 / PII 検査)]
    Secrets[Secrets Store<br/>マスター / API キー]
end

subgraph Main[メインシステム]
    MainWorker[Main Worker<br/>app.open-faq.example.com<br/>/internal/admin-integration/v1/]
    MainDB[(メイン D1)]
end

SPA -- Cookie + CSRF --> AdminConsole
Stripe -- POST /webhooks/stripe<br/>HMAC-SHA256 署名 --> BillingWebhook
Resend -- POST /webhooks/resend --> BillingWebhook
```

AdminConsole --> D1Admin
AdminConsole --> KV
AdminConsole --> Secrets
BillingWebhook --> D1Admin
BillingWebhook --> R2
BillingWebhook --> Queues
Scheduler --> D1Admin
Scheduler --> Queues
ChainVerifier --> D1Admin
MonthlyBilling --> D1Admin
MonthlyBilling --> Stripe
RetentionPurge --> D1Admin
RetentionPurge --> R2
AuditArchive --> D1Admin
AuditArchive --> R2
DlqBackoff --> Queues
DlqBackoff --> D1Admin

AdminConsole -- mTLS + JWT
 連携 IF #1, #2, #4, #5, #6, #7, #12 --> MainWorker
MainWorker -- mTLS + JWT
 連携 IF #8, #9 --> AdminConsole
BillingWebhook -- mTLS + JWT
 連携 IF #10 --> MainWorker
Scheduler -- 連携 IF #7 --> MainWorker
AdminConsole -. 読取参照 (必要時).-> MainDB

1.3.2 2.2 Worker 責務マトリクス

Worker	責務	バインド	主な処理時間
AdminConsole	運営者ログイン、MFA、IP 許可、再認証、4-eyes 承認、SCR-090~099 の全 API、内部 API クライアント (連携 IF 発信側)	D1: admin_db / KV: admin_cache / Secrets / Queues producer / R2 reader	リクエスト駆動
BillingWebhook	Worker / Resend Webhook 一次受信、署名検証、event_id 冪等、ペイロード正規化 + 差分検出、内部転送 (連携 IF #10)、DLQ 投入 + R2 退避	D1 / R2 / Queues / Secrets	リクエスト駆動 (30 秒以内)
Announcement	お知らせ配信予約の 1 分ポーリング、scheduled_at ☒ now+5min を sending 遷移、連携 IF #7 でメイン転送	D1 / Queues	1 分間隔
AuditChainVerifier	監査ログ全件のハッシュチェーン再計算 (D-04)、不一致時の運営者 inbox + メール通知	D1 / Secrets (チェーン)	日次 02:00 JST
MonthlyBilling	月初 02:00 JST の請求書発行 (D-11)、(owner_account_id, billing_year_month) 冪等、Stripe Invoice API 発行、契約通知 + 監査記録	D1 / Stripe / Queues	月次
RetentionPurge	監査ログの 3 区分 (1y / 5y / 7y) 物理削除バッチ、R2 アーカイブ書出後に DELETE	D1 / R2	日次 03:00 JST
R2AuditArchive	Worker の年次 R2 圧縮アーカイブ書出	D1 / R2	年次 12/31 04:00 JST
DLQAutoBackoff	Worker 側 Queues DLQ の自動指数 BO (1m → 4m → 16m、最大 3 回)、1 時間経過で dlq_manual_replay 遷移	Queues / D1	5 分間隔

1.3.3 2.3 ドメイン・URL 分離 (D-01)

ホスト	用途	入口制御
admin.open-faq.example.com	運営者プレーン全機能	IP 許可リスト (エッジで 403 Forbidden 早期返却)
admin.open-faq.example.com/login	運営者ログイン	利用者側からのリンクなし (発見可能性を下げる)
admin.open-faq.example.com/webhooks/stripe	Stripe Webhook 受信専用 (D-10、唯一の受信先)	Stripe-Signature 検証、IP 許可リスト適用外

ホスト	用途	入口制御
admin.open-faq.example.com/webhooks/resend	Resend Webhook 受信	Resend Signature 検証、IP 許可リスト適用外
app.open-faq.example.com	利用者ブレーン(メインシステム正本)	本書側 SPA からは Cookie スコープ分離 (Domain=admin.open-faq.example.com)
app.open-faq.example.com/internal/admin-integration/v1/*	メイン側内部 API(連携 IF #1～#12)	mTLS + JWT(本書側 Worker からのみアクセス可能)

/webhooks/stripe と /webhooks/resend は IP 許可リストの適用外とする (外部サービスからのコールバックを受信するため) が、署名検証で代替する。それ以外のすべてのパスは IP 許可リスト適用後にルーティングされる。

1.3.4 2.4 環境構成

環境	用途	wrangler.toml 環境名	データ
dev	開発 (各開発者ローカル)	local	Miniflare 内 D1 / KV / R2
stg	結合テスト・E2E	staging	本番別 D1 / 別 R2 バケット
prod	本番	production	admin_db / admin_cache / admin_archive

各環境で個別の Stripe / Resend Webhook Secret、HKDF マスター 、JWT 署名 を発行する。staging から本番への昇格は GitHub OIDC + Environment Protection Rules で 2 名承認を必須化。

1.3.5 2.5 リクエストフロー

sequenceDiagram

```

participant Browser as 運営者ブラウザ
participant Edge as Cloudflare Edge
participant Worker as AdminConsoleWorker
participant KV
participant D1
participant Main as Main Worker

Browser->>Edge: GET /admin/api/v1/...
Edge->>Edge: IP 許可リスト判定 <br/>(KV: operator-ip-allowlist:*)
alt 未許可 IP
    Edge-->>Browser: 403 Forbidden
else 許可 IP
    Edge->>Worker: ルーティング
    Worker->>KV: GET operator-session:<sid>
    alt セッションなし / 失効
        Worker-->>Browser: 401 UNAUTHENTICATED
    else 有効
        Worker->>D1: SELECT operator_sessions WHERE id=?<br/>(KV TTL 60s ヒット時はスキップ)
        Worker->>Worker: MFA 検証フラグ確認
        alt MFA 未完了
            Worker-->>Browser: 403 FORBIDDEN_MFA_REQUIRED
        else MFA 完了
            Worker->>Worker: 再認証要否判定 <br/>(5 分以内のクリティカル操作)
            alt 再認証必須 & 期限切れ
                Worker-->>Browser: 403 RE_AUTH_REQUIRED
            else
                Worker->>Worker: 4-eyes 要否判定 <br/>(KV: feature:hard-gate:<action>)
                alt ハードゲート ON & 未承認
                    Worker-->>Browser: 403 FORBIDDEN_HARD_GATE
                else
                    Worker->>D1: 処理実行
                    Worker->>D1: audit_logs INSERT
                    Worker->>Main: 連携 IF 発火 (必要時)
                end
            end
        end
    end
end

```

```

Worker-->>Browser: 200 OK
end
end
end
end
end
end
end

```

1.3.6 2.6 ディレクトリ構成例

実装プロジェクトの参考レイアウト (別 wrangler プロジェクト):

```

admin/
├── wrangler.toml                # AdminConsoleWorker
├── wrangler.billing-webhook.toml # BillingWebhookWorker
├── wrangler.scheduler.toml      # AnnouncementSchedulerWorker, ...
├── src/
│   ├── admin-console/
│   │   ├── handlers/           # /admin/api/v1/* ハンドラ
│   │   │   ├── auth.ts
│   │   │   ├── approvals.ts
│   │   │   ├── deleted-resources.ts
│   │   │   ├── restorations.ts
│   │   │   ├── ai-parameters.ts
│   │   │   ├── overrides.ts
│   │   │   ├── announcements.ts
│   │   │   ├── audit-logs.ts
│   │   │   ├── webhooks-ops.ts
│   │   │   └── pii-fp-reports.ts
│   │   ├── middleware/
│   │   │   ├── ip-allowlist.ts
│   │   │   ├── session.ts
│   │   │   ├── mfa.ts
│   │   │   ├── reauth.ts
│   │   │   ├── csrf.ts
│   │   │   ├── ticket-id.ts
│   │   │   └── four-eyes.ts
│   │   └── lib/
│   │       ├── audit.ts        # audit_logs INSERT + ハッシュチェーン
│   │       ├── hkdf.ts         # HKDF info 値別派生
│   │       ├── totp.ts
│   │       ├── argon2.ts
│   │       ├── pii.ts
│   │       └── stripe.ts
│   ├── billing-webhook/
│   │   ├── handler.ts
│   │   ├── verify.ts          # HMAC-SHA256 署名検証
│   │   ├── canonical.ts       # JSON 正規化 + SHA-256
│   │   └── forward.ts          # 連携 IF #10 mTLS+JWT
│   ├── scheduler/
│   │   ├── announcements.ts    # 1 分 cron
│   │   ├── audit-chain.ts      # 日次 02:00 JST
│   │   ├── monthly-billing.ts  # 月初 02:00 JST
│   │   ├── retention-purge.ts  # 日次 03:00 JST
│   │   └── r2-audit-archive.ts  # 年次 12/31
│   └── shared/
│       ├── errors.ts           # 全エラーコード定義
│       ├── action-codes.ts     # 監査 action コード列挙
│       └── kv-keys.ts           # KV キー定数

```

		logger.ts	# 構造化ログ
		types.ts	
		migrations/	
		d1-admin/	
		0001_init_operators.sql	
		0002_init_audit_logs.sql	
		0003_init_webhook_events.sql	
		...	
		tests/	
		unit/	# Vitest
		integration/	# Miniflare
		e2e/	# Playwright(SCR-090~094, SCR-096~099)
		webhook/	# Stripe Test Mode
		scripts/	
		audit-export-verify.ts	# HMAC 署名検証ツール (運営者配布)
		canonical-json.ts	# 正規化アルゴリズム参考実装
		openapi/	
		admin-api.yaml	# OpenAPI v3.1 抜粋

実装プロジェクトの実際のディレクトリ名は実装段階で確定して構わない(本書はあくまで責務マッピングの参考)。

1.4 3. 共通処理方針

1.4.1 3.1 メール通知

メール通知の正本は [基本設計 / メッセージ一覧](#) § 8(運営者向け通知 + IF #12 経由通知)。通知契機 13 種 (運営者宛)+ 5 種 (IF #12 経由 利用者向け)+ テンプレートは当該ドキュメントを正本とする。

通知重要度は **low/normal/high/critical** の 4 値。**critical** は強制送信 (オーナー + **users:manage** フラグ 保持メンバー全員に必ずメール配信) を予約する。10 分集約窓 (**notify-batch:<owner_account_id>:<kind>**) は D-19 で確定。

1.4.2 3.2 セキュリティ

セキュリティ詳細は [基本設計 / セキュリティ設計](#) を正本とする。本書では以下のサブ領域を詳細化する:

- 監査ログハッシュチェーン構造 (prev_hash + record_hash、HKDF info=**audit-chain**)
- 3 区分保持 (**general/billing/operator_high_priv**) の物理対応
- **accounts_retired** 永久保持 (法令対応)
- GDPR 越境移転対応
- Webhook ペイロード差分検出 (D-06、付録 H の除外フィールドリスト)
- HKDF info 値リスト (**internal-api/audit-chain/audit-export/pii-encryption** 等)

1.4.3 3.3 構造化ログ

エラー設計の正本は [基本設計 / エラー設計](#)。本書では構造化ログ JSON Schema、必須フィールド、機密項目マスキング規約を確定する (DD12 参照)。

1.4.4 3.4 リリース戦略

リリース運用とフィーチャーフラグ運用は [運用設計](#) 配下の運用手順書へ移管した。本書では設計決定 (D-01~D-20) との対応のみを扱う (§ 7 リリース戦略参照)。

1.5 4. 詳細設計ファイル一覧

ファイル	ドメイン	対応 FR	主関連章
DD01_運営者認証・6 段認可.md	運営者認証・6 段認可 (MFA / IP / 再認証)	FR-007, FR-220～FR-225, NFR-311	§ 3, § 5 SCR-AUTH/HOME, § 12
DD02_4-eyes 承認フロー.md	4-eyes 申請・承認・実行・自己承認禁止	FR-226	§ 5 SCR-APPROVALS, § 6.4, § 7.4, 付録 C.X
DD03_監査ハッシュチェーン.md	監査ハッシュチェーン構造・日次検証	FR-229～FR-232, NFR-306, NFR-602	§ 6 監査機能, § 12 ハッシュチェーン, § 16
DD04_Stripe_Webhook 一次受信.md	Stripe Webhook 一次受信・冪等性・差分検出	FR-302, NFR-808, NFR-809	§ 10 IF #10, § 6 webhook, § 7, 付録 B.2
DD05_削除データ復元.md	削除データ参照・復元 (副作用 a～g)	FR-200～FR-211, FR-222, FR-223	§ 5 SCR-090/091, § 6 復元, § 7.5, 付録 B.7
DD06_お知らせ承認・配信.md	お知らせ作成・予約・配信・取消	FR-149, FR-188, FR-189	§ 5 SCR-094, § 6 お知らせ, § 7.7, 付録 B.4
DD07_KV・R2 オブジェクト.md	KV キー全表 + R2 オブジェクトパス	TH-7 全般	§ 9 全文, 付録 J
DD08_Cron 実装.md	Cron UTC 式 + 擬似コード + 失敗時通知	FR-303, AC-042, D-04/05/09/11	§ 14 全文, 付録 L
DD09_監査 action コード.md	監査 action コード一覧 (約 60 件)	NFR-602, D-08	§ 15 全文, 付録 F
DD10_PII 偽陽性報告.md	PII 誤検出報告 3 営業日判定 + ルール改定	FR-060, FR-064, NFR-805, AC-036	§ 5 SCR-098, § 6 PII, 付録 B.5
DD11_状態遷移詳細.md	主要 5 状態機械 + ペイロード差分 state	D-08, AC-041	付録 B 全文
DD12_運営者用ログ・テスト.md	構造化ログ + テスト戦略 + 受入条件	NFR-103～106, AC-036～046	§ 16, § 17 全文

1.6 5. 実装順序

依存関係を踏まえた MVP 実装順序の推奨は次のとおり:

1. **認証 / 認可基盤** (DD01)—— 全機能の前提。MFA / IP / 再認証 / セッション。
2. **監査ログ基盤** (DD03, DD09)—— ハッシュチェーン + action コード一覧。以降の全ての操作で `audit_logs INSERT` が必須となる。
3. **4-eyes 基盤** (DD02)—— ハードゲート 3 操作の前提。`operator_approvals` テーブル + 6 段認可判定。
4. **KV / R2 / Cron 基盤** (DD07, DD08)—— 機能フラグ / レート上書き / 月次請求 / ハッシュ検証の土台。
5. **個別運営機能** (DD05 削除復元 / DD06 お知らせ / DD10 PII)—— 4-eyes と監査の上に乗る業務機能。
6. **Webhook / 差分検出** (DD04, DD11)—— Stripe 連携 + 差分検出 + ペイロードレビュー。
7. **構造化ログ / テスト** (DD12)—— 横串で全機能の検証。

順序は CI/CD パイプライン構築と並行可能。E2E テスト (Playwright) は DD02 / DD05 / DD06 の各 PR で追加し、4-eyes E2E は基盤完成後にまとめて整備する。

1.7 6. 非機能・運用設計サマリ

非機能・運用詳細は [運用設計](#) の「顧客管理システム 非機能・運用詳細由来」へ移管した。本書では cron、監査 action、API、DB など実装詳細との接点のみを扱う。

代表的な非機能目標 (運用設計から再掲):

- ・ 管理画面一覧 p95 ☒ 800ms (NFR-105)
- ・ 監査ログ検索 p95 ☒ 1000ms (100 万件以下)
- ・ 監査ログエクスポート ☒ 60s (10 万行)
- ・ ハッシュチェーン日次検証 ☒ 3600s (1000 万行以下)
- ・ DLQ 1h 超過件数 = 0 (高アラート閾値)
- ・ 4-eyes 承認待ち 72h 超過件数 = 0 (高アラート閾値)

詳細な KPI 動的閾値の初期値は [DD07_KV・R2 オブジェクト.md](#) § J.X を参照。

1.8 7. リリース戦略

1.8.1 7.1 リリース運用

リリース運用とフィーチャーフラグ運用は [運用設計](#) 配下の「顧客管理システム リリース戦略由来」へ移管した。本書では設計決定との対応のみを扱う。

1.8.2 7.2 設計決定 D-01～D-20 詳細化マッピング

#	決定事項	採用方針 (基本設計 §15 抜粋)	本書での詳細化箇所
D-01	運営者ログイン URL 分離	admin.open-faq.example.com 専用	index § 2.3, DD01
D-02	運営者 Worker のデプロイ単位	別 wrangler プロジェクト	index § 2.1, § 2.6
D-03	mTLS + JWT 管理	Origin CA + HKDF info=internal-api、60 日 dual-decrypt	DD01, DD04
D-04	ハッシュチェーン検証バッチ	日次 02:00 JST 全件再計算	DD03, DD08
D-05	祝日マスタ取得バッチ	年次 11/1 03:00 JST	DD08
D-06	Webhook ペイロード比較除外フィールド	付録 H に固定列挙	DD04
D-07	FR-204 副作用ロールバック (a)～(g)	7 ステップ具体手順	DD05
D-08	監査ログ 3 区分の物理分離方式	単一テーブル + retention_class 列	DD03, DD09
D-09	お知らせ配信予約スケジューラ	Cron Triggers + D1 ポーリング 1 分	DD06, DD08
D-10	課金 Webhook 一次受信エンドポイント	唯一の受信先	DD04
D-11	月次請求確定 cron 実装方式	月初 02:00 JST、(owner_account_id, year_month) UNIQUE	DD08
D-12	4-eyes 申請承認 UI/データモデル	operator_approvals テーブル	DD02
D-13	PII 誤検出ルール更新の即時反映	KV pii-rules:*、過去データ修正なし	DD10, DD07
D-14	契約別レート/予算上書き即時反映	KV TTL 30s + IF #5	DD07
D-15	AI 推論パラメータ 3 階層	KV ai-params:*、project > owner > global	DD07
D-17	監査エクスポート HMAC 署名	HKDF info=audit-export、SHA-256	DD03
D-18	運営者セッショントークン TTL	MVP 8h	DD01
D-19	運営者操作通知の集約窓	10 分集約 (owner_account_id, operation_kind)	DD06
D-20	運営者 inbox の保持	1 年 + retention_class='5y' リンク	DD06

1.8.3 7.3 詳細設計引継ぎ事項 確定マッピング

基本設計 § 16 引継ぎ事項 12 区分の本書での確定箇所:

#	引継ぎ事項 (基本設計 §16)	本書での確定箇所
TH-1	API 詳細	各 DD + 基本設計 / API 設計
TH-2	DDL	基本設計 / テーブル設計
TH-3	画面詳細	基本設計 / 画面設計
TH-4	HTML サニタイザ	DD06(お知らせ)
TH-5	4-eyes UI	DD02

#	引継ぎ事項 (基本設計 §16)	本書での確定箇所
TH-6	監査 action コード	DD09
TH-7	KV キー一覧	DD07
TH-8	Webhook 除外フィールド	DD04
TH-9	Stripe API	DD04
TH-10	Cron 実装	DD08
TH-11	エラーログ詳細	DD12
TH-12	HKDF info 値	DD01, DD03

全 12 区分確定済。未確定が発生した場合は運用設計の残課題管理へエスカレーション。

1.9 8. 未確定事項・確認事項

確認事項 ID	確認内容	優先度	ステータス
-	v1.0 リリース時点で全項目確定済み	低	確認済

\newpage

2 DD01: 運営者認証・6 段認可 (運営者システム)

2.1 0. 文書情報

項目	内容
文書名	DD01: 運営者認証・6 段認可 (運営者システム)
詳細設計 ID	DD01
対象システム	FAQ AI ウィジェット SaaS / 運営者システム
関連機能 ID	FR-007, FR-220, FR-221, FR-222, FR-223, FR-224, FR-225, NFR-310, NFR-311
作成日	2026-05-17
版数	v1.0
ステータス	承認済

2.2 1. 対象範囲

種別	ID	名称
機能	FR-007	再認証 (クリティカル操作 5 分以内)
機能	FR-220	運営者 MFA(TOTP)
機能	FR-221	運営者 IP 許可リスト
機能	FR-222	運営者操作監査
機能	FR-223	削除データ参照 (認可境界)
機能	FR-224	契約上書き (運営者操作通知)
機能	FR-225	運営者アカウント・ライフサイクル
画面	SCR-AUTH	運営者ログイン
画面	SCR-AUTH-M1	MFA 初回セットアップモダール
画面	SCR-HOME	運営者ダッシュボード (認証後ランディング)
画面	全画面	認証ミドルウェア共通
API	POST /auth/login / POST /auth/mfa/verify / POST /auth/reauth / POST /auth/logout	認証フロー
テーブル	operator_accounts / operator_sessions / operator_mfa_secrets / operator_ip_allowlists / operator_reauths	運営者主管

2.3 2. 収録ロジック・対応章

元章	元タイトル	概要
§ 3	利用者・権限詳細設計	単一 service_operator ロール、MFA、IP 許可、運営者ライフサイクル
§ 5 SCR-AUTH / SCR-HOME	画面詳細 (参照)	認証画面・ホーム画面のレイアウト・操作
§ 6 認証関連	機能詳細 (参照)	ログイン → MFA → セッション発行 → 再認証
§ 12 セキュリティ詳細 (認証部分)	認証セキュリティ	セッショントークン TTL、HKDF info 値、CSRF、Cookie 属性
§ 2.5	リクエストフロー	6 段認可判定 (IP → セッション → MFA → role → 再認証 → 4-eyes)

2.4 3. 詳細設計本文

2.4.1 3.1 6 段認可判定モデル

運営者リクエストはエッジ + Worker 内で次の 6 段階を **早期返却 / 同一順序** で評価する。失敗した段で即座にレスポンスを返却し、以降の段は評価しない。

段	判定対象	失敗時レスポンス	評価場所	参照キー
1	IP 許可リスト	403 Forbidden(エッジ)	Cloudflare Edge	KV: operator-ip-allowlist:<operator_id>
2	セッション有効性	401 UNAUTHENTICATED	AdminConsoleWorker	KV: operator-session:<sid>
3	MFA 検証フラグ	403 FORBIDDEN_MFA_REQUIRED	AdminConsoleWorker	セッション内 mfaVerifiedAt
4	ロール (service_operator)	403 FORBIDDEN_ROLE	AdminConsoleWorker	operator_accounts.role
5	再認証 (クリティカル操作)	403 RE_AUTH_REQUIRED	AdminConsoleWorker	re-auth:<sid>
6	4-eyes 承認	403 FORBIDDEN_HARD_GATE	AdminConsoleWorker	feature:hard-gate:<action>

判定の正本は [基本設計 / 認証・認可設計](#) § 3～§ 7。本書は実装上のミドルウェア順序と早期返却ルールを確定する。

2.4.2 3.2 ログインフロー

```
sequenceDiagram
    participant Browser as 運営者ブラウザ
    participant Edge as Cloudflare Edge
    participant Worker as AdminConsoleWorker
    participant D1
    participant KV

    Browser->>Edge: POST /admin/api/v1/auth/login {email, password}
    Edge->>Edge: IP 許可リスト判定
    alt 未許可 IP
        Edge-->>Browser: 403 Forbidden
    else 許可 IP
        Edge->>Worker: ルーティング
        Worker->>D1: SELECT operator_accounts WHERE email=?
        Worker->>Worker: Argon2id 検証
        alt 失敗
            Worker->>KV: INCR login-lockout:<ip_hash>:<account_id>
            Worker->>D1: audit_logs(operator.login.failed, 5y)
            Worker-->>Browser: 401 INVALID_CREDENTIALS
        else 成功
            Worker->>D1: audit_logs(operator.login.attempt, 1y)
```

```

    Worker-->>Browser: 200 {requireMfa: true, mfaChallengeId}
  end
end

```

2.4.3 3.3 MFA(TOTP) 検証

項目	仕様
アルゴリズム	TOTP(RFC 6238、SHA-1、30 秒、6 桁)
シークレット保管	operator_mfa_secrets.secret_encrypted(マスター + HKDF info=mfa-secret で派生暗号化)
初回セットアップ	mfa-setup:<account_id>(72h) に QR コードトークンを格納し、SCR-AUTH-M1 で読取・確認後 mfa_enabled=1
リカバリーコード	10 個、SHA-256 ハッシュで保管、使用時に該当エントリ無効化+監査 (mfa.recovery_code.used)
検証成功時	セッション内 mfaVerifiedAt = now() を記録し、KV へ即時反映
失敗時ロックアウト	5 回連続失敗で 15 分 (login-lockout)/ 同 IP・アカウントペア単位

2.4.4 3.4 セッション管理

項目	仕様
セッショントークン	ULID 26 文字 + ランダム 32 バイト + HKDF info=operator-session で派生 HMAC 検証用
TTL	8 時間 (D-18)、最終アクセスから滑走式に延長 (最大 24h、絶対上限)
Cookie 属性	Domain=admin.open-faq.example.com; Path=/; Secure; HttpOnly; SameSite=Strict; __Host- プレフィックス
CSRF	Cookie とは別の X-CSRF-Token ヘッダ (Worker 起動時に発行、セッション JSON 内に保管)
KV キャッシュ	operator-session:<sid>、TTL 60 秒。失効時は D1 にフォールバックし operator_sessions テーブルを参照
同時セッション	アカウント毎 最大 3 セッション、超過時は最古を revoked 遷移 (operator.session.revoke)

2.4.5 3.5 再認証 (クリティカル操作)

項目	仕様
対象操作	4-eyes 対象 10 操作 + お知らせ schedule / 監査エクスポート / Webhook リプレイ
検証ウィンドウ	直近 5 分以内 (FR-007)
認証要素	パスワード再入力 OR TOTP 再入力 (運営者選択)
状態保管	KV re-auth:<sid> に { operatorId, reauthenticatedAt, consumed }, TTL 15 分、consumed=true で即時失効
1 回限り	1 つの再認証トークンは 1 回のクリティカル操作のみで消費
監査	reauth(5y、{operatorId, reauthId})

2.4.6 3.6 IP 許可リスト

項目	仕様
値形式	CIDR 配列 (IPv4 / IPv6 両対応)、operator_ip_allowlists.cidr
適用範囲	エッジ Cloudflare Worker(/webhooks/* を除く全パス)
KV キャッシュ	operator-ip-allowlist:<operator_id>、TTL 60 秒
変更操作	operator_ip.grant / operator_ip.revoke(5y)、変更時に KV Invalidate
緊急バイパス	紙ベース回復コードで master_key.emergency_bypass 監査経路のみ (MVP)

2.4.7 3.7 運営者アカウント・ライフサイクル (FR-225)

状態	説明	遷移条件
invited	招待メール送信済、未受諾	既存運営者が招待発行

状態	説明	遷移条件
active	通常稼働	operator.accept 後
disabled	一時無効化	operator.disable(理由付き)
revoked	解任(永続無効)	operator.disable 後 + 90 日経過

すべての遷移は `operator_accounts.lifecycle_state` 列で管理し、対応 action コード (`operator.invite` / `operator.accept` / `operator.disable` / `operator.subrole.grant` / `operator.subrole.revoke`、すべて 5y) を `audit_logs` に記録する。

2.4.8 3.8 ミドルウェア実装ガイドライン

`admin/src/admin-console/middleware/` 配下に次の順序でミドルウェアを連結する:

1. `ip-allowlist.ts` —— エッジで判定済の場合はパススルー、Worker からの直接呼び出しに備えて再判定
2. `session.ts` —— KV → D1 フォールバック、`mfaVerifiedAt` を `Context` に注入
3. `mfa.ts` —— `mfaVerifiedAt` 必須、ログインフロー専用パスは除外
4. `csrf.ts` —— `__Host-` プレフィックス Cookie + `X-CSRF-Token` 一致
5. `ticket-id.ts` —— `X-Op-Ticket-Id` ヘッダ必須 (クリティカル操作のみ、`^[A-Za-z0-9_\-]{1,64}$`)
6. `reauth.ts` —— `re-auth:<sid>` 存在 + `now - reauthenticatedAt ≤ 5min`
7. `four-eyes.ts` —— `feature:hard-gate:<action>` 真 → `X-Approval-Id` 必須、未付与で 403

2.4.9 3.9 HKDF info 値リスト (認証関連、★TH-12)

info 値	用途	キー長	派生元
operator-session	セッショントークン HMAC 検証	32 バイト	Secrets Store マスター
mfa-secret	MFA TOTP シークレット暗号化	32 バイト	同上
internal-api	mTLS + JWT 署名	32 バイト	同上 (連携 IF #1~#12 共通)
audit-chain	ハッシュチェーン HMAC(DD03 参照)	32 バイト	同上
audit-export	監査エクスポート HMAC 署名 (D-17)	32 バイト	同上
pii-encryption	PII 暗号化 (メイン主管、参照のみ)	32 バイト	同上

完全な HKDF info 値リストは [基本設計 / セキュリティ設計 §6](#) を正本とする。

2.4.10 3.10 主要エラーコード

エラー ID	HTTP	発生段	説明
E-OP-AUTH-001	401	段 2	UNAUTHENTICATED(セッションなし / 失効)
E-OP-AUTH-002	401	ログイン	INVALID_CREDENTIALS
E-OP-AUTH-003	403	段 3	FORBIDDEN_MFA_REQUIRED
E-OP-AUTH-004	403	段 4	FORBIDDEN_ROLE
E-OP-AUTH-005	403	段 5	RE_AUTH_REQUIRED
E-OP-AUTH-006	403	段 1	IP_NOT_ALLOWED
E-OP-AUTH-007	429	ログイン	LOCKED_OUT(15 分)
E-OP-4EYES-001	403	段 6	FORBIDDEN_HARD_GATE

完全な E-OP-AUTH-* 一覧は [基本設計 / エラー設計 §4](#) を正本とする。

2.5 4. 関連設計

種別	参照先
要件	../01_ 要件定義/index.md
基本設計	../02_ 基本設計/index.md

種別	参照先
認証・認可設計(正本)	../02_基本設計/08_認証・認可設計.md
権限設計(正本)	../02_基本設計/04_権限設計.md
セキュリティ設計(正本)	../02_基本設計/09_セキュリティ設計.md
関連 DD	DD02_4-eyes 承認フロー.md / DD03_監査ハッシュチェーン.md
運用設計	../04_運用設計/index.md
将来対応	../05_future/index.md

2.6 5. テスト観点

2.6.1 5.1 ユニットテスト

- ・ IP 許可リスト判定 (IPv4 / IPv6 / CIDR 境界 / KV キャッシュヒット・ミス)
- ・ Argon2id パスワード検証 (成功 / 失敗 / タイミング攻撃耐性)
- ・ TOTP 検証 (時刻スキュー ±30 秒許容 / 再使用拒否)
- ・ セッション TTL 計算 (8h 滑走 + 24h 絶対上限)
- ・ 再認証ウィンドウ判定 ($\text{now} - \text{reauthenticatedAt} \leq 5\text{min}$ 、consumed=true で失効)
- ・ HKDF info 値別派生 の確定的計算

2.6.2 5.2 結合テスト (Miniflare)

- ・ ログイン → MFA → セッション発行 → 任意 API 呼出
- ・ ロックアウト (5 回失敗 → 15 分)
- ・ 同時セッション 3 → 4 件目で最古 revoke
- ・ IP 変更時のセッション継続 (セッショントークンは IP に縛らない、再認証は要)
- ・ CSRF トークン不一致で 403

2.6.3 5.3 E2E テスト (Playwright)

テスト ID	シナリオ
e2e-auth-001	ログイン正常 → ホーム表示
e2e-auth-002	未許可 IP → 403 Forbidden(エッジ)
e2e-auth-003	MFA 未セットアップ → SCR-AUTH-M1 強制表示
e2e-auth-004	クリティカル操作で再認証要 → モーダル表示 → 認証後実行
e2e-auth-005	ロックアウト 15 分計測

2.6.4 5.4 受入条件マッピング

AC	検証手段
AC-037(運営者 MFA + 再認証)	認証テスト全件 + E2E

2.7 6. 未確定事項・確認事項

確認事項 ID	確認内容	優先度	ステータス
-	v1.0 リリース時点で全項目確定済み	低	確認済

\newpage

3 DD02: 4-eyes 承認フロー (運営者システム)

3.1 0. 文書情報

項目	内容
文書名	DD02: 4-eyes 承認フロー (運営者システム)
詳細設計 ID	DD02
対象システム	FAQ AI ウィジェット SaaS / 運営者システム
関連機能 ID	FR-226, D-12
作成日	2026-05-17
版数	v1.0
ステータス	承認済

3.2 1. 対象範囲

種別	ID	名称
機能	FR-226	4-eyes 申請承認 (MVP 3 ハードゲート + 7 承認ログ)
画面	SCR-APPROVALS	承認待ち一覧
画面	SCR-APPROVALS-M1	申請モーダル (CMP-E)
画面	SCR-APPROVALS-M2	承認モーダル (CMP-F)
API	POST /approvals	申請作成
API	POST /approvals/{id}/start-review	レビュー 開始
API	POST /approvals/{id}/approve	承認
API	POST /approvals/{id}/reject	却下
API	POST /approvals/{id}/withdraw	自己取下げ
API	POST /approvals/{id}/execute	実行
テーブル	operator_approvals	申請・承認状態管理

3.3 2. 収録ロジック・対応章

元章	元タイトル	概要
§ 5 SCR-APPROVALS	画面詳細 (参照)	承認待ち一覧 + 申請モーダル + 承認モーダル
§ 6.4	4-eyes 機能詳細	申請 → 承認 → 実行のロジック
§ 7.4	4-eyes API	OpenAPI(申請 / start-review / approve / reject / withdraw)
付録 C.X	4-eyes フロー E2E テストシナリオ	5 シナリオ (happy / self-block / expire / rollback / hash)
付録 E	4-eyes 操作詳細 (MVP)	対象 10 操作 + 申請/承認モーダル ASCII レイアウト

3.4 3. 詳細設計本文

3.4.1 3.1 4-eyes 対象 10 操作

#	action_code	MVP ハードゲート	MVP 承認ログ
1	owner.physical_delete	必須	-
2	ai_parameter.update	必須	-
3	master_key.rotate	必須	-
4	owner.suspend	-	単独 + 事後監査
5	owner.restore	-	対象
6	pricing.update	-	対象
7	rate_limit.override / budget_limit.override	-	対象
8	widget.force_stop	-	対象
9	feature.hard_gate.toggle	-	対象
10	owner.restore_data(FR-204、削除データ復元)	-	対象

ハードゲート 3 操作は **X-Approval-Id** ヘッダ必須 + 承認済 (**approved** 状態) 申請に紐付かない限り **403 FORBIDDEN_HARD_GATE** を返す。承認ログのみ 7 操作は単独実行可能だが、**audit_logs** 上に必ず 1 件の **operator_approval.*** 履歴を残す。

3.4.2 3.2 状態遷移 (operator_approvals.state)

From	To	トリガー	ガード
-	requested	POST /approvals	expires_at = +72h
requested	reviewing	start-review	requested_by ≠ reviewer
reviewing	approved	approve	DB CHECK 自己承認禁止
reviewing	rejected	reject	requested_by ≠ rejected_by、コメント必須
requested	withdrawn	withdraw(申請者本人)	withdrawn_by == requested_by(自己取下げ)
requested/reviewing	expired	72h 経過	バッチ
approved	executed	execute	now < approved_at + 72h、payload_hash 一致
approved	expired	72h 経過	バッチ

詳細状態遷移は [DD11_状態遷移詳細.md](#) § B.3 を参照。

3.4.3 3.3 payload_hash 検証

項目	仕様
計算式	sha256(canonical_json(payload))(RFC 8785 JSON Canonicalization)
申請時	申請者送信の payload を正規化し、operator_approvals.payload_hash に保管
承認時	承認者が確認した payloadHash を送信 (改ざん検知用)、DB 値と一致しなければ 409
実行時	実行 API 呼出時の payload を再正規化し、operator_approvals.payload_hash と一致確認。不一致なら 403

payload_hash 不一致は承認 → 実行間で payload を改ざんしようとする攻撃を検出する仕組み。承認モードでは ✓ payload_hash: sha256:... (検証 OK) を表示する。

3.4.4 3.4 自己承認禁止

実装層を 4 重で防御する:

1. **アプリケーション層**: start-review / approve で requestedBy === currentOperator を拒否 (403)
2. **DB CHECK 制約**: CHECK (requested_by != approved_by) を operator_approvals テーブルに付与
3. **監査チェック**: 監査ログ集計で同一運営者の request + approve ペアを検出する日次バッチ
4. **承認モード UI ガード**: SelfApprovalGuard コンポーネントでボタン無効化

3.4.5 3.5 4-eyes 申請モデル (CMP-E、★TH-5)

```

+-----+
| ★ 申請モデル: AI 推論パラメータ更新 |
+-----+
| action_code: ai_parameter.update |
| 申請者: 田中 (現在ログイン中) |
| 申請理由 【必須、最大 1000 文字】 |
| [ ] |
| 対応チケット ID 【必須、最大 64 文字】 |
| TKT-[ ] |
| payload プレビュー (編集不可、整形済 JSON) |
| { |
|   "scope": "owner",

```

<pre> "scopeId": "01J9...", "confidenceThreshold": 0.65, ... } </pre>
payload_hash: sha256:abc123def... 承認 TTL: 72 時間
[キャンセル] [申請する] →

3.4.6 3.6 4-eyes 承認モデル (CMP-F、★TH-5)

★ 承認モデル: AI 推論パラメータ更新
申請者: 田中 (2026-05-12 10:00) あなた: 佐藤 (承認者) 申請 ID: 01J9V0... 申請から: 1 時間 15 分前 期限: 70 時間 45 分後
申請理由: 契約 acme 向け関連度しきい値を緩和 #TKT-1234
payload プレビュー (整形済 JSON、改ざんチェック済): <pre> { "scope": "owner", "scopeId": "01J9...", "confidenceThreshold": 0.65, ... } </pre>
✓ payload_hash: sha256:abc123def... (検証 OK)
コメント [任意]
[保留] [却下 (コメント必須)] [承認]

3.4.7 3.7 4-eyes API OpenAPI 抜粋

```

paths:
  /approvals:
    post:
      summary: 4-eyes 申請作成
      parameters:
        - $ref: "#/components/parameters/X0pTicketId"
        - $ref: "#/components/parameters/IdempotencyKey"
        - $ref: "#/components/parameters/XCsrftoken"
      requestBody:
        required: true
        content:

```

```

    application/json:
      schema:
        type: object
        required: [actionCode, payload, reason]
        properties:
          actionCode: { type: string }
          payload: { type: object }
          reason: { type: string, maxLength: 1000 }
  responses:
    "201":
      description: 申請完了
      content:
        application/json:
          schema:
            type: object
            properties:
              approvalId: { type: string }
              expiresAt: { type: string, format: date-time }
              payloadHash: { type: string }
    "409":
      content:
        application/problem+json:
          schema: { $ref: "#/components/schemas/Problem" }

/approvals/{id}/start-review:
  post:
    summary: 4-eyes 承認確認開始
    parameters:
      - in: path
        name: id
        required: true
        schema: { type: string }
      - $ref: "#/components/parameters/XOpTicketId"
      - $ref: "#/components/parameters/IdempotencyKey"
    responses:
      "200": { description: reviewing 遷移完了 }
      "403":
        content: { application/problem+json: { schema: { $ref: "#/components/schemas/Problem" } } }

/approvals/{id}/approve:
  post:
    summary: 4-eyes 承認
    parameters:
      - in: path
        name: id
        required: true
        schema: { type: string }
      - $ref: "#/components/parameters/XOpTicketId"
      - $ref: "#/components/parameters/IdempotencyKey"
      - $ref: "#/components/parameters/XCsrftoken"
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required: [payloadHash]
            properties:
              payloadHash: { type: string }
              comment: { type: string }
    responses:
      "200": { description: approved 遷移完了 }

```

```

    "403":
      content: { application/problem+json: { schema: { $ref: "#/components/schemas/Problem" } } }

/approvals/{id}/reject:
  post:
    summary: 4-eyes 却下 (別運営者)
    parameters:
      - in: path
        name: id
        required: true
        schema: { type: string }
      - $ref: "#/components/parameters/XOpTicketId"
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required: [comment]
            properties:
              comment: { type: string }
    responses:
      "200": { description: rejected 遷移完了 }
      "403":
        content: { application/problem+json: { schema: { $ref: "#/components/schemas/Problem" } } }

/approvals/{id}/withdraw:
  post:
    summary: 4-eyes 自己取下げ (申請者本人)
    parameters:
      - in: path
        name: id
        required: true
        schema: { type: string }
      - $ref: "#/components/parameters/XOpTicketId"
    responses:
      "200": { description: withdrawn 遷移完了 }
      "403":
        content: { application/problem+json: { schema: { $ref: "#/components/schemas/Problem" } } }
      "409":
        content: { application/problem+json: { schema: { $ref: "#/components/schemas/Problem" } } }

```

完全な API スキーマは [基本設計 / API 設計](#) §4 を正本とする。

3.4.8 3.8 楽観ロック・同時実行制御

- `operator_approvals` テーブルに `version` 列 (INTEGER、初期値 0) を保持
- 状態遷移 UPDATE は `WHERE id=? AND version=?` で実行し、`version = version + 1` を SET
- 競合 (同時に 2 名が承認ボタン押下) は影響行数 = 0 で検出、後勝ちを 409 で拒否
- 負荷試験 A4(100 申請を 10 秒で集中、別運営者 100 並列承認) で楽観ロック動作を検証

3.4.9 3.9 期限切れ自動バッチ

72 時間経過で `requested / reviewing / approved` 状態を一斉 `expired` に遷移させる:

```

function approvalExpiry():
  candidates = D1.query("SELECT id, state FROM operator_approvals
                        WHERE state IN ('requested','reviewing','approved')
                        AND expires_at <= ?", now)
  for c in candidates:
    D1.exec("UPDATE operator_approvals
            SET state='expired', expired_at=?, version=version+1

```

```
WHERE id=? AND state=?", now, c.id, c.state)
audit_logs(operator_approval.expire, 5y, payload={approvalId: c.id})
```

バッチは1時間ごとに別 cron Worker(または **AnnouncementSchedulerWorker** 内のサブタスク)で実行する。

3.4.10 3.10 4-eyes フロー E2E テストシナリオ (付録 C.X)

申請 → 承認 → 実行 → 失敗ロールバックの完全フロー E2E を別途定義し、SCR 個別 E2E と分離管理する:

シナリオ ID	内容	対象 action_code 例	テストファイル
e2e-4eyes-happy-001	正常: 申請 → start_review → 承認 → 実行 → 監査ログ 4 件記録	ai_parameter.update	apps/admin-console/e2e/4eyes/happy-path.spec.ts
e2e-4eyes-self-block-001	自己承認拒否: requester==approver で 403	ai_parameter.update	apps/admin-console/e2e/4eyes/self-approve-block.spec.ts
e2e-4eyes-expire-001	72h 期限切れ自動 expire	owner.physical_delete	apps/admin-console/e2e/4eyes/expire-flow.spec.ts
e2e-4eyes-rollback-001	実行失敗 → ロールバック → 監査ログ execute_failed 記録	owner.restore	apps/admin-console/e2e/4eyes/execute-failure-rollback.spec.ts
e2e-4eyes-hash-001	payload_hash 改ざん検出(承認時に Hash 不一致で却下)	master_key.rotate	apps/admin-console/e2e/4eyes/payload-tampering.spec.ts

3.5 4. 関連設計

種別	参照先
要件	../01_要件定義/index.md
基本設計	../02_基本設計/index.md
API 設計 (正本)	../02_基本設計/02_API 設計.md
画面設計 (正本)	../02_基本設計/01_画面設計.md
認証・認可設計 (正本)	../02_基本設計/08_認証・認可設計.md
関連 DD	DD01_運営者認証・6 段認可.md / DD03_監査ハッシュチェーン.md / DD11_状態遷移詳細.md
運用設計	../04_運用設計/index.md
将来対応	../05_future/index.md

3.6 5. テスト観点

3.6.1 5.1 ユニットテスト

- **payload_hash** 計算 (RFC 8785 canonical_json + sha256)
- 自己承認禁止 (アプリケーション層)
- 状態遷移ガード (**requested** → **reviewing** → **approved** → **executed** のみ正常パス、その他は遷移不可)
- 楽観ロック競合検出 (影響行数 = 0)

3.6.2 5.2 結合テスト (Miniflare)

- 申請 → 承認 → 実行の完全パスで監査ログ 4 件 (**request** / **start_review** / **approve** / **execute**) が記録される
- 自己承認試行で 403 + 監査ログ未記録
- 72h 経過 expiry バッチで **expired** 遷移 + 監査記録
- payload_hash 不一致での承認・実行拒否

3.6.3 5.3 E2E テスト (Playwright)

付録 C.X の 5 シナリオすべて。

3.6.4 5.4 負荷試験

シナリオ	構成	合格基準
(A3) 4-eyes 承認バースト	同時申請 50 件 / 承認 50 件	楽観ロック・状態遷移整合性、レース条件で重複承認 = 0 件
(A4) 4-eyes 同時申請	100 申請を 10 秒で集中、別運営者から 100 並列承認	レース条件で重複承認 = 0 件、申請 → 承認 → 実行が単調進行

3.6.5 5.5 受入条件マッピング

AC	検証手段
AC-038(運営者操作監査 + 通知)	E2E + 月次監査 (関連)

3.7 6. 未確定事項・確認事項

確認事項 ID	確認内容	優先度	ステータス
-	v1.0 リリース時点で全項目確定済み	低	確認済

\newpage

4 DD03: 監査ハッシュチェーン (運営者システム)

4.1 0. 文書情報

項目	内容
文書名	DD03: 監査ハッシュチェーン (運営者システム)
詳細設計 ID	DD03
対象システム	FAQ AI ウィジェット SaaS / 運営者システム
関連機能 ID	FR-229, FR-230, FR-231, FR-232, NFR-306, NFR-602, NFR-705, D-04, D-08, D-17
作成日	2026-05-17
版数	v1.0
ステータス	承認済

4.2 1. 対象範囲

種別	ID	名称
機能	FR-229	監査ログ全件記録
機能	FR-230	監査ログ検索
機能	FR-231	監査ログハッシュチェーン
機能	FR-232	監査ログエクスポート (HMAC 署名)
画面	SCR-096	監査ログ閲覧・エクスポート
API	GET /audit-logs	検索
API	POST /audit-logs/exports	エクスポート開始
API	GET /audit-logs/exports/{job_id}	エクスポート進捗
テーブル	audit_logs	ハッシュチェーン主管 (運営者主管)

4.3 2. 収録ロジック・対応章

元章	元タイトル	概要
§6 監査ログ機能詳細 (参照)	ハッシュチェーン書込・検索・エクスポート	記録 → 検索 → エクスポート

元章	元タイトル	概要
§12 ハッシュチェーン詳細 (参照)	チェーン構造 + 管理	prev_hash + record_hash + HKDF info=audit-chain
§14.2.2	AuditChainVerifierWorker 擬似コード	日次 02:00 JST 全件再計算
§16 構造化ログ (監査関連)	監査関連の構造化ログ要件	actor_id / action / trace_id
付録 F	3 区分保持の物理対応	action_code × retention_class 全件マップ

4.4 3. 詳細設計本文

4.4.1 3.1 監査ハッシュチェーン構造

各 audit_logs レコードは前レコードの record_hash を prev_hash 列に保持し、暗号学的に連結する。

```
record_hash = HMAC-SHA256(
    key = HKDF(masterKey, info="audit-chain"),
    message = prev_hash || canonical_json({
        actor_id, action, target_type, target_id,
        payload_hash, retention_class, occurred_at
    })
)
```

項目	仕様
初期値 prev_hash	"0" * 64(全 0、ジェネシス)
HMAC アルゴリズム	HMAC-SHA256
HKDF info	audit-chain
record_hash 長	64 文字 (16 進エンコード)
順序保証	(occurred_at, id) の昇順、ULID 採番で時系列単調保証
改ざん検知	1 レコードを書換えると以降全レコードの record_hash が不一致

4.4.2 3.2 書込トランザクション (D-04)

監査ログ INSERT は次のトランザクションで実行する:

```
BEGIN;
-- 直前レコードを排他取得
SELECT id, record_hash FROM audit_logs
ORDER BY occurred_at DESC, id DESC LIMIT 1
FOR UPDATE;
-- 新レコード INSERT
INSERT INTO audit_logs (
    id, actor_id, action, target_type, target_id,
    payload, payload_hash, retention_class,
    prev_hash, record_hash, occurred_at, trace_id
) VALUES (?, ..., ?, computed_record_hash, ?, ?);
COMMIT;
```

D1 のトランザクション境界制約 (現状 batch API ベース) を踏まえ、実装では `db.batch([...])` で SELECT + INSERT を 1 つの論理単位として送出する。並行 INSERT で同一 prev_hash を使う競合は影響行数 = 0 で検出し、リトライ (最大 3 回)+ それでも失敗時は `audit.write.conflict` 監査 + 運営者 high alert。

4.4.3 3.3 日次ハッシュチェーン検証 (AuditChainVerifierWorker、D-04)

```
function auditChainVerify():
    verify_date = today
    runId = ULID()
    audit_logs(audit.chain.verify.start, 5y, payload={verify_date, run_id: runId})

    prev_hash = "0" * 64
    mismatch_count = 0
    cursor = null
```

```

do:
    rows = D1.query("SELECT id, prev_hash, record_hash, ... FROM audit_logs WHERE id > ? OR
    for r in rows:
        expected = sha256(prev_hash + canonical_json(r.{actor_id, action, ..., retention_cl
        if expected != r.record_hash:
            audit_logs(audit.chain.verify.fail, 5y, payload={row_id: r.id, expected, actual
            notifyOperator(high, "Audit chain mismatch at row=${r.id}")
            mismatch_count++
        prev_hash = r.record_hash
        cursor = r.id
while rows.length > 0

```

```

audit_logs(audit.chain.verify, 5y, payload={verify_date, run_id: runId, mismatch_count, tot

```

cron 設定: 0 17 * * *(UTC、毎日 17:00 UTC = 翌日 02:00 JST)、Cron 詳細は [DD08_Cron 実装.md](#) 参照。

所要時間目標: 1000 万行で 3600s 以内 (KV: monitoring: thresholds: chain-verify-duration で監視、超過時に分割検証へ切替)。

4.4.4 3.4 不一致検出時の通知ポリシー

検出件数	通知レベル	担当
1 件以上	high alert(運営者 inbox + メール強制送信)	当番運営者
累積 10 件以上 / 日	critical alert(オーナー + users:manage 保持メンバー全員、強制送信)	セキュリティ責任者

通知契機は [基本設計 / メッセージ一覧 §8](#) を正本とする。

4.4.5 3.5 監査ログ検索 API

```

/audit-logs:
  get:
    summary: 監査ログ検索
    parameters:
      - in: query
        name: action
        schema: { type: string }
      - in: query
        name: from
        schema: { type: string, format: date-time }
      - in: query
        name: to
        schema: { type: string, format: date-time }
      - in: query
        name: cursor
        schema: { type: string }
      - in: query
        name: limit
        schema: { type: integer, default: 100, maximum: 200 }
    responses:
      "200":
        content:
          application/json:
            schema:
              type: object
              properties:
                items:
                  type: array
                  items:
                    type: object
                nextCursor: { type: string, nullable: true }

```

検索自体も監査される (`audit.search`, 1y, {`operatorId`, `filter`, `hitCount`, `ticketId`})——APPI 30 条準拠。

性能目標: 100 万件以下で p95 $\leq$ 1000ms(KV: `monitoring:thresholds:audit-search-p95`)。インデックス: (`action`, `occurred_at`) 複合 + (`actor_id`, `occurred_at`) 複合。

4.4.6 3.6 監査エクスポート (HMAC 署名、D-17)

エクスポートは非同期ジョブ (`audit-export:<job_id>` KV TTL 24h) で実行する:

ステップ	処理
1	POST /audit-logs/exports(format: csv
2	Worker が D1 クエリ + R2 マルチパート PUT(10 万行/ファイル、5 並列上限)
3	全ファイル完了後 HMAC 署名 (HKDF info=audit-export、SHA-256) を .sig で生成
4	GET /audit-logs/exports/{job_id} で署名付き URL を取得 (R2 Pre-signed、TTL 24h)
5	エクスポート完了で audit.export(5y、{exportId, format, rows}) 記録

外部検証ツール: `admin/scripts/audit-export-verify.ts` で .sig + マスター 公開 OK の HKDF 派生 から HMAC 再計算 → 完全性確認。運営者は別保管した 情報で第三者検証可能。

HMAC 計算:

```
hmacKey = HKDF(masterKey, info="audit-export")
signature = HMAC-SHA256(
  key = hmacKey,
  message = sha256(file_content) || jobId || createdAt
)
```

性能目標: 10 万行/ファイルで $\leq$ 60s(KV: `monitoring:thresholds:audit-export-duration-p95`)。

4.4.7 3.7 3 区分保持 (D-08、付録 F)

retention_class	期間	物理削除タイミング	主な対象 action
general(1y)	365 日	RetentionPurgeWorker(日次 03:00 JST)	account.login / inbox.read / announcement.preview / pii_fp_report.create / audit.search
operator_high_priv(25y)	同上	同上	owner.* / operator.* / operator_approval.* / mfa.* / ai_parameter.update / master_key.rotate 等
billing(7y)	2555 日	同上	billing.* / stripe.event.processed / webhook.receive

物理削除前に R2 アーカイブ (`audit-archive/<retention_class>/<year>/<month>.tar.gz`) へ書出 → DELETE。完全な action_code × retention_class マップは [DD09_監査 action コード.md](#) を参照。

4.4.8 3.8 accounts_retired 永久保持

退会・物理削除されたオーナーは `accounts_retired` テーブルに移し、**法令対応のため永久保持**。audit_logs の target_id 参照を維持し、復元時のリンクを切らない。詳細は [基本設計 / セキュリティ設計](#) を参照。

4.4.9 3.9 構造化ログ (監査関連)

監査 INSERT 時に追加で構造化ログを出力する (Logpush / Logflare 等の集約に乗せる)。フォーマット:

```
{
  "level": "info",
  "worker": "AdminConsoleWorker",
  "request_id": "<cf-ray>",
  "trace_id": "01J...",
  "operator_id": "01J...",
  "action": "owner.suspend",
  "target_type": "owner",
  "target_id": "01J...",
```

```

    "retention_class": "operator_high_priv",
    "occurred_at": "2026-05-12T10:00:00Z",
    "ticket_id": "TKT-1234",
    "approval_id": "01J..."
  }

```

構造化ログの詳細スキーマは [DD12_運営者用ログ・テスト.md](#) §16 を参照。

4.4.10 3.10 主要エラーコード

エラー ID	HTTP	説明
E-OP-AUDIT-001	500	INTERNAL_CHAIN_INSERT_FAILED(リトライ後失敗)
E-OP-AUDIT-002	409	CHAIN_INSERT_CONFLICT(並行 INSERT 競合)
E-OP-AUDIT-003	400	EXPORT_FILTER_INVALID
E-OP-AUDIT-004	503	EXPORT_QUEUE_FULL(5 並列上限)

完全な E-OP-AUDIT-* 一覧は [基本設計 / エラー設計](#) を正本とする。

4.5 4. 関連設計

種別	参照先
要件	../01_要件定義/index.md
基本設計	../02_基本設計/index.md
セキュリティ設計(正本)	../02_基本設計/09_セキュリティ設計.md
テーブル設計(正本)	../02_基本設計/03_テーブル設計.md
関連 DD	DD08_Cron 実装.md / DD09_監査 action コード.md / DD12_運営者用ログ・テスト.md
運用設計	../04_運用設計/index.md
将来対応	../05_future/index.md

4.6 5. テスト観点

4.6.1 5.1 ユニットテスト

- HMAC-SHA256 + HKDF info=**audit-chain** 派生 の確定的計算
- **canonical_json** 正規化(キー順序ソート、Unicode NFC、数値表記)
- 単一 INSERT で **prev_hash + record_hash** チェーン整合
- 改ざんシミュレーション(1レコード書換 → 検証で不一致検出)
- HMAC 監査エクスポート署名計算 + 外部ツール検証

4.6.2 5.2 結合テスト (Miniflare)

- 1000 件 INSERT 後にチェーン全件検証 = 不一致 0
- 並行 INSERT で **prev_hash** 競合発生 → リトライ後成功
- 監査検索カーソルページング動作
- エクスポート(1000 行 × 3 ファイル)の HMAC 署名生成 + 検証

4.6.3 5.3 E2E テスト (Playwright)

テスト ID	シナリオ
e2e-scr096-001	検索 → 詳細 → エクスポート → HMAC 検証ツールでの完全性確認
e2e-audit-chain-001	チェーン検証バッチ実行 → 不一致なし
e2e-audit-chain-002	意図的に 1 レコード書換 → 検証バッチで検出 + 通知

4.6.4 5.4 負荷試験

シナリオ	構成	合格基準
(A1) 監査ログ大量検索	200 契約 × 5 年分 (約 550 万行) を 100 並列検索	p95 ☒ 1000ms
(A2) 監査ログ大量エクスポート	10 万行を 5 並列同時起動	全ジョブ完了 ☒ 90s、メモリ上限 128MB ヒット率 0%

4.6.5 5.5 受入条件マッピング

AC	検証手段
AC-038(運営者操作監査 + 通知)	E2E + 月次監査

4.7 6. 未確定事項・確認事項

確認事項 ID	確認内容	優先度	ステータス
-	v1.0 リリース時点で全項目確定済み	低	確認済

\newpage

5 DD04: Stripe Webhook 一次受信 (運営者システム)

5.1 0. 文書情報

項目	内容
文書名	DD04: Stripe Webhook 一次受信 (運営者システム)
詳細設計 ID	DD04
対象システム	FAQ AI ウィジェット SaaS / 運営者システム
関連機能 ID	FR-302, NFR-808, NFR-809, AC-041, D-06, D-10, TH-8, TH-9
作成日	2026-05-17
版数	v1.0
ステータス	承認済

5.2 1. 対象範囲

種別	ID	名称
機能	FR-302	Webhook 受信 (Stripe / Resend)
画面	SCR-097	Webhook リプレイ・DLQ 操作
画面	SCR-099	Webhook ペイロード差分検出レビュー
API	POST /webhooks/stripe	Stripe Webhook 一次受信 (IP 制限適用外)
API	POST /webhooks/resend	Resend Webhook 一次受信 (同上)
API	POST /webhooks/replay	手動リプレイ
API	POST /webhook-payload-diffs/{id}/reprocess	差分検出後の再処理
API	POST /webhook-payload-diffs/{id}/dismiss	差分検出の却下
テーブル	webhook_events	受信イベント主台帳
テーブル	webhook_payload_diffs	ペイロード差分検出記録
テーブル	dlq_replay_log	DLQ リプレイ履歴
連携 IF	#10	課金 Webhook 一次受信 → メイン転送 (本書主管)

5.3 2. 収録ロジック・対応章

元章	元タイトル	概要
§10 連携 IF #10 詳細	Stripe Webhook 一次受信 → メイン転送	唯一の受信先 (D-10)
§6 webhook 機能 (参照)	署名検証 / 冪等性 / 差分検出 / DLQ	一次受信ロジック
§7 webhook API	手動リプレイ + 差分 reprocess/dismiss	API スキーマ
付録 B.2	webhook_events.state	12 状態遷移
付録 H	Webhook 除外フィールド完全リスト (★TH-8)	正規化対象除外

5.4 3. 詳細設計本文

5.4.1 3.1 受信エンドポイント (D-10)

エンドポイント	用途	IP 制限	署名検証
POST /webhooks/stripe	Stripe イベント受信 (唯一の受信先)	適用外	Stripe-Signature HMAC-SHA256
POST /webhooks/resend	Resend パウンス・配信通知	適用外	Resend Signature(HMAC)

メイン側は /webhooks/stripe を一切受けない (D-10)。本書側のみが Stripe との接続点を持つ。

5.4.2 3.2 受信処理フロー

```
sequenceDiagram
    participant Stripe
    participant Worker as BillingWebhookWorker
    participant KV
    participant D1
    participant R2
    participant Main as Main Worker

    Stripe->>Worker: POST /webhooks/stripe + Stripe-Signature
    Worker->>Worker: HMAC-SHA256 署名検証
    alt 署名 NG
        Worker->>D1: audit_logs(webhook.signature.invalid, 5y)
        Worker-->>Stripe: 401
    else 署名 OK
        Worker->>D1: INSERT webhook_events (state='received', payload_hash 計算済)
        Worker->>KV: GET webhook:idempotency:<event_id>
        alt 既処理 + payload_hash 一致
            Worker->>D1: UPDATE state='duplicate_skipped_hash_match'
            Worker-->>Stripe: 200(冪等成功)
        else 既処理 + payload_hash 不一致
            Worker->>D1: INSERT webhook_payload_diffs (state='detected')
            Worker->>D1: UPDATE webhook_events state='duplicate_diff_detected_high_alert'
            Worker->>R2: PUT webhook-payload-snapshots/<event_id>-<ts>.json
            Worker->>Main: 通知 (SCR-099 へ運営者誘導)
            Worker-->>Stripe: 200(差分は SCR-099 レビュー待ち)
        else 未処理
            Worker->>D1: UPDATE state='processing'
            Worker->>R2: PUT dlq-stripe-events/<event_id>.json
            Worker->>Main: 連携 IF #10 (mTLS + JWT)
            alt メイン 200
                Worker->>D1: UPDATE state='succeeded'
                Worker->>D1: audit_logs(stripe.event.processed, 7y)
                Worker-->>Stripe: 200
            else メイン 5xx/Timeout
                Worker->>D1: UPDATE state='failed', next_retry_at=now+1min
                Worker-->>Stripe: 200(後続リトライ)
            end
        end
    end
```

```

        end
    end
end

```

5.4.3 3.3 状態遷移 (webhook_events.state)

From	To	トリガー	副作用
-	received	POST 受信	row INSERT
received	verifying_signature	即時	-
verifying_signature	rejected	HMAC NG	401 + high alert
verifying_signature	checking_idempotency	HMAC OK	payload_hash 計算
checking_idempotency	processing	未処理	INSERT + R2 退避
checking_idempotency	duplicate_skipped_hash_match	既処理 + 一致	200 + ログ
checking_idempotency	duplicate_diff_detected_high_alert	既処理 + 不一致	webhook_payload_diffs INSERT、SCR-099 待ち
processing	succeeded	IF #10 200	audit:stripe.event.processed(7y)
processing	failed	5xx/Timeout	DLQ 投入
failed	dlq_retrying	DLQAutoBackoffWork指数 BO	
dlq_retrying	succeeded	再試行成功	-
dlq_retrying	dlq_manual_replay	1h 経過	運営者 high
dlq_manual_replay	succeeded	SCR-097 リプレイ	audit:webhook.replay(5y)
dlq_manual_replay	dlq_archived	30 日経過	リプレイ不可

5.4.4 3.4 幂等性ストア

キー	値	TTL	用途
webhook:idempotency:<event_id>	{payloadHash, state, processedAt}	30 日	D1 webhook_events のキャッシュ

幂等性判定の正本は **webhook_events.event_id** UNIQUE 制約 (D1)。KV は高速判定のためのキャッシュ層であり、KV ミス時は D1 にフォールバック。

5.4.5 3.5 ペイロード正規化 + 差分検出 (D-06)

5.4.5.1 3.5.1 正規化アルゴリズム

```

function canonical_json(payload, api_version):
    excludeFields = getExcludeFields(api_version)
    filtered = removeFields(payload, excludeFields)
    return RFC8785_canonicalize(filtered) // キーソート + Unicode NFC + 数値正規化

```

```

function payload_hash(payload, api_version):
    return sha256(canonical_json(payload, api_version))

```

5.4.5.2 3.5.2 全バージョン共通除外フィールド (ルートレベル)

```

created
request.id
request.idempotency_key
idempotency_key_resent
livemode
api_version
pending_webhooks

```

5.4.5.3 3.5.3 全バージョン共通除外 (data.object レベル)

```

data.object.created
data.object.updated_at
data.object.test_clock

```

```
data.object.metadata.test_*
data.object.metadata._stripe_internal_*
data.previous_attributes
```

5.4.5.4 3.5.4 全バージョン共通除外 (配列再帰)

```
*,created
*,updated_at
```

5.4.5.5 3.5.5 Stripe API バージョン 2024-06-20 固有 (現時点で追加除外なし)

5.4.6 3.6 複数バージョン併存運用ルール

Stripe ダッシュボードで API バージョンをアップグレードすると、同一契約への Webhook が新旧バージョンを跨いで配信される。この前提で本書では以下のルールを定める:

項目	仕様
バージョン保管	webhook_events.stripe_api_version 列に受信時の api_version を保管
ハッシュ計算	EXCLUDE_FIELDS_BY_VERSION[stripe_api_version] を選択して canonical_json 呼出
マップ未登録時	EXCLUDE_FIELDS_BY_VERSION["default"] = 共通リストにフォールバックし、運営者 inbox normal(webhook.unknown_api_version、5y) で通知
同 event_id 異 API バージョン	既処理 event_id + api_version 不一致 → 通常フローでは webhook_payload_diffs に記録するが、stripe_api_version_mismatch フラグを立てて SCR-099 で見分け可能にする
廃止予告	Stripe が API バージョンを EOL 通知してから 180 日以内に該当バージョンのサポートを終了。本書改訂で EXCLUDE_FIELDS_BY_VERSION から削除
サポート対象範囲	同時にサポートするバージョン数は 最大 2(現行 + 直前の 1 つ)。3 つ以上は技術的負債とみなし要件レビュー対象

5.4.6.1 3.6.1 バージョン管理マップ実装例

```
const EXCLUDE_FIELDS_BY_VERSION: Record<string, string[]> = {
  "default": [...H1_FIELDS, ...H2_FIELDS, ...H3_FIELDS],
  "2024-06-20": [...H1_FIELDS, ...H2_FIELDS, ...H3_FIELDS], // 追加なし
  // バージョン追加時:
  // "2024-12-XX": [...H1_FIELDS, ..., "data.object.new_field_xxx"],
};

function getExcludeFields(apiVersion: string): string[] {
  if (EXCLUDE_FIELDS_BY_VERSION[apiVersion]) {
    return EXCLUDE_FIELDS_BY_VERSION[apiVersion];
  }
  log.warn("unknown_api_version", { apiVersion });
  notifyOperator("normal", `Stripe API version ${apiVersion} not in exclude list`);
  return EXCLUDE_FIELDS_BY_VERSION["default"];
}
```

5.4.6.2 3.6.2 バージョン追加手順

1. Stripe 公式変更履歴を確認
2. 同一 event の再送で値が変わるフィールドを特定
3. 本書の除外フィールド表に追記
4. `BillingWebhookWorker` の正規化関数(`canonical_json`)の `EXCLUDE_FIELDS_BY_VERSION` マップを更新
5. 既存 Webhook イベントの再ハッシュ計算は実施しない(同 API バージョンでの整合性のみ保証)
6. 監査記録: `audit:webhook.exclude_list.update(5y)` を追加 (action コードを [DD09_監査 action コード.md](#) に追加)

5.4.6.3 3.6.3 監視

- `webhook_events.stripe_api_version` の分布を SCR-096 KPI セクションで可視化 (同時稼働バージョン数の追跡)
- 未登録バージョン受信時の通知頻度が日次 10 件超で運営者 high(`webhook.unknown_api_version.spike`, 5y)

5.4.7 3.7 除外フィールド適用例

入力 payload:

```
{
  "id": "evt_xxx",
  "created": 1715500000,
  "livemode": false,
  "api_version": "2024-06-20",
  "request": { "id": "req_xxx", "idempotency_key": "abc" },
  "pending_webhooks": 0,
  "data": {
    "object": {
      "id": "in_xxx",
      "created": 1715500000,
      "amount_paid": 10000,
      "metadata": { "owner_account_id": "01J...", "test_run": "true" }
    },
    "previous_attributes": { "status": "open" }
  },
  "type": "invoice.paid"
}
```

除外後の正規化対象:

```
{
  "id": "evt_xxx",
  "data": {
    "object": {
      "id": "in_xxx",
      "amount_paid": 10000,
      "metadata": { "owner_account_id": "01J..." }
    }
  },
  "type": "invoice.paid"
}
```

`payload_hash = sha256(canonical_json(filtered_payload))`

5.4.8 3.8 DLQ 自動指数バックオフ + 手動リプレイ

`DLQAutoBackoffWorker`(5 分 cron) が `state='failed'` のイベントを最大 3 回リトライする:

attempt	バックオフ	累計経過
1	1 分	1 分
2	4 分	5 分
3	16 分	21 分
失敗継続 (1h 経過)	dlq_manual_replay 遷移	運営者 high alert

詳細擬似コードは [DD08_Cron 実装.md](#) §14.2.6 を参照。

手動リプレイ API:

`/webhooks/replay:`

`post:`

`summary:` Webhook 手動リプレイ

`parameters:`

```

- $ref: "#/components/parameters/XOpTicketId"
- $ref: "#/components/parameters/IdempotencyKey"
- $ref: "#/components/parameters/XCsrfToken"
requestBody:
  required: true
  content:
    application/json:
      schema:
        type: object
        required: [eventId]
        properties:
          eventId: { type: string }
responses:
  "200": { description: リプレイ実行完了 }
  "409":
    content:
      application/problem+json:
        schema: { $ref: "#/components/schemas/Problem" }
  "410":
    content:
      application/problem+json:
        schema: { $ref: "#/components/schemas/Problem" }

```

409 は既処理 (成功状態)、410 は 30 日経過 dlq_archived 状態。

5.4.9 3.9 ペイロード差分検出後のレビュー (SCR-099)

差分検出後は `webhook_payload_diffs.state` で SCR-099 上のレビューフローを管理:

From	To	トリガー
-	detected	BillingWebhookWorker が差分検出
detected	reviewed	start-review
reviewed	reprocessed_manually	reprocess(再認証 + チケット必須)
reviewed	dismissed_no_action	dismiss(理由必須)

詳細は [DD11_状態遷移詳細.md](#) § B.6 を参照。

5.4.10 3.10 Stripe API 呼出 (★TH-9)

月次請求 cron で Stripe API を呼び出す:

操作	エンドポイント	冪等キー
Invoice 作成	POST /v1/invoices	monthly-billing- <code>{owner.id}</code> - <code>{target_month}</code>
Invoice Item 作成	POST /v1/invoiceitems	同上 + -item
Invoice Finalize	POST /v1/invoices/ <code>{id}</code> /finalize	同上 + -final
Credit Note 発行	POST /v1/credit_notes	credit-note- <code>{invoiceId}</code> - <code>{reason}</code>
Subscription Re-sume	POST /v1/subscriptions/ <code>{id}</code> cancel_at_period_end=false	sub-resume- <code>{subscriptionId}</code>

詳細疑似コードは [DD08_Cron 実装.md](#) § 14.2.1 を参照。

5.4.11 3.11 連携 IF #10(本書 → メイン)

本書側 BillingWebhookWorker は受信 → 正規化 → メイン側 `/internal/admin-integration/v1/billing/webhook-event` へ mTLS + JWT で転送する。完全な JSON Schema は [基本設計 / API 設計](#) § 5.8 + [基本設計 / 課金・請求設計](#) § 13 を正本とする。

5.4.12 3.12 主要エラーコード

エラー ID	HTTP	説明
E-OP-WEBHOOK-001	401	SIGNATURE_INVALID
E-OP-WEBHOOK-002	409	EVENT_ALREADY_PROCESSED(リプレイ拒否)
E-OP-WEBHOOK-003	410	EVENT_ARCHIVED(30 日経過)
E-OP-WEBHOOK-004	502	UPSTREAM_MAIN_FAILED(連携 IF #10 5xx)

完全な E-OP-WEBHOOK-* 一覧は [基本設計 / エラー設計](#) を正本とする。

5.5 4. 関連設計

種別	参照先
要件	../01_要件定義/index.md
基本設計	../02_基本設計/index.md
API 設計 (正本)	../02_基本設計/02_API 設計.md
課金・請求設計 (正本)	../02_基本設計/10_課金・請求設計.md
メインテーブル設計	../01_メインシステム/02_基本設計/03_テーブル設計.md
関連 DD	DD08_Cron 実装.md / DD11_状態遷移詳細.md
運用設計	../04_運用設計/index.md
将来対応	../05_future/index.md

5.6 5. テスト観点

5.6.1 5.1 ユニットテスト

- ・ HMAC-SHA256 署名検証 (正当 / 改ざん / タイミング攻撃耐性)
- ・ `canonical_json` 正規化 (除外フィールド + キーソート + 配列再帰)
- ・ 同 event_id + 同 payload_hash → duplicate_skipped_hash_match
- ・ 同 event_id + 異 payload_hash → diff detected
- ・ 指数バックオフ計算 (1m / 4m / 16m)
- ・ 30 日経過判定 (dlq_archived)

5.6.2 5.2 結合テスト (Miniflare + Stripe Test Mode)

- ・ 正常受信 → 連携 IF #10 → succeeded
- ・ 一時失敗 → 自動 BO 3 回 → dlq_manual_replay
- ・ 手動リプレイ → succeeded
- ・ 差分検出 → reviewed → reprocess
- ・ 30 日経過 → dlq_archived
- ・ 不明な API バージョン → デフォルトリスト + 通知

5.6.3 5.3 E2E テスト (Playwright)

テスト ID	シナリオ
e2e-scr097-001	DLQ 一覧 → 30 日ウィンドウ → 手動リプレイ
e2e-scr099-001	差分検出 → SCR-099 表示 → reprocess / dismiss

5.6.4 5.4 負荷試験

シナリオ	構成	合格基準
(A4) Stripe Webhook 受信	100 event/分 + 10% で差分検出	冪等性 + DLQ 挙動
(A3) Webhook 集中受信	Stripe Webhook 1000 件/分 × 5 分	p95 ☒ 500ms、webhook_events.state 整合性 100%

シナリオ	構成	合格基準
(A5) DLQ 大量リプレイ	1000 件の dlq_manual_replay を同時起動	サーキットブレーカ open 率が想定通り、5xx → 自動 BO で全件最終完了

5.6.5 5.5 受入条件マッピング

AC	検証手段
AC-041(Webhook 冪等性 + 差分検出)	統合テスト (差分検出ケース含)+ リプレイ運用

5.7 6. 未確定事項・確認事項

確認事項 ID	確認内容	優先度	ステータス
-	v1.0 リリース時点で全項目確定済み	低	確認済

\newpage

6 DD05: 削除データ復元 (運営者システム)

6.1 0. 文書情報

項目	内容
文書名	DD05: 削除データ復元 (運営者システム)
詳細設計 ID	DD05
対象システム	FAQ AI ウィジェット SaaS / 運営者システム
関連機能 ID	FR-200, FR-201, FR-202, FR-203, FR-204, FR-205, FR-206, FR-207, FR-208, FR-209, FR-211, FR-222, FR-223, D-07
作成日	2026-05-17
版数	v1.0
ステータス	承認済

6.2 1. 対象範囲

種別	ID	名称
機能	FR-200～FR-209, FR-211	削除データ参照・復元 (副作用 ロールバック)
機能	FR-222	運営者操作監査
機能	FR-223	削除データ参照 (認可境界)
画面	SCR-090	削除データ参照一覧
画面	SCR-091	削除データ復元 (承認ログ + 副作用ロールバック)
API	GET /admin/api/v1/deleted-resources	一覧
API	GET /admin/api/v1/deleted-resources/{id}	詳細
API	POST /admin/api/v1/restorations	復元実行 (4-eyes 承認ログ対象)
テーブル	accounts_retired	退会・物理削除保持 (永久)
テーブル	restorations	復元履歴
テーブル	accounts(参照、メイン主管)	オーナー contract_status

6.3 2. 収録ロジック・対応章

元章	元タイトル	概要
§ 5 SCR-090	削除データ一覧	検索 / 詳細遷移
§ 5 SCR-091	削除データ復元	副作用 (a)～(g) ロールバック + 423 ケース

元章	元タイトル	概要
§ 6 復元機能(参照)	復元実行ロジック	4-eyes 承認ログ + トランザクション
§ 7 復元 API	OpenAPI	POST /restorations
付録 B.7	accounts.contract_status(オーナー行)	active / suspended / deleted_pending / deleted

6.4 3. 詳細設計本文

6.4.1 3.1 削除データのライフサイクル (D-07)

accounts.contract	説明	物理データ
active	通常稼働	accounts テーブル本体
suspended	一時停止 (運営者 owner.suspend 経由)	本体保持、機能フラグで停止
deleted_pending	退会受付済、猶予期間中 (30 日)	本体保持、復元可能
deleted	物理削除済	accounts_retired テーブルへ移行、accounts 本体は DELETE

復元可能範囲:

- **suspended / deleted_pending** —— 通常復元 (POST /restorations、承認ログ対象)
- **deleted** —— 物理削除後の復元は **不可** (accounts_retired は法令対応で永久保持されるが、本体への戻しは MVP では不可)

6.4.2 3.2 復元時の副作用ロールバック (FR-204、D-07)

退会・無効化処理で行った 7 種類の副作用を、復元時に逆順でロールバックする:

副作用 ID	削除時の処理	復元時のロールバック
(a)	全プロジェクト archived 遷移	archived → active 戻し (削除前の状態に基づく)
(b)	全 FAQ unpublished 遷移	unpublished → 削除前の状態に基づく
(c)	全アカウント (管理者ユーザー) の accounts.disabled=1	disabled=0 戻し
(d)	API 全失効	旧 API は再発行せず、復元後に新規発行を促す通知
(e)	Webhook 配信先全停止	旧 Webhook URL を disabled 解除
(f)	Stripe Subscription cancel_at_period_end=true	POST /v1/subscriptions/{id} で cancel_at_period_end=false
(g)	通知配信停止	inbox 通知再開 + 復元完了メール送信

ロールバックは **全 7 ステップを単一トランザクション** で実行し、途中失敗時は全件ロールバック + restorations.state='failed' 遷移 + 運営者 high alert。

6.4.3 3.3 復元実行 API

```

/restorations:
  post:
    summary: 削除データ復元
    parameters:
      - $ref: "#/components/parameters/XOpTicketId"
      - $ref: "#/components/parameters/XApprovalId"
      - $ref: "#/components/parameters/IdempotencyKey"
      - $ref: "#/components/parameters/XCsrfToken"
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required: [resourceType, resourceId, reason]
            properties:

```

```

    resourceType:
      type: string
      enum: [owner, project, faq, account, announcement]
    resourceId: { type: string }
    reason: { type: string, maxLength: 1000 }
  responses:
    "200":
      content:
        application/json:
          schema:
            type: object
            properties:
              restorationId: { type: string }
              restoredAt: { type: string, format: date-time }
              rollback: { type: object }
    "423":
      content:
        application/problem+json:
          schema: { $ref: "#/components/schemas/Problem" }

```

423 Locked は復元対象が既に物理削除済 (accounts_retired 移行済) で復元不可。

6.4.4 3.4 復元実行フロー

sequenceDiagram

```

participant SPA as Admin SPA
participant Worker as AdminConsoleWorker
participant D1
participant Stripe
participant Main as Main Worker
participant Queues

```

```

SPA->>Worker: POST /restorations {resourceType, resourceId, reason}<br/>+ X-Op-Ticket-Id +
Worker->>D1: SELECT accounts WHERE id=? AND contract_status IN ('suspended','deleted_pendin

```

alt 既に物理削除済

```

    Worker-->>SPA: 423 Locked

```

else 復元可能

```

    Worker->>D1: BEGIN; INSERT restorations (state='executing')
    Worker->>D1: UPDATE accounts SET contract_status='active'
    Worker->>D1: ロールバック (a) (b) (c) (d) (e)
    Worker->>Stripe: ロールバック (f) Subscription resume
    Worker->>D1: COMMIT; UPDATE restorations state='succeeded'
    Worker->>D1: audit_logs(owner.restore_data, 5y)
    Worker->>Main: 連携 IF #4 復元通知
    Worker->>Queues: 通知集約 (FR-211、10 分集約)
    Worker-->>SPA: 200 {restorationId, rollback}

```

end

6.4.5 3.5 4-eyes 承認ログ対象 (MVP)

owner.restore_data は MVP Log Only / Beta Hard Gate:

- MVP: 単独実行可能、承認ログ (operator_approval.*) を必ず残す + 事後監査
- Beta 以降: ハードゲート化、X-Approval-Id 必須

詳細は [DD02_4-eyes 承認フロー.md](#) を参照。

6.4.6 3.6 検索・一覧 (SCR-090)

検索条件	バリデーション
resourceType	owner / project / faq / account / announcement
slug 部分一致	1~64 文字

検索条件	バリデーション
削除日時範囲	from ☒ to、最大 5 年遡及
contract_status	suspended / deleted_pending / deleted(deleted は閲覧のみ、復元不可)

検索結果は (deleted_at, id) カーソルページング (limit 既定 100、最大 200)。検索自体も監査 (audit.search、1y)。

6.4.7 3.7 副作用 (a)～(g) ロールバック詳細

6.4.7.1 (a) プロジェクト復元

-- 削除前の状態を *deleted_projects* から取得し、*accounts.projects* に戻す

```
INSERT INTO projects (...)  
SELECT ... FROM deleted_projects WHERE owner_account_id = ?;  
UPDATE projects SET archived = 0 WHERE owner_account_id = ? AND deleted_at > ?;
```

6.4.7.2 (b) FAQ 復元 削除前の is_published 状態を復元 (削除時に is_published=0 強制したものは元の値に戻す)。

6.4.7.3 (c) 管理者ユーザー復元

```
UPDATE accounts SET disabled = 0  
WHERE owner_account_id = ? AND disabled_at > ?;
```

6.4.7.4 (d) API 鍵 旧 API 鍵 は再発行しない (セキュリティ上、退会時点で失効済の 鍵 を復活させない)。復元完了後に inbox 通知で「新規 API 鍵 を発行してください」と促す。

6.4.7.5 (e) Webhook 配信先

```
UPDATE webhook_endpoints SET disabled = 0  
WHERE owner_account_id = ? AND disabled_at > ?;
```

6.4.7.6 (f) Stripe Subscription

```
const subscription = await stripe.subscriptions.retrieve(owner.stripe_subscription_id);  
if (subscription.cancel_at_period_end === true) {  
  await stripe.subscriptions.update(  
    owner.stripe_subscription_id,  
    { cancel_at_period_end: false },  
    { idempotencyKey: `restore-${restorationId}-sub-resume` }  
  );  
}
```

6.4.7.7 (g) 通知配信再開 accounts.notifications_disabled=0 に戻し、復元完了メール (MSG-SCR-091-001) を強制送信。

6.4.8 3.8 復元結果の記録

```
CREATE TABLE restorations (  
  id TEXT PRIMARY KEY,  
  resource_type TEXT NOT NULL CHECK (resource_type IN ('owner', 'project', 'faq', 'account', 'annou  
  resource_id TEXT NOT NULL,  
  state TEXT NOT NULL CHECK (state IN ('executing', 'succeeded', 'failed')),  
  rollback_summary TEXT, -- JSON: (a)～(g) 各ステップの結果  
  requested_by TEXT NOT NULL,  
  approval_id TEXT,  
  reason TEXT NOT NULL,  
  ticket_id TEXT NOT NULL,  
  started_at TIMESTAMP NOT NULL,  
  completed_at TIMESTAMP,
```

```

FOREIGN KEY (requested_by) REFERENCES operator_accounts(id)
);
CREATE INDEX idx_restorations_resource ON restorations(resource_type, resource_id);
CREATE INDEX idx_restorations_started_at ON restorations(started_at);

```

完全な DDL は [基本設計 / テーブル設計](#) を正本とする。

6.4.9 3.9 通知集約 (FR-211、D-19)

復元完了通知は 10 分集約窓で運営者操作通知として送信:

キー	値	TTL
notify-batch:<owner_account_id>:owner.restore_data	{ entries: [...], firstAt }	10 分

集約後、**OperatorNotifyAggregatorWorker**(1 分 cron で 10 分窓を判定) が連携 IF #12 経由でメインへ送信し、メインがオーナー + **users:manage** 保持メンバーへ inbox 配信。

6.4.10 3.10 主要エラーコード

エラー ID	HTTP	説明
E-OP-RESTORE-001	423	RESOURCE_PHYSICALLY_DELETED(accounts_retired 移行済)
E-OP-RESTORE-002	409	RESTORATION_IN_PROGRESS(同一リソースで executing 状態あり)
E-OP-RESTORE-003	502	STRIPE_RESUME_FAILED(副作用 (f) 失敗)
E-OP-RESTORE-004	500	ROLLBACK_TRANSACTION_FAILED(副作用 (a)~(e) 失敗)

完全な E-OP-RESTORE-* 一覧は [基本設計 / エラー設計](#) を正本とする。

6.5 4. 関連設計

種別	参照先
要件	../01_要件定義/index.md
基本設計	../02_基本設計/index.md
API 設計 (正本)	../02_基本設計/02_API 設計.md
テーブル設計 (正本)	../02_基本設計/03_テーブル設計.md
メインテーブル設計	../01_メインシステム/02_基本設計/03_テーブル設計.md
関連 DD	DD02_4-eyes 承認フロー.md / DD03_監査ハッシュチェーン.md
運用設計	../04_運用設計/index.md
将来対応	../05_future/index.md

6.6 5. テスト観点

6.6.1 5.1 ユニットテスト

- (a)~(g) 各ステップのロールバック関数 (個別単体)
- 422 / 423 / 409 各エラーケース
- 単一トランザクション内での全件成功 / 全件失敗 の保証
- **contract_status** 遷移ガード (**deleted** からは復元不可)

6.6.2 5.2 結合テスト (Miniflare)

- **suspended** → **active** 復元 (全副作用ロールバック含む)
- **deleted_pending** → **active** 復元
- **deleted** 状態で 423 返却
- 復元中 (**executing**) で同一リソースに再要求 → 409
- Stripe Subscription resume 失敗時の全件ロールバック

6.6.3 5.3 E2E テスト (Playwright)

テスト ID	シナリオ
e2e-scr090-001	一覧表示 + 詳細遷移
e2e-scr091-001	復元フロー全体 + 423 ケース

6.6.4 5.4 受入条件マッピング

AC	検証手段
AC-038(運営者操作監査 + 通知)	E2E + 月次監査

6.7 6. 未確定事項・確認事項

確認事項 ID	確認内容	優先度	ステータス
-	v1.0 リリース時点で全項目確定済み	低	確認済

\newpage

7 DD06: お知らせ承認・配信 (運営者システム)

7.1 0. 文書情報

項目	内容
文書名	DD06: お知らせ承認・配信 (運営者システム)
詳細設計 ID	DD06
対象システム	FAQ AI ウィジェット SaaS/ 運営者システム
関連機能 ID	FR-149, FR-180~FR-192, FR-188, FR-189, FR-211, D-09, D-19, D-20
作成日	2026-05-17
版数	v1.0
ステータス	承認済

7.2 1. 対象範囲

種別	ID	名称
機能	FR-149	お知らせ作成・配信 (運営者)
機能	FR-188	お知らせ予約配信
機能	FR-189	お知らせ取消・訂正告知
機能	FR-211	運営者操作通知 10 分集約
画面	SCR-094	お知らせ作成・配信 (運営者)
API	POST /announcements	作成
API	POST /announcements/{id}/preview	プレビュー
API	POST /announcements/{id}/schedule	予約
API	POST /announcements/{id}/cancel	取消
API	POST /announcements/{id}/send	即時送信
API	POST /announcements/{id}/test-send	テスト送信
テーブル	announcement_drafts	下書き~配信状態
連携 IF	#7	お知らせ配信 (本書 → メイン)

7.3 2. 収録ロジック・対応章

元章	元タイトル	概要
§ 5 SCR-094	お知らせ作成・配信画面	4 種類 kind + 3 severity + scope + opt-out
§ 5.7	HTML サニタイザ (★TH-4)	許可タグ・属性ホワイトリスト
§ 6 お知らせ機能(参照)	下書き → preview → schedule → sending → sent	ライフサイクル
§ 7 お知らせ API	OpenAPI 抜粋	リクエスト/レスポンス
§ 14.2.4	AnnouncementSchedulerWorker 擬似コード	1分ポーリング (D-09)
付録 B.4	announcement_drafts.state	8 状態遷移
付録 D.2	連携 IF #7 JSON Schema	本書主管

7.4 3. 詳細設計本文

7.4.1 3.1 状態遷移 (announcement_drafts.state)

From	To	トリガー
-	draft	POST /announcements
draft	preview	preview
preview	scheduled	schedule(再認証 + チケット必須)
scheduled	sending	AnnouncementSchedulerWorker(scheduled_at ☒ now+5min)
scheduled	cancelled	cancel(now < scheduled_at - 5min)
sending	sent	連携 IF #7 200 OK
sending	failed	連携 IF #7 一時失敗 (5xx/Timeout)、attempt_count++
failed	sending	自動指数 BO(1m → 4m → 16m、最大 3 回)
failed	dlq	自動 BO 3 回失敗、運営者 inbox high

詳細は [DD11_状態遷移詳細.md](#) § B.4 を参照。

7.4.2 3.2 お知らせ作成 API

/announcements:

post:

summary: お知らせ作成

parameters:

- \$ref: "#/components/parameters/XOpTicketId"
- \$ref: "#/components/parameters/IdempotencyKey"
- \$ref: "#/components/parameters/XCsrfToken"

requestBody:

required: true

content:

application/json:

schema:

type: object

required: [kind, severity, scope, subject, bodyHtml]

properties:

kind: { type: string, enum: [maintenance, incident, feature, policy] }

severity: { type: string, enum: [info, normal, high] }

scope: { type: string, enum: [all, owner_subset] }

ownerAccountIds: { type: array, items: { type: string } }

subject: { type: string }

bodyHtml: { type: string }

optOut: { type: string, enum: [none, optional, mandatory] }

responses:

"201":

content:

application/json:

schema:

type: object

```

    properties:
      announcementId: { type: string }
      state: { type: string, enum: [draft] }
"422":
  description: HTML サニタイザ拒否
  content: { application/problem+json: { schema: { $ref: "#/components/schemas/Problem" } }

```

7.4.3 3.3 二段階 HTML サニタイズ (R-011)

ステージ	タイミング	対象	拒否条件
永続化前	POST /announcements 受信時	bodyHtml	許可タグ外、<script> 含む、href="javascript:" 等
表示時	メイン側 inbox 表示時	保管済 HTML	同上 (防御深層)

7.4.3.1 3.3.1 許可タグホワイトリスト

a, b, br, code, em, i, li, ol, p, pre, span, strong, ul

7.4.3.2 3.3.2 許可属性ホワイトリスト

a: href, target="_blank", rel="noopener noreferrer"
span: class (限定: "highlight" のみ)

7.4.3.3 3.3.3 拒否されるパターン例

- ・ <script>...</script> —— タグ拒否
- ・ —— タグ拒否 (画像は MVP では非対応)
- ・ —— 属性値スキーム拒否
- ・ —— 同上
- ・ HTML コメント <!-- --> —— 全削除

7.4.3.4 3.3.4 拒否時のレスポンス

```

{
  "type": "https://admin.open-faq.example.com/errors/E-OP-ANN-001",
  "title": "HTML サニタイザ拒否",
  "status": 422,
  "code": "E-OP-ANN-001",
  "detail": "Disallowed tag: <script>",
  "trace_id": "01J..."
}

```

7.4.4 3.4 予約配信スケジューラ (D-09)

```

function announcementSchedule():
  candidates = D1.query("SELECT id FROM announcement_drafts WHERE state='scheduled' AND sched
  for c in candidates:
    D1.exec("UPDATE announcement_drafts SET state='sending' WHERE id=? AND state='scheduled
    try:
      result = callMainIF(7, {announcementId: c.id, ...})
      if result.ok:
        D1.exec("UPDATE announcement_drafts SET state='sent', sent_at=? WHERE id=?", no
        audit_logs(announcement.send, 5y, payload={announcement_id: c.id, recipients: r
      else:
        handleRetry(c.id)
    catch e:
      handleRetry(c.id)

```

項目	仕様
cron 周期	***** (毎分)

項目	仕様
早期判定窓	scheduled_at ☒ now + 5min(早送りで境界跨ぎを吸収)
最大件数	KV feature:announcement-batch-size(既定 100、運営者設定可)
取消可能期限	now < scheduled_at - 5min
自動 BO	1m / 4m / 16m、最大 3 回 (failed → sending 再試行)
DLQ 遷移	3 回連続失敗で dlq、運営者 inbox high 通知

7.4.5 3.5 連携 IF #7 JSON Schema(本書 → メイン)

```
{
  "type": "object",
  "required": ["announcementId", "kind", "severity", "scope", "subject", "bodyHtml"],
  "properties": {
    "announcementId": { "type": "string" },
    "kind": { "type": "string", "enum": ["announcement", "system"] },
    "severity": { "type": "string", "enum": ["low", "normal", "high"] },
    "scope": {
      "type": "object",
      "required": ["type"],
      "properties": {
        "type": { "type": "string", "enum": ["all", "owners", "user_types"] },
        "ownerAccountIds": { "type": "array", "items": { "type": "string" } },
        "userTypes": { "type": "array", "items": { "type": "string", "enum": ["admin"] } }
      }
    },
    "subject": { "type": "string", "maxLength": 200 },
    "bodyHtml": { "type": "string", "maxLength": 10000 },
    "optOut": { "type": "string", "enum": ["optional", "mandatory"] },
    "scheduledAt": { "type": "string", "format": "date-time" }
  }
}
```

メインは受信後、対象オーナー + 管理者ユーザーの inbox に配信し、MSG-SCR-094-* テンプレートに従ってメール送信 (opt-out 設定に従う)。

7.4.6 3.6 訂正告知 (FR-189)

既に **sent** 状態のお知らせに対し、訂正告知を作成できる:

項目	仕様
kind	incident(訂正告知も同 kind を使う、correctionOf フィールドで紐付け)
correctionOf	訂正元 announcementId
監査	announcement.correction.issue(5y、{newAnnouncementId, correctionOf})

訂正告知の本文 HTML には自動的に「訂正前: [元告知タイトル]」リンクが挿入される (**SanitizedHtml** レンダラ側で **correctionOf** を参照)。

7.4.7 3.7 テスト送信 (POST /announcements/{id}/test-send)

項目	仕様
送信先	申請者運営者本人のメール
監査	announcement.test_send(1y、{announcementId, sentTo})
Resend 直叩き	本書側 BillingWebhookWorker 同様、運営者通知 + テスト送信のみ Resend 直叩き

7.4.8 3.8 通知配信 (D-20)

- ・ **オーナー + users:manage 保持メンバー全員** に配信 (severity=high の場合は強制送信)

- メインの `inbox_messages` テーブルに `kind=announcement` で1年保持 (`retention_class='general'` 相当)、配信メタ情報を `audit_logs` に `retention_class='operator_high_priv'`(5y) で記録
- ユーザー個別の opt-out 設定が `mandatory` の場合はメールも強制送信

通知契機の正本は [基本設計 / メッセージ一覧](#) §8 を参照。

7.4.9 3.9 10 分集約窓 (D-19、運営者操作通知)

`announcement.send` 自体は集約対象外 (個別に通知)。一方、SCR-094 経由の運営者操作 (`announcement.schedule` / `announcement.cancel`) は同一オーナー宛通知が 10 分以内に複数発生する場合に集約。

KV キー	TTL	集約処理
notify-batch:<owner_account_id>:announcement.schedule	10 分	OperatorNotifyAggregatorWorker が 10 分窓終了で連携 IF #12 送信

7.4.10 3.10 主要エラーコード

エラー ID	HTTP	説明
E-OP-ANN-001	422	HTML_SANITIZER_REJECT(タグ / 属性 / スキーム拒否)
E-OP-ANN-002	409	INVALID_STATE_TRANSITION(draft → sent 等、許可されない直接遷移)
E-OP-ANN-003	410	TOO_LATE_TO_CANCEL(now ☒ scheduled_at - 5min)
E-OP-ANN-004	502	UPSTREAM_MAIN_FAILED(連携 IF #7 5xx)
E-OP-ANN-005	503	DLQ_AFTER_3_ATTEMPTS

完全な E-OP-ANN-* 一覧は [基本設計 / エラー設計](#) を正本とする。

7.5 4. 関連設計

種別	参照先
要件	../01_ 要件定義/index.md
基本設計	../02_ 基本設計/index.md
API 設計 (正本)	../02_ 基本設計/02_API 設計.md
メッセージ一覧 (正本)	../02_ 基本設計/06_ メッセージ一覧.md
メインテーブル設計	../01_ メインシステム/02_ 基本設計/03_ テーブル設計.md
関連 DD	DD02_4-eyes 承認フロー.md / DD08_Cron 実装.md / DD11_状態遷移詳細.md
運用設計	../04_ 運用設計/index.md
将来対応	../05_future/index.md

7.6 5. テスト観点

7.6.1 5.1 ユニットテスト

- HTML サニタイザ (50 ペイロード攻撃ケース)
- 状態遷移ガード (`draft` → `sent` 直接遷移拒否、`scheduled` → `cancelled` の 5 分前境界)
- 連携 IF #7 JSON Schema バリデーション
- 訂正告知の `correctionOf` リンク挿入

7.6.2 5.2 結合テスト (Miniflare)

- 1 分 cron でスケジューラが `scheduled` → `sending` → `sent` 遷移
- 連携 IF #7 5xx → 自動 BO 3 回 → `dlq` 遷移 + 高アラート通知
- 取消 5 分前境界 (`now = scheduled_at - 5min`)
- テスト送信 (Resend Test Mode)

7.6.3 5.3 E2E テスト (Playwright)

テスト ID	シナリオ
e2e-scr094-001	作成 → プレビュー → 予約 → 取消 → 即時送信 → テスト送信 → 訂正告知

7.6.4 5.4 品質回帰

データセット	件数
HTML サニタイズ攻撃	50 ペイロード (<script>/javascript:/data:/SVG/コメント/入れ子)

7.6.5 5.5 受入条件マッピング

AC	検証手段
-	E2E のみ (MVP では個別 AC なし、運営者通知としては FR-211/NFR-704 で参照)

7.7 6. 未確定事項・確認事項

確認事項 ID	確認内容	優先度	ステータス
-	v1.0 リリース時点で全項目確定済み	低	確認済

\newpage

8 DD07: KV・R2 オブジェクト (運営者システム)

8.1 0. 文書情報

項目	内容
文書名	DD07: KV・R2 オブジェクト (運営者システム)
詳細設計 ID	DD07
対象システム	FAQ AI ウィジェット SaaS / 運営者システム
関連機能 ID	TH-7, D-13, D-14, D-15, D-17, D-19
作成日	2026-05-17
版数	v1.0
ステータス	承認済

8.2 1. 対象範囲

種別	ID	名称
機能	TH-7	KV キー・R2 オブジェクト一覧の確定
KV namespace	admin_cache	運営者プレーン専用 KV
R2 bucket	admin_archive	運営者プレーン専用 R2
API	全機能横断	KV 参照型機能フラグ / レート上書き / AI パラメータ等

8.3 2. 収録ロジック・対応章

元章	元タイトル	概要
§9 全文	KV キー・R2 オブジェクト一覧 (★TH-7)	全 KV キーの命名・値スキーマ・TTL・サンプル・参照 Worker
付録 J	KV キー早見表	1 ページ集約 + 監視 KPI 動的閾値 + サーキットブレーカ

8.4 3. 詳細設計本文

8.4.1 3.1 KV キー命名規則

項目	規約
一般形式	<feature>:<scope>:<id>(英小文字 + ハイフン + コロン区切り)
機能ごとに TTL を設定	認証系 60s~15min、フラグ系 60s、ルール系 60s、レート/予算 30s、トークン系 個別
文字制限	英数 + - + _ + :、最大 512 バイト
値形式	JSON 文字列、bool、数値、または ID 文字列
Namespace	admin_cache(本書側専用 KV namespace)

8.4.2 3.2 KV キー全表

8.4.2.1 3.2.1 認証・セッション

キー形式	値スキーマ	TTL	更新タイミング	参照 Worker	サンプル
operator-session:<session_id>	{ operatorId, mfaVerifiedAt, reauthenticatedAt, csrfToken, expiresAt }	60 秒(キャッシュ)	ログイン時、ログアウト時失効	AdminConsoleWorker	{ "operatorId": "01J...", "mfaVerifiedAt": "...", "csrfToken": "...", "reauthenticatedAt": "...", "expiresAt": "..." }
operator-ip-allowlist:<operator_id>	["203.0.113.0/24", "2001:db8::1"](CIDR 配列)	360 秒	IP リスト変更時に Invalidate	エッジ (Admin-Console-Worker)	["203.0.113.0/24"]
mfa-setup:<account_id>	{ token, secret, expiresAt }	72 時間	MFA セットアップ開始時	AdminConsoleWorker	
password-reset:<token_hash>	{ operatorId, expiresAt }	60 分	リセット要求時	AdminConsoleWorker	
re-auth:<session_id>	{ operatorId, reauthenticatedAt, consumed }	15 分(1 回限り、consumed=true で即時失効)	再認証成功時	AdminConsoleWorker	{ "operatorId": "01J...", "consumed": false }
login-lockout:<ip_hash>:<account_id>	{ failedCount, lockedUntil }	15 分	ログイン失敗時	AdminConsoleWorker	{ "failedCount": 5, "lockedUntil": "..." }

8.4.2.2 3.2.2 機能フラグ

キー形式	値	TTL	用途
feature:hard-gate:<action_code>	true / false	60 秒(キャッシュ)	MVP ハードゲート 3 操作を制御
feature:ai-model:rollout:<version>	{ percentage: 0 10 50 100 }	60 秒	AI モデルロールアウト
feature:pii-rule-rollout:<revision_id>	{ percentage: 0 10 50 100 }	60 秒	PII ルールロールアウト
feature:announcement-batch-size	数値	60 秒	1 cron tick あたりのお知らせ最大処理件数(既定 100)

8.4.2.3 3.2.3 ルール・パラメータ

キー形式	値スキーマ	TTL	主管	更新元
pii-rules:regex	[{ id, pattern, enabled }, ...]	60 秒 (D-13)	本書	SCR-098 POST /pii-rules/revisions
pii-rules:classifier	{ threshold, weights, modelId }	60 秒 (D-13)	本書	同上
ai-params:global	{ confidenceThreshold, relevanceThreshold, modelId }	60 秒 (D-15)	本書	SCR-092
ai-params:owner:<owner_account_id>	同上	60 秒	本書	SCR-092

キー形式	値スキーマ	TTL	主管	更新元
ai-params:project:<project_id>	同上	60 秒	本書	SCR-092
ai-models:available	["@cf/meta/llama-3.1-8b-instruct", ...]	60 秒	本書	運営者手動
ai-cost:unit-prices	{ "<model_id>": { "input_per_1k_tokens": <yen>, "output_per_1k_tokens": <yen> } }	60 秒	本書	運営者手動 (NFR-804 (m) FR-304)

8.4.2.4 3.2.4 レート / 予算上書き

キー形式	値スキーマ	TTL	用途
rate-limit:<owner_account_id>	{ widgetAskPerMin, chatEndUserPerMin, chatStaffPerMin }	30 秒 (D-14)	メイン側参照
budget-limit:<owner_account_id>	{ monthlyBudgetYen }	30 秒	同上
budget-limit:min	数値	永続 (運営者更新時のみ Invalidate)	バリデーション下限
budget-limit:max	数値	永続	バリデーション上限

8.4.2.5 3.2.5 Webhook・トークン

キー形式	値スキーマ	TTL	用途
webhook:idempotency:<event_id>	{ payloadHash, state, processedAt }	30 日	event_id 冪等 (D1 webhook_events のキャッシュ)
audit-export:<job_id>	{ status, files, signature }	24 時間	エクスポート進捗

8.4.2.6 3.2.6 通知集約

キー形式	値スキーマ	TTL	用途
notify-batch:<owner_account_id>:<operation_kind>	{ entries: [...], firstAt }	10 分 (D-19)	FR-211 集約窓

8.4.3 3.3 R2 オブジェクトパス

Namespace: **admin_archive**(本書側専用 R2 バケット)

パス	用途	保持	アクセス
dlq-stripe-events/<event_id>.json	DLQ 退避 Webhook ペイロード	30 日 (life-cycle rule)	BillingWebhookWorker / SCR-097
audit-export/<job_id>-<seq>.csv	監査ログエクスポート (CSV)	24 時間	AdminConsoleWorker
audit-export/<job_id>-<seq>.jsonl	同 (JSONL)	24 時間	同上
audit-export/<job_id>-<seq>.sig	HMAC 署名ファイル (D-17)	24 時間	同上
webhook-payload-snapshots/<event_id>-<received_at>.json	ペイロード差分検出時のスナップショット	30 日	BillingWebhookWorker
audit-archive/<retention_class>/<year>-<month>.tar.gz	監査ログ年次アーカイブ	法令対応期間内	R2AuditArchiveWorker
backup/d1-snapshots/<env>/<date>.sqlite	D1 日次バックアップ (NFR-803 三層)	日次 30 日 / 週次 12 週 / 月次 12 ヶ月	Backup Worker(別 Worker)

8.4.4 3.4 KV キー早見表 (付録 J 同等)

キー形式	値	TTL	用途
operator-session:<sid>	JSON	60s	セッションキャッシュ
operator-ip-allowlist:<operator_id>	JSON 配列	60s	エッジ IP 判定
mfa-setup:<account_id>	JSON	72h	初回 MFA
password-reset:<token_hash>	JSON	60m	リセット
re-auth:<sid>	JSON	15m	1 回限り
login-lockout:<ip_hash>:<account_id>	JSON	15m	ロックアウト
feature:hard-gate:<action_code>	bool	60s	4-eyes 切替
feature:pii-layer2:enabled	bool	60s	PII NER
feature:ai-model:rollout:<version>	JSON	60s	モデル段階展開
feature:pii-rule-rollout:<revision>	JSON	60s	PII ルール段階
feature:announcement-batch-size	int	60s	お知らせバッチ
pii-rules:regex	JSON 配列	60s	PII 第 1 層
ai-params:global	JSON	60s	AI 既定値
ai-params:owner:<owner_account_id>	JSON	60s	契約上書き
ai-params:project:<project_id>	JSON	60s	プロジェクト上書き
ai-models:available	JSON 配列	60s	モデル一覧
ai-cost:unit-prices	JSON	60s	単価表 (FR-304)
rate-limit:<owner_account_id>	JSON	30s	レート上書き
budget-limit:<owner_account_id>	JSON	30s	予算上書き
budget-limit:min / max	int	永続	バリデーション
webhook:idempotency:<event_id>	JSON	30d	幂等キャッシュ
audit-export:<job_id>	JSON	24h	エクスポート進捗
notify-batch:<owner_account_id>:<kind>	JSON	10m	FR-211 集約
monitoring:thresholds:<kpi_id>	JSON	永続	KPI 動的閾値 (下表で初期値定義)
cb:internal-api:<endpoint>	JSON	60s (open 時)	サーキットブレーカ状態
sre:planned-outage:<id>	JSON	永続	計画停止期間 (SLA 除外用)

8.4.5 3.5 監視 KPI 動的閾値の初期値 (monitoring:thresholds:<kpi_id>)

SLAComputeWorker が KPI 集計時に参照する。MVP の初期値を以下に固定し、運営が wrangler kv key put で CLI 更新する。

kpi_id	初期値	単位	意味	チューニングガイド
NFR-103-ai-p95	2500	ms	AI 推論 p95 目標	MVP で 2500ms を維持。p95 が 80% を超える日が月内 3 回以上 → 緩和判定 (運営合議)
NFR-105-admin-p95	800	ms	管理画面一覧 p95	4 週連続達成で AC-046 判定
NFR-106-unread-p95	200	ms	未読件数バッジ p95	同上
audit-search-p95	1000	ms	監査ログ検索 p95	100 万件超過時に運営者へ通知
audit-export-duration-p95	60	s	エクスポート完了時間 p95	行平均サイズ実測値で運営者へ通知
chain-verify-duration	3600	s	ハッシュチェーン日次検証時間	1000 万行超過時に分割検証へ
dlq-stale-1h-count	0	count	1h 超過 DLQ 件数	0 維持、1 件で high alert
four-eyes-pending-72h	0	count	72h 経過承認待ち件数	0 維持、1 件で high alert
webhook-payload-diff-detected-24h	0	count	24h 内 detected 件数	0 維持、1 件で normal alert

kpi_id	初期値	単位	意味	チューニングガイド
pii-fp-pending-3bd	0	count	3 営業日経過 PII 誤検出報告	0 維持、1 件で normal alert
owner-mau-total	—	count	全契約横断 MAU	観測のみ、閾値なし
ai-cost-per-owner-monthly	50000	yen	契約月次 AI 原価 (運営判断用)	5 万円超過契約は月次レビュー対象

変更履歴監査: 閾値変更時は `monitoring.threshold.update` action コード (`retention_class='5y'`) で `audit_logs` に記録。before/after 値を `audit_logs.diff` に格納。

8.4.6 3.6 サーキットブレーカ KV(`cb:internal-api:<endpoint>`)

値の形式:

```
{
  "state": "closed" | "open" | "half_open",
  "openedAt": 1715600000000,
  "failureCount": 3
}
```

TTL は `open` 状態時 60s、`closed` 復帰時は明示 DELETE。

8.4.7 3.7 計画停止 KV(`sre:planned-outage:<id>`)

SLA 集計時に除外する計画停止期間を管理:

```
{
  "outageId": "01J...",
  "scope": ["api", "webhook"],
  "startsAt": "2026-06-01T19:00:00Z",
  "endsAt": "2026-06-01T20:00:00Z",
  "reason": "Cloudflare maintenance"
}
```

SLAComputeWorker が集計時に重複期間を除外する。

8.4.8 3.8 KV 運用ガイドライン

項目	ガイドライン
TTL 既定	キャッシュ 60 秒、レート/予算 30 秒、トークン 個別、フラグ 60 秒
Invalidate	値変更後に必ず DELETE(TTL 待ちでは設計上の遅延が許容できる場合のみ放置)
値サイズ	1 KB 以下を推奨、超過時は R2 回避を検討
KV API レート	admin_cache namespace 全体で 1000 RPS 上限を想定。超過時は L1 cache(Worker メモリ) 併用
監査	KV 値の運営者操作変更は対応する <resource>.<verb> action_code を必ず記録 (feature.hard_gate.toggle 等)

8.4.9 3.9 R2 運用ガイドライン

項目	ガイドライン
Lifecycle Rule	dlq-stripe-events/* は 30 日後自動削除、webhook-payload-snapshots/* も 30 日
監査エクスポート	24 時間後自動削除 (必要なら運営者が手元 PC ヘダウンロード)
アーカイブ	audit-archive/* は法令保持期間まで保持、Lifecycle なし
バックアップ	backup/d1-snapshots/<env>/* は日次 30 / 週次 12 / 月次 12
アクセス制御	公開 URL は Pre-signed URL のみ、TTL 24h、HMAC 署名 (HKDF info=r2-presigned)

8.5 4. 関連設計

種別	参照先
要件	../01_ 要件定義/index.md
基本設計	../02_ 基本設計/index.md
セキュリティ設計 (正本)	../02_ 基本設計/09_ セキュリティ設計.md
関連 DD	DD01_ 運営者認証・6 段認可.md / DD03_ 監査ハッシュチェーン.md / DD04_ Stripe_Webhook 一次受信.md / DD06_ お知らせ承認・配信.md / DD10_PII 偽陽性報告.md
運用設計	../04_ 運用設計/index.md
将来対応	../05_future/index.md

8.6 5. テスト観点

8.6.1 5.1 ユニットテスト

- KV キー命名規則バリデーション (英小文字 + ハイフン + コロン)
- TTL 設定値の確認 (KV 値書込時に **expirationTtl** 指定)
- 機能フラグ KV キャッシュヒット・ミス

8.6.2 5.2 結合テスト (Miniflare)

- レート上書き → KV 書込 → 30 秒以内のメイン側参照で反映
- ハードゲート KV トグル → 60 秒以内に該当 action ハードゲート発動
- KV 値変更後の Invalidate 動作確認

8.6.3 5.3 E2E テスト

- 監視 KPI 閾値変更 → **monitoring.threshold.update** 監査記録
- サーキットブレーカ open / half_open / closed 遷移

8.6.4 5.4 運用テスト

- R2 Lifecycle Rule 30 日後の自動削除動作確認 (月次運用テスト)
- バックアップ三層保持の最古ファイル参照可能性

8.7 6. 未確定事項・確認事項

確認事項 ID	確認内容	優先度	ステータス
-	v1.0 リリース時点で全項目確定済み	低	確認済

\newpage

9 DD08: Cron 実装 (運営者システム)

9.1 0. 文書情報

項目	内容
文書名	DD08: Cron 実装 (運営者システム)
詳細設計 ID	DD08
対象システム	FAQ AI ウィジェット SaaS / 運営者システム
関連機能 ID	FR-303, AC-042, TH-10, D-04, D-05, D-08, D-09, D-11
作成日	2026-05-17
版数	v1.0
ステータス	承認済

9.2 1. 対象範囲

種別	ID	名称
機能	FR-303	月次請求確定 cron
受入条件	AC-042	月次請求 cron 冪等
Worker	MonthlyBillingCronWorker	月初請求書発行
Worker	AuditChainVerifierWorker	日次ハッシュチェーン検証
Worker	AnnouncementSchedulerWorker	1分ポーリング配信予約
Worker	DLQAutoBackoffWorker	5分指数BO + 1h で dlq_manual_replay 遷移
Worker	RetentionPurgeWorker	日次 3 区分物理削除
Worker	R2AuditArchiveWorker	年次 R2 圧縮アーカイブ
Worker	OperatorNotifyAggregatorWorker	1分 10 分集約処理

9.3 2. 収録ロジック・対応章

元章	元タイトル	概要
§ 14 全文	Cron 実装詳細 (★TH-10)	UTC cron 式表 + 擬似コード + 失敗時通知ポリシー
付録 L	Cron UTC 一覧	1 ページ集約

9.4 3. 詳細設計本文

基本設計 § 6.5 / § 6.2.7 / § 10.4 + D-04 / D-05 / D-09 / D-11 を物理化し、UTC cron 式を確定する。

9.4.1 3.1 Cron Triggers UTC cron 式表 (全 cron)

前提 Cloudflare Cron Triggers は標準 cron 構文 (分・時・日・月・曜日) のみをサポートし、**L**(last day of month)・**W**・**#** 等の拡張構文は使用できない。月末日付や JST の月初判定は **Worker 内のロジック** で行う (後述 § 3.1.1 ガード)。JST = UTC + 9 時間。

Worker	JST 想定	UTC cron 式	UTC 換算	Worker 内ガード	冪等性キー	リトライ	関連 D
MonthlyBillingCronWorker	月初 02:00 JST	0 17 * * * (毎日 17:00 UTC)	翌日 02:00 JST	if (jstNow().day !== 1) return;	(owner_account_id, billing_year_month) UNIQUE	3 回	D-11
AuditChainVerifierWorker	日次 02:00 JST	0 17 * * *	翌日 02:00 JST	-	(verify_date)	-	D-04
AnnouncementSchedulerWorker	1分ポーリング	* * * * *	毎分	-	-	-	D-09
DLQAutoBackoffWorker	5分ポーリング	* / 5 * * * *	5 分間隔	-	(event_id, attempt)	指数 BO 3 回	-
RetentionPurgeWorker	日次 03:00 JST	0 18 * * *	翌日 03:00 JST	-	(purge_date, retention_class)	-	D-08
R2AuditArchiveWorker	年次 12/31 04:00 JST	0 19 30 12 *	UTC 12/30 19:00 = JST 12/31 04:00	-	(archive_year)	-	-
OperatorNotifyAggregatorWorker	10 分集約処理	* * * * *	毎分	-	KV TTL	-	D-19

9.4.1.1 3.1.1 Worker 内 JST 判定ガード (MonthlyBillingCronWorker) Cloudflare Cron Triggers は **L**(月末日) 構文をサポートしないため、MonthlyBillingCronWorker は **毎日 17:00 UTC に起動し、JST 換算で当日が月初 (day === 1) である場合のみ処理を進める**。

```
function shouldRun(): boolean {
  const utcNow = new Date(); // Worker 起動時刻 (UTC)
  const jstNow = new Date(utcNow.getTime() + 9 * 60 * 60 * 1000);
  return jstNow.getUTCDate() === 1; // JST の日付が 1 日のとき true
}
```

```

}

if (!shouldRun()) {
  log.info("monthly-billing: skipped (jstDay != 1)");
  return;
}
// 以降が §3.2.1 の処理

```

ガード判定自体は毎日実行されるが、`shouldRun()=false` で即 return するため副作用は発生しない。閏年・夏時間影響を受けず、確定的に「JST の月初日に1回だけ」実行される。

9.4.1.2 3.1.2 月末日のずれ対策 `R2AuditArchiveWorker` は `0 19 30 12 *`(UTC 12/30 19:00 = JST 12/31 04:00)を採用。JST の意図時刻を起点に UTC 計算した結果で、`L` 構文に依存しない。

UTC 換算の検算ルール(本書改訂時の必須チェック):

JST 想定	UTC 換算	cron 式	検算
03:00 JST	前日 18:00 UTC	0 18 <day-1> <month> *	翌日 03:00 JST に発火 ☒
02:00 JST	前日 17:00 UTC	0 17 <day-1> <month> *	翌日 02:00 JST に発火 ☒
04:00 JST	前日 19:00 UTC	0 19 <day-1> <month> *	翌日 04:00 JST に発火 ☒
12/31 04:00 JST	12/30 19:00 UTC	0 19 30 12 *	12/31 04:00 JST に発火 ☒

9.4.2 3.2 各 cron の擬似コード

9.4.2.1 3.2.1 MonthlyBillingCronWorker(D-11)

```

function monthlyBilling():
  target_month = (now - 1 day).format("YYYY-MM") // 月初実行時に前月を対象
  audit_logs(billing.cron.run, 7y, payload={month: target_month, started_at: now})

  owners = D1.query("SELECT id FROM accounts WHERE is_owner = 1 AND contract_status='active'")
  success = 0; failed = 0
  for owner in owners:
    try:
      usage = computeUsage(owner.id, target_month)
      amount = computeAmount(usage)
      existing = D1.query("SELECT id FROM billing_invoices WHERE owner_account_id=? AND b")
      if existing:
        continue // 冪等
      invoiceId = ULID()
      D1.exec("INSERT INTO billing_invoices (id, owner_account_id, billing_year_month, am")
      stripeInvoice = stripe.invoiceItems.create({customer: owner.stripe_customer_id, amo")
      stripeInv = stripe.invoices.create({customer: owner.stripe_customer_id, auto_advanc")
      D1.exec("UPDATE billing_invoices SET stripe_invoice_id=? WHERE id=?", stripeInv.id,")
      audit_logs(billing.invoice.issued, 7y, payload={invoice_id: invoiceId})
      sendMail(owner, FR-148)
      sendInbox(owner, FR-187)
      success++
    catch e:
      failed++
      if attempt >= 3:
        notifyOperator(high, "Monthly billing failed for owner=${owner.id}")

  audit_logs(billing.cron.run, 7y, payload={summary: {success, failed}})

```

9.4.2.2 3.2.2 AuditChainVerifierWorker(D-04)

```

function auditChainVerify():
  verify_date = today
  runId = ULID()
  audit_logs(audit.chain.verify.start, 5y, payload={verify_date, run_id: runId})

```

```

prev_hash = "0" * 64
mismatch_count = 0
cursor = null
do:
    rows = D1.query("SELECT id, prev_hash, record_hash, ... FROM audit_logs WHERE id > ? OR")
    for r in rows:
        expected = sha256(prev_hash + canonical_json(r.{actor_id, action, ..., retention_cl
        if expected != r.record_hash:
            audit_logs(audit.chain.verify.fail, 5y, payload={row_id: r.id, expected, actual
            notifyOperator(high, "Audit chain mismatch at row=${r.id}")
            mismatch_count++
        prev_hash = r.record_hash
        cursor = r.id
    while rows.length > 0

    audit_logs(audit.chain.verify, 5y, payload={verify_date, run_id: runId, mismatch_count, tot

```

9.4.2.3 3.2.3 AnnouncementSchedulerWorker(D-09)

```

function announcementSchedule():
    candidates = D1.query("SELECT id FROM announcement_drafts WHERE state='scheduled' AND sched
    for c in candidates:
        D1.exec("UPDATE announcement_drafts SET state='sending' WHERE id=? AND state='scheduled
        try:
            result = callMainIF(7, {announcementId: c.id, ...})
            if result.ok:
                D1.exec("UPDATE announcement_drafts SET state='sent', sent_at=? WHERE id=?", no
                audit_logs(announcement.send, 5y, payload={announcement_id: c.id, recipients: r
            else:
                handleRetry(c.id)
        catch e:
            handleRetry(c.id)

```

9.4.2.4 3.2.4 DLQAutoBackoffWorker

```

function dlqAutoBackoff():
    candidates = D1.query("SELECT * FROM webhook_events WHERE state='failed' AND next_retry_at
    for e in candidates:
        try:
            payload = R2.get(`dlq-stripe-events/${e.event_id}.json`)
            result = callMainIF(10, payload)
            if result.ok:
                D1.exec("UPDATE webhook_events SET state='succeeded', last_transition_at=? WHERE
                D1.exec("INSERT INTO dlq_replay_log (id, event_id, replay_type, attempted_at, r
                audit_logs(stripe.event.processed, 7y)
            else:
                attempt = e.attempt_count + 1
                backoffMs = [1*60_000, 4*60_000, 16*60_000][attempt - 1]
                D1.exec("UPDATE webhook_events SET attempt_count=?, next_retry_at=?, last_trans
                if attempt >= 3:
                    D1.exec("UPDATE webhook_events SET state='dlq_manual_replay' WHERE event_id
                    notifyOperator(high, "Webhook DLQ stuck: ${e.event_id}")

    // 30 日経過 archived
    D1.exec("UPDATE webhook_events SET state='dlq_archived' WHERE state='dlq_manual_replay' AND

```

9.4.2.5 3.2.5 RetentionPurgeWorker(D-08)

```

function retentionPurge():
    today = today
    runId = ULID()
    audit_logs(retention.purge.run.start, 5y, payload={run_id: runId, date: today})

```

```

for cls, days in [('1y', 365), ('5y', 365*5), ('7y', 365*7)]:
    cutoff = today - days
    deletedCount = D1.exec("DELETE FROM audit_logs WHERE retention_class=? AND occurred_at < ?", today - days, cls)
    audit_logs(retention.purge.run, 5y, payload={run_id: runId, retention_class: cls, deleted_count: deletedCount})

// 通知ログ・エラーログも個別保持
D1.exec("DELETE FROM inbox_messages WHERE created_at < ?", today - 365)

```

9.4.2.6 3.2.6 R2AuditArchiveWorker

```

function r2AuditArchive():
    year = now.year - 1
    for cls in ['5y', '7y']:
        for month in 1..12:
            rows = D1.query("SELECT * FROM audit_logs WHERE retention_class=? AND occurred_at < ?", today - days, cls)
            if rows.length == 0: continue
            jsonl = rows.map(r => JSON.stringify(r)).join('\n')
            gzipped = gzip(jsonl)
            R2.put(`audit-archive/${cls}/${year}/${month}.tar.gz`, gzipped)
        audit_logs(retention.archive.run, 5y, payload={archive_year: year})

```

9.4.3 3.3 Cron 失敗時の通知ポリシー

Worker	連続失敗回数	通知
MonthlyBillingCronWorker	3 回	運営者 high
AuditChainVerifierWorker	1 回 (不一致検出)	運営者 high
AnnouncementSchedulerWorker	5 回 (同一 ID)	運営者 normal、対象 announcement を failed 遷移
DLQAutoBackoffWorker	3 回 / event	運営者 high (dlq_manual_replay 遷移時)

9.4.4 3.4 Cron UTC 一覧 (付録 L 同等)

Worker	UTC cron 式	JST 想定時刻	用途
MonthlyBillingCronWorker	0 17 * * * (毎日 17:00 UTC、Worker 内で JST 月初 1 日のみ実行)	翌月 02:00 JST	月次請求
AuditChainVerifierWorker	0 17 * * *	02:00 JST	ハッシュチェーン検証
AnnouncementSchedulerWorker	* * * * *	毎分	お知らせ予約配信
DLQAutoBackoffWorker	* / 5 * * * *	5 分間隔	DLQ 自動 BO
RetentionPurgeWorker	0 18 * * *	03:00 JST	保持期間別物理削除
R2AuditArchiveWorker	0 19 31 12 *	12/31 04:00 JST	年次 R2 アーカイブ
OperatorNotifyAggregatorWorker	* * * * *	毎分	FR-211 10 分集約処理

UTC ☒ JST 変換: JST = UTC + 9 時間。Cloudflare Cron Triggers は **L**(last day of month) をサポートしないため、月初・月末判定は Worker 内ガードで行う。

9.4.5 3.5 wrangler.toml 設定例

MonthlyBillingCronWorker の例:

```

name = "monthly-billing-cron"
main = "src/scheduler/monthly-billing.ts"
compatibility_date = "2026-01-01"

```

```

[triggers]
crons = ["0 17 * * *"] # 毎日 17:00 UTC、Worker 内で JST 月初 1 日のみ実行

```

```
[[d1_databases]]
binding = "DB"
database_name = "admin_db"
database_id = "<env-specific>"
```

```
[vars]
TIME_ZONE = "Asia/Tokyo"
```

9.4.6 3.6 cron 実装ガイドライン

項目	ガイドライン
冪等性	必ず冪等性キー (UNIQUE 制約またはアプリ層チェック) を持つ
実行時間制限	Cloudflare Workers の CPU 時間制限 (30 秒) に注意。大量処理はカーソルペー징 + 別 invocation へ分割
エラーハンドリング	try/catch で例外を吸収し、必ず audit_logs + notifyOperator を呼ぶ
再実行可能性	冪等性キーで重複実行を防ぐ。失敗ジョブは次回 cron tick で自動再試行 (Monthly-BillingCronWorker は 3 回まで)
監視	各 cron 開始時に <cron>.run.start、終了時に <cron>.run 監査ログを記録 (retention_class はジョブ性質に従う)

9.5 4. 関連設計

種別	参照先
要件	../01_ 要件定義/index.md
基本設計	../02_ 基本設計/index.md
課金・請求設計 (正本)	../02_ 基本設計/10_ 課金・請求設計.md
関連 DD	DD03_ 監査ハッシュチェーン.md / DD04_Stripe_Webhook 一次受信.md / DD06_ お知らせ承認・配信.md
運用設計	../04_ 運用設計/index.md
将来対応	../05_future/index.md

9.6 5. テスト観点

9.6.1 5.1 ユニットテスト

- ・ JST 判定ガード (`shouldRun()`) の閏年 / 12 月 → 1 月 境界 / DST 影響なし
- ・ 指数バックオフ計算 (1m / 4m / 16m)
- ・ カーソルペーjing境界
- ・ 冪等性キー競合検出

9.6.2 5.2 結合テスト (Miniflare)

- ・ 月初 cron で 100 オーナー分の請求書発行 → `billing_invoices` UNIQUE 制約検証
- ・ ハッシュチェーン検証で 1000 行検証 → 改ざんなし
- ・ お知らせ scheduler が 1 分以内に `scheduled` → `sent` 完了
- ・ DLQ auto BO で `failed` → `succeeded` または `failed` → `dlq_manual_replay`
- ・ 保持期間 purge で 1y / 5y / 7y の cutoff 計算

9.6.3 5.3 E2E テスト

- ・ 月次請求 cron の手動トリガ (`wrangler cron trigger`) で本番相当データ処理
- ・ ハッシュチェーン検証バッチの不一致シミュレーション

9.6.4 5.4 受入条件マッピング

AC	検証手段
AC-042(月次請求 cron 冪等)	統合テスト

9.7 6. 未確定事項・確認事項

確認事項 ID	確認内容	優先度	ステータス
-	v1.0 リリース時点で全項目確定済み	低	確認済

\newpage

10 DD09: 監査 action コード (運営者システム)

10.1 0. 文書情報

項目	内容
文書名	DD09: 監査 action コード (運営者システム)
詳細設計 ID	DD09
対象システム	FAQ AI ウィジェット SaaS / 運営者システム
関連機能 ID	TH-6, NFR-602, D-08
作成日	2026-05-17
版数	v1.0
ステータス	承認済

10.2 1. 対象範囲

種別	ID	名称
機能	TH-6	監査 action コード一覧の確定
テーブル	audit_logs.action	全 action コードの列挙
関連実装	admin/src/shared/action-codes.ts	TypeScript 列挙体

10.3 2. 収録ロジック・対応章

元章	元タイトル	概要
§ 15 全文	監査 action コード一覧 (★TH-6)	命名規約 + 全 action コード約 60 件 + 使用ルール
付録 F	3 区分保持の物理対応 (action_code × retention_class)	1y / 5y / 7y マップ

10.4 3. 詳細設計本文

<resource>.<verb> 命名規約で全 action コードを確定する。

10.4.1 3.1 命名規約

項目	規約
形式	<resource>.<verb> または <resource>.<sub-resource>.<verb>
文字	英小文字 + ハイフン + ドット + アンダースコア
動詞	過去形は使わない (create/update/delete/approve/reject/execute 等)

10.4.2 3.2 全 action コード一覧 (約 60 件)

action コード	retention	4-eyes 対象	payload スキーマ参照
owner.*			
owner.suspend	5y	MVP Log Only / Beta Hard Gate	{ownerAccountId, reason, suspendedAt}

action コード	retention	4-eyes 対象	payload スキーマ参照
owner.restore	5y	MVP Log Only / Beta Hard Gate	{ownerAccountId, restorationId, reason, rollback}
owner.physical_delete	5y	MVP Hard Gate	{ownerAccountId, slug, reason, ticketId}
owner.restore_data	5y	MVP Log Only / Beta Hard Gate	{ownerAccountId, resourceType, resourceId, reason, rollback}
owner.update	5y	-	{ownerAccountId, before, after}
operator.*			
operator.invite	5y	-	{invitedId, invitedBy, email}
operator.accept	5y	-	{operatorId}
operator.disable	5y	-	{operatorId, reason}
operator.session.revoke	5y	-	{operatorId, sessionId}
operator.login.attempt	1y	-	{email, success, ipMasked}
operator.login.success	5y	-	{operatorId, sessionId}
operator.login.failed	5y	-	{email, failedCount}
operator.lockout	5y	-	{accountId, lockedUntil}
operator.password.reset.request	5y	-	{email}
operator.password.reset	5y	-	{operatorId, approvalId}
operator.subrole.grant	5y	-	{operatorId, subrole}
operator.subrole.revoke	5y	-	{operatorId, subrole}
operator_approval.*			
operator_approval.request	5y	-	{approvalId, actionCode, payloadHash}
operator_approval.start	5y	review	{approvalId, reviewerId}
operator_approval.approve	5y	-	{approvalId, approvedBy}
operator_approval.reject	5y	-	{approvalId, rejectedBy, comment}
operator_approval.withdraw	5y	-	{approvalId, withdrawnBy, comment?}
operator_approval.execute	5y	-	{approvalId, executedBy, result}
operator_approval.expire	5y	-	{approvalId}
operator_mfa.*			
mfa.setup	5y	-	{operatorId}
mfa.verify	5y	-	{operatorId, success}
mfa.recovery_code.use	5y	-	{operatorId, codeIdMasked}
reauth	5y	-	{operatorId, reauthId}
operator_ip.*			
operator_ip.grant	5y	-	{operatorId, cidr}
operator_ip.revoke	5y	-	{operatorId, cidr}
ai_parameter.*			
ai_parameter.update	5y	MVP Hard Gate	{scope, scopeld, before, after, version}
ai_model.switch	5y	-	{from, to, rolloutPercentage}
rate_limit.* / budget_limit.*			
rate_limit.override	5y	MVP Log Only / Beta Hard Gate	{ownerAccountId, before, after, overrideId}
budget_limit.override	5y	MVP Log Only / Beta Hard Gate	{ownerAccountId, before, after, overrideId}
suppress_list.restore	5y	MVP Log Only	{ownerAccountId, email}
widget.force_stop	5y	MVP Log Only / Beta Hard Gate	{ownerAccountId, reason}
announcement.*			
announcement.create	5y	-	{announcementId, kind, severity, scope}
announcement.preview	5y	-	{announcementId}
announcement.schedule	5y	-	{announcementId, scheduledAt}
announcement.cancel	5y	-	{announcementId, cancelledBy}

action コード	retention	4-eyes 対象	payload スキーマ参照
announcement.test_send	5y	-	{announcementId, sentTo}
announcement.send	5y	-	{announcementId, recipients}
announcement.dispatch_by_dlq	5y	-	{announcementId, attemptCount, failedReason}(自動 BO 3 回失敗で永久失敗)
announcement.correct_by_issue	5y	-	{newAnnouncementId, correctionOf}
webhook.*			
webhook.receive	7y	-	{eventId, eventType, payloadHash}
webhook.replay	5y	-	{eventId, replayId, attemptedBy}
webhook.signature.invalidate	5y	-	{ipMasked, attemptedAt}
webhook.payload_diff_detect	5y	-	{diffId, eventId, originalHash, newHash}
webhook.payload_diff_review	5y	-	{diffId, reviewedBy}
webhook.payload_diff_reprocess	5y	-	{diffId, replayId}
webhook.payload_diff_dismiss	5y	-	{diffId, reason}
billing.*			
billing.invoice.issued	7y	-	{invoiceId, ownerAccountId, amount, billingYearMonth}
billing.invoice.finalized	7y	-	{invoiceId}
billing.credit_note.issue	7y	-	{creditNoteId, invoiceId, amount, reason}
billing.cron.run	7y	-	{month, success, failed}
pricing.update	7y	MVP Log Only / Beta Hard Gate	{pricingVersion, before, after, effectiveFrom}
stripe.*			
stripe.event.processed	7y	-	{eventId, eventType}
stripe.subscription.resume	5y	-	{subscriptionId, ownerAccountId}
pii.*			
pii_fp_report.create	1y	-	{reportId, detectionLayer}
pii_fp_report.transition	5y	-	{reportId, from, to}
pii_rule.update	5y	-	{revisionId, rolloutPercentage}
audit.*			
audit.export	5y	-	{exportId, format, rows}
audit.search	1y	-	{operatorId, filter, hitCount, ticketId}(APPI 30 条: 監査ログ閲覧自体の記録)
audit.chain.verify	5y	-	{verifyDate, runId, mismatchCount}
audit.chain.verify.fail	5y	-	{runId, rowId, expected, actual}
master_key.*			
master_key.rotate	5y	MVP Hard Gate	{rotationId, oldKeyId, newKeyId}
master_key.emergency_bypass	5y	-	{bypassId, reason, witnesses, ticketId}(RB-014 MVP の紙ベース回復コード使用記録)
retention.*			
retention.purge.run	5y	-	{runId, retentionClass, deletedCount}
retention.archive.run	5y	-	{archiveYear}
prod.*			
prod.direct_change	5y	-	{operatorId, query, ticketId}
mail.*			
mail.suppress.add	5y	-	{email, reason}
mail.suppress.remove	5y	-	{email, approvalId}
mail.footer.update	5y	-	{before, after, operatorId}(特定電子メール法フッタ設定変更)
feature.*			
feature.hard_gate.toggle	5y	MVP Log Only / Beta Hard Gate	{actionCode, before, after, reason}(ハードゲート KV フラグ切替)
monitoring.*			
monitoring.threshold.update	5y	-	{kpId, before, after, reason}(監視 KPI しきい値変更)

10.4.3 3.3 action コード使用ルール

1. 新規 action コード追加時は **本表に追記 + src/shared/action-codes.ts** 定数を更新する。
2. CI で本表と実装の整合性を検証 (grep ベース + TypeScript 型)。
3. 既存 action コードの retention_class 変更は RB-018 に従う。
4. 4-eyes 対象列の変更 (ハードゲート ☒ Log Only) は KV **feature:hard-gate:<action>** の操作で行い、本表の備考を更新する。

10.4.4 3.4 3 区分保持の物理対応 (action_code × retention_class)

§ 3.2 + [DD03_ 監査ハッシュチェーン.md](#) § 3.7 と整合した完全列挙:

10.4.4.1 3.4.1 1 年保持 (NFR-602(a) 業務監査)

action_code	主管
faq.create / faq.update / faq.publish / faq.unpublish / faq.delete	メイン
chat.reply / chat.close / chat.reopen	メイン
account.login / account.logout	メイン + 本書 (本書側は operator.login.*)
inbox.read	メイン + 本書
announcement.preview	本書
announcement.test_send	本書
pii_fp_report.create	本書

10.4.4.2 3.4.2 5 年保持 (NFR-602(b) 運営者高権限)

action_code	4-eyes
owner.suspend / owner.restore / owner.physical_delete / owner.update	☒ (physical_delete のみハードゲート)
ai_parameter.update / ai_model.switch	☒ ハードゲート
rate_limit.override / budget_limit.override / suppress_list.restore / widget.force_stop	☒ (Beta から)
announcement.create / announcement.schedule / announcement.cancel / announcement.send / announcement.correction.issue	-
operator.* 全般 (invite / accept / disable / session.revoke / login.success / login.failed / logout / password.reset.request / password.reset / subrole.grant / subrole.revoke)	-
operator_approval.* 全般 (request / start_review / approve / reject / withdraw / execute / expire)	-
mfa.setup / mfa.verify / mfa.recovery_code.used / reauth	-
operator_ip.grant / operator_ip.revoke	-
webhook.replay / webhook.signature.invalid / webhook.payload_diff.*	-
pii_fp_report.transition / pii_rule.update	-
audit.export / audit.chain.verify / audit.chain.verify.fail	-
master_key.rotate	☒ ハードゲート
retention.purge.run / retention.archive.run	-
prod.direct_change	-
mail.suppress.add / mail.suppress.remove	-

10.4.4.3 3.4.3 7 年保持 (NFR-602(c) 課金・取引)

action_code
billing.invoice.issued / billing.invoice.finalized
billing.credit_note.issued
billing.cron.run
stripe.event.processed / stripe.subscription.resume
webhook.receive

10.4.5 3.5 retention_class 別の物理削除契機

retention_class	DELETE 契機	担当 Worker
general(1y)	occurred_at < today - 365	RetentionPurgeWorker(日次 03:00 JST)
operator_high_priv(5y)	occurred_at < today - 1825	同上
billing(7y)	occurred_at < today - 2555	同上

物理削除前に R2 アーカイブ (`audit-archive/<retention_class>/<year>/<month>.tar.gz`、年次 12/31) へ書出。詳細は [DD08_Cron 実装.md](#) § 3.2.5 / § 3.2.6 を参照。

10.4.6 3.6 action コード追加プロセス

1. 本表に新規 action コード追加 + `retention_class` + 4-eyes 対象有無を決定
2. `admin/src/shared/action-codes.ts` の TypeScript 列挙体を更新
3. CI(`03_script/check-action-codes.sh`、TODO: 整備中) で本表と実装の差分検出
4. 関連 DD(認証 / 4-eyes / 監査 / 機能領域) に追記
5. `feature.hard_gate.toggle` で運用ハードゲート切替時は KV `feature:hard-gate:<action>` の状態と本表の備考を整合

10.4.7 3.7 監査記録の必須項目

すべての action 記録時に次のフィールドを必ず設定:

カラム	内容
actor_id	操作者 ID(運営者 ID または system)
action	本表の action コード
target_type / target_id	操作対象のリソース型 + ID
payload	JSON、本表「payload スキーマ参照」列の通り
payload_hash	sha256(canonical_json(payload))
retention_class	本表第 2 列 (general / operator_high_priv / billing)
prev_hash	チェーン直前レコードの record_hash
record_hash	HMAC-SHA256 計算結果
occurred_at	UTC ISO 8601
trace_id	リクエストトレース ID(構造化ログと一致)
ticket_id	クリティカル操作のチケット ID(任意)
approval_id	4-eyes 経由実行時の operator_approvals.id(任意)

完全な DDL は [基本設計 / テーブル設計](#) を正本とする。

10.5 4. 関連設計

種別	参照先
要件	../01_要件定義/index.md
基本設計	../02_基本設計/index.md
テーブル設計(正本)	../02_基本設計/03_テーブル設計.md
セキュリティ設計(正本)	../02_基本設計/09_セキュリティ設計.md
関連 DD	DD03_監査ハッシュチェーン.md / DD02_4-eyes 承認フロー.md / DD08_Cron 実装.md
運用設計	../04_運用設計/index.md
将来対応	../05_future/index.md

10.6 5. テスト観点

10.6.1 5.1 ユニットテスト

- 本表と `src/shared/action-codes.ts` の TypeScript 列挙体一致 (snapshot test)

- ・各 action コードの payload スキーマバリデーション
- ・retention_class 値の正当性 (general / operator_high_priv / billing のみ)

10.6.2 5.2 結合テスト (Miniflare)

- ・各 action コード発火 → audit_logs 1行 INSERT + payload_hash 確認
- ・retention_class 別 cutoff 計算 (general = 365 / operator_high_priv = 1825 / billing = 2555)
- ・4-eyes 対象 action(ハードゲート 3 操作) の approval_id 必須化

10.6.3 5.3 CI 整合性チェック

- ・本表に存在しない action コード使用で CI fail
- ・実装に存在しない action コードが本表にある場合 CI warning
- ・retention_class 変更時は RB-018 手順書のリンクが必要

10.6.4 5.4 月次運用テスト

- ・ランダム抽出 10 件で record_hash 再計算 → 一致確認
- ・retention 物理削除前の最終件数 + R2 アーカイブとの突合

10.7 6. 未確定事項・確認事項

確認事項 ID	確認内容	優先度	ステータス
-	v1.0 リリース時点で全項目確定済み	低	確認済

\newpage

11 DD10: PII 偽陽性報告 (運営者システム)

11.1 0. 文書情報

項目	内容
文書名	DD10: PII 偽陽性報告 (運営者システム)
詳細設計 ID	DD10
対象システム	FAQ AI ウィジェット SaaS / 運営者システム
関連機能 ID	FR-060, FR-064, NFR-805, AC-036, D-13
作成日	2026-05-17
版数	v1.0
ステータス	承認済

11.2 1. 対象範囲

種別	ID	名称
機能	FR-060	PII 第 1 層検出 (正規表現)
機能	FR-064	PII 誤検出報告フロー
受入条件	AC-036	PII 3 層 + 報告フロー
画面	SCR-098	PII 誤検出報告管理 (3 営業日判定)
API	POST /pii-fp-reports	報告作成
API	POST /pii-fp-reports/{id}/transition	状態遷移 (運営者)
API	POST /pii-rules/revisions	ルール改定 (段階ロールアウト)
テーブル	pii_false_positive_reports	報告履歴
テーブル	pii_rule_revisions	ルール改定履歴

11.3 2. 収録ロジック・対応章

元章	元タイトル	概要
§ 5 SCR-098	PII 誤検出報告管理画面	3 営業日タイマー + ルール改定
§ 6 PII 機能(参照)	報告 → 判定 → ルール更新 → KV 即時反映	D-13
付録 B.5	pii_false_positive_reports.state	6 状態遷移

11.4 3. 詳細設計本文

11.4.1 3.1 PII 検出層 (参考)

メイン側で実装される 3 層検出のうち、本書側で運営する対象:

層	検出方式	主管	本書側関与
第 1 層	正規表現 (電話番号 / メール / 住所 / クレカ 等)	メイン実装、ルール本書管理	KV pii-rules:regex 更新 + ロールアウト
第 2 層	NER 分類器 (MVP では機能フラグ OFF)	将来対応	KV feature:pii-layer2:enabled 制御
第 3 層	FAQ 整合性検査 (LLM)	メイン	-

詳細は [基本設計 / セキュリティ設計](#) § PII を正本とする。

11.4.2 3.2 状態遷移 (pii_false_positive_reports.state)

From	To	トリガー
-	reported	報告転送
reported	under_review	SCR-098 開始 (3 営業日タイマー起動)
under_review	ruled_false_positive	判定
under_review	ruled_correct_detection	判定
ruled_false_positive	rule_updated	KV ルール更新
-	archived	90 日経過 or rule_updated 後

詳細は [DD11_状態遷移詳細.md](#) § B.5 を参照。

11.4.3 3.3 3 営業日判定タイマー (NFR-805)

`under_review` 遷移時点から **3 営業日** (JST、土日祝日除外) 以内に `ruled_*` 判定を行う必要がある。超過時に運営者 normal alert。

項目	仕様
営業日カウント	JST タイムゾーンで月～金、祝日マスタ (D-05、年次 11/1 取得) を除外
タイマー駆動	1 時間 cron (OperatorNotifyAggregatorWorker 内のサブタスク) で <code>under_review</code> 状態かつ <code>last_transition_at + 3 営業日 < now</code> を検出
通知	KV monitoring:thresholds:pii-fp-pending-3bd = 0 を超えると <code>pii_fp_report.transition.overdue</code> を運営者 normal で通知

11.4.4 3.4 PII 偽陽性報告作成 API

```
/pii-fp-reports:
  post:
    summary: PII 誤検出報告作成
    parameters:
      - $ref: "#/components/parameters/IdempotencyKey"
      - $ref: "#/components/parameters/XCsrfToken"
    requestBody:
      required: true
```

```

content:
  application/json:
    schema:
      type: object
      required: [detectionLayer, sampleText, reason]
      properties:
        detectionLayer: { type: string, enum: [regex, classifier, llm] }
        sampleText: { type: string }
        reason: { type: string }
    responses:
      "201":
        content: { application/json: { schema: { type: object, properties: { reportId: { type:

```

通常はエンドユーザーまたは管理者ユーザー側から連携 IF #8 / #9 経由で報告が転送される。detectionLayer でどの層の誤検出かを区別する。

11.4.5 3.5 PII ルール改定 API(段階ロールアウト、D-13)

```

/pii-rules/revisions:
  post:
    summary: PII ルール改定 (段階ロールアウト)
    parameters:
      - $ref: "#/components/parameters/XOpTicketId"
      - $ref: "#/components/parameters/IdempotencyKey"
      - $ref: "#/components/parameters/XCsrftoken"
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required: [layer, rules, rolloutPercentage]
            properties:
              layer: { type: string, enum: [regex, classifier] }
              rules: { type: array, items: { type: object } }
              rolloutPercentage: { type: integer, enum: [0, 10, 50, 100] }
    responses:
      "201":
        content: { application/json: { schema: { type: object, properties: { revisionId: { type:

```

項目	仕様
反映方式 (D-13)	KV pii-rules:regex / pii-rules:classifier を即時更新、TTL 60 秒で全 Worker に伝播
過去データ修正	しない (過去の検出結果は新ルールでは再評価しない)
段階ロールアウト	feature:pii-rule-rollout:<revision_id> で 0% / 10% / 50% / 100% 制御
監査	pii_rule.update(5y、{revisionId, rolloutPercentage})
報告状態遷移	ルール改定後、対応する pii_false_positive_reports.state を rule_updated に遷移

11.4.6 3.6 SCR-098 主要機能

機能	説明
報告一覧	reported / under_review のフィルタ + 3 営業日タイマー表示 (色分け: 緑 = 余裕 / 黄 = 1 日以内 / 赤 = 超過)
報告詳細	sampleText (PII マスク済) + reason + detectionLayer + 過去判定履歴
状態遷移ボタン	under_review → ruled_false_positive / ruled_correct_detection
ルール改定モダール	ruled_false_positive 後に直接「該当ルールを修正する」モダールを呼び出し
ロールアウト進捗	KV feature:pii-rule-rollout:<revision_id> の現値を確認 + 段階更新

11.4.7 3.7 ルール改定例

正規表現ルール例:

```
{
  "layer": "regex",
  "rules": [
    {
      "id": "phone-jp-mobile",
      "pattern": "0[789]0[-\\s]?\\d{4}[-\\s]?\\d{4}",
      "enabled": true,
      "description": " 日本の携帯電話番号"
    },
    {
      "id": "credit-card",
      "pattern": "\\d{4}[-\\s]?\\d{4}[-\\s]?\\d{4}[-\\s]?\\d{4}",
      "enabled": true,
      "description": " クレジットカード番号 (MOD10 検証は省略、第 3 層で実施)"
    }
  ],
  "rolloutPercentage": 10
}
```

11.4.8 3.8 ロールアウト判定アルゴリズム (メイン側で参照)

メインの PII 検出 Worker は次の判定で新ルール採用率を決定する:

```
function shouldUseNewRevision(revisionId: string, ownerAccountId: string): boolean {
  const rollout = KV.get(`feature:pii-rule-rollout:${revisionId}`);
  if (!rollout || rollout.percentage === 0) return false;
  if (rollout.percentage === 100) return true;
  const hash = sha256(`${revisionId}:${ownerAccountId}`);
  const bucket = parseInt(hash.substring(0, 4), 16) % 100; // 0..99
  return bucket < rollout.percentage;
}
```

ハッシュベース判定により、同一オーナーは常に同じバケットに割り当てられる (段階上げでサンプルが移動しない安定性)。

11.4.9 3.9 PII マスキング (報告データ)

報告本文 (sampleText) に含まれる PII は永続化前に伏字化:

種別	マスク方式	例
電話番号	中央 4 桁 ****	090-****-1234
メール	local-part の最初 2 文字以外 ***	ab***@example.com
クレカ	中央 8 桁 ****-****	4111-****-****-1111
住所	番地以下削除	東京都千代田区

メイン側で実装する PII マスク関数を利用。マスク前の生データは保存しない。

11.4.10 3.10 主要エラーコード

エラー ID	HTTP	説明
E-OP-PII-001	409	INVALID_STATE_TRANSITION(reported → rule_updated 等、許可されない直接遷移)
E-OP-PII-002	400	INVALID_RULE_REGEX(正規表現コンパイル失敗)
E-OP-PII-003	422	ROLLOUT_PERCENTAGE_INVALID(0/10/50/100 以外)
E-OP-PII-004	410	REPORT_ARCHIVED(90 日経過、操作不可)

完全な E-OP-PII-* 一覧は [基本設計 / エラー設計](#) を正本とする。

11.5 4. 関連設計

種別	参照先
要件	../01_要件定義/index.md
基本設計	../02_基本設計/index.md
セキュリティ設計 (正本)	../02_基本設計/09_セキュリティ設計.md
API 設計 (正本)	../02_基本設計/02_API 設計.md
メインテーブル設計	../01_メインシステム/02_基本設計/03_テーブル設計.md
関連 DD	DD07_KV・R2 オブジェクト.md / DD11_状態遷移詳細.md
運用設計	../04_運用設計/index.md
将来対応	../05_future/index.md

11.6 5. テスト観点

11.6.1 5.1 ユニットテスト

- ・ 状態遷移ガード (**reported** → **rule_updated** 直接遷移拒否)
- ・ 3 営業日計算 (JST + 祝日マスタ + 週末除外)
- ・ 正規表現コンパイル検証
- ・ ハッシュベースロールアウト判定 (同一 ownerId は同バケット)
- ・ PII マスキング全種別

11.6.2 5.2 結合テスト (Miniflare)

- ・ 報告作成 → **under_review** → **ruled_false_positive** → ルール改定 → **rule_updated**
- ・ KV **pii-rules:regex** 更新後 60 秒以内のメイン側参照で反映
- ・ 3 営業日超過で通知発火
- ・ 90 日経過で **archived** 自動遷移

11.6.3 5.3 E2E テスト (Playwright)

テスト ID	シナリオ
e2e-scr098-001	報告詳細 → 判定 → ルール改定モダール → ロールアウト 10%

11.6.4 5.4 品質回帰

データセット	件数
PII 第 1 層	陽性 100 + 陰性 100 / 各国内パターン

11.6.5 5.5 受入条件マッピング

AC	検証手段
AC-036(PII 3 層 + 報告フロー)	単体 + 報告 E2E + 都度運用

11.7 6. 未確定事項・確認事項

確認事項 ID	確認内容	優先度	ステータス
-	v1.0 リリース時点で全項目確定済み	低	確認済

\newpage

12 DD11: 状態遷移詳細 (運営者システム)

12.1 0. 文書情報

項目	内容
文書名	DD11: 状態遷移詳細 (運営者システム)
詳細設計 ID	DD11
対象システム	FAQ AI ウィジェット SaaS / 運営者システム
関連機能 ID	D-08, AC-041, FR-302, FR-188, FR-189, FR-226, FR-064
作成日	2026-05-17
版数	v1.0
ステータス	承認済

12.2 1. 対象範囲

種別	ID	名称
テーブル	webhook_events.state	12 状態
テーブル	operator_approvals.state	7 状態
テーブル	announcement_drafts.state	8 状態
テーブル	pii_false_positive_reports.state	6 状態
テーブル	webhook_payload_diffs.state	4 状態
テーブル	accounts.contract_status(参照、メイン主管)	オーナー行 4 値

12.3 2. 収録ロジック・対応章

元章	元タイトル	概要
付録 B 全文	状態遷移詳細表 (本書版)	5 状態機械の遷移表
§ 4	状態詳細設計 (参照)	個別設計書の正本リンク

12.4 3. 詳細設計本文

12.4.1 3.1 webhook_events.state(12 状態、AC-041)

From	To	トリガー	副作用
-	received	POST 受信	row INSERT
received	verifying_signature	即時	-
verifying_signature	rejected	HMAC NG	401 + high alert
verifying_signature	checking_idempotency	HMAC OK	payload_hash 計算
checking_idempotency	processing	未処理	INSERT + R2 退避
checking_idempotency	duplicate_skipped_hash_match	既処理 + 一致	200 + ログ
checking_idempotency	duplicate_diff_detected_high_alert	既処理 + 不一致	webhook_payload_diffs INSERT、SCR-099 待ち
processing	succeeded	IF #10 200	audit:stripe.event.processed(7y)
processing	failed	5xx/Timeout	DLQ 投入
failed	dlq_retrying	DLQAutoBackoffWork	指数 BO
dlq_retrying	succeeded	再試行成功	-
dlq_retrying	dlq_manual_replay	1h 経過	運営者 high
dlq_manual_replay	succeeded	SCR-097 リプレイ	audit:webhook.replay(5y)
dlq_manual_replay	dlq_archived	30 日経過	リプレイ不可

主管担当: [DD04_Stripe_Webhook 一次受信.md](#)。

12.4.2 3.2 operator_approvals.state(7 状態、FR-226)

From	To	トリガー	ガード
-	requested	POST /approvals	expires_at = +72h
requested	reviewing	start-review	requested_by ≠ reviewer
reviewing	approved	approve	DB CHECK 自己承認禁止
reviewing	rejected	reject	requested_by ≠ rejected_by、コメント必須
requested	withdrawn	withdraw(申請者本人)	withdrawn_by == requested_by(自己取下げ)
requested/reviewing	expired	72h 経過	バッチ
approved	executed	execute	now < approved_at + 72h、payload_hash 一致
approved	expired	72h 経過	バッチ

主管担当: [DD02_4-eyes 承認フロー.md](#)。

12.4.3 3.3 announcement_drafts.state(8 状態、FR-188 / FR-189)

From	To	トリガー
-	draft	POST /announcements
draft	preview	preview
preview	scheduled	schedule(再認証 + チケット必須)
scheduled	sending	AnnouncementSchedulerWorker(scheduled_at ☒ now+5min)
scheduled	cancelled	cancel(now < scheduled_at - 5min)
sending	sent	連携 IF #7 200 OK
sending	failed	連携 IF #7 一時失敗 (5xx/Timeout)、attempt_count++
failed	sending	自動指数 BO(1m → 4m → 16m、最大 3 回)
failed	dlq	自動 BO 3 回失敗、運営者 inbox high

主管担当: [DD06_お知らせ承認・配信.md](#)。

12.4.4 3.4 pii_false_positive_reports.state(6 状態、FR-064)

From	To	トリガー
-	reported	報告転送
reported	under_review	SCR-098 開始 (3 営業日タイマー起動)
under_review	ruled_false_positive	判定
under_review	ruled_correct_detection	判定
ruled_false_positive	rule_updated	KV ルール更新
-	archived	90 日経過 or rule_updated 後

主管担当: [DD10_PII 偽陽性報告.md](#)。

12.4.5 3.5 webhook_payload_diffs.state(4 状態、AC-041 異常系)

From	To	トリガー
-	detected	BillingWebhookWorker が差分検出
detected	reviewed	start-review
reviewed	reprocessed_manually	reprocess(再認証 + チケット必須)
reviewed	dismissed_no_action	dismiss(理由必須)

主管担当: [DD04_Stripe_Webhook 一次受信.md](#) § 3.9。

12.4.6 3.6 accounts.contract_status(参考、メイン主管)

オーナー行のみで意味を持つ 4 値:

値	説明	遷移経路
active	通常稼働	初期値 + owner.restore_data で復活
suspended	一時停止	owner.suspend(運営者)
deleted_pending	退会受付済、猶予期間中 (30 日)	利用者の退会操作
deleted	物理削除済 (accounts_retired 移行)	owner.physical_delete(ハードゲート、運営者)

メイン主管。本書側は SCR-090 / SCR-091 で参照表示のみ。詳細は [メインシステム / contract_status 設計](#) を正本参照。

12.4.7 3.7 状態遷移実装ガイドライン

項目	ガイドライン
楽観ロック	全状態テーブルに version 列を持ち、UPDATE ... WHERE id=? AND version=? AND state=? パターンで競合検出
影響行数チェック	UPDATE 後に rowsAffected=0 で 409 Conflict 返却
監査記録	状態遷移ごとに対応 action コード (DD09_監査 action コード.md) を audit_logs に必ず記録
DB CHECK 制約	多重ガードのため、可能な状態値を CHECK 制約で列挙 (CHECK (state IN ('requested','reviewing','approved','rejected','withdrawn','executed','expired')))
バッチ駆動遷移	expired / archived / dlq_archived 等の時間経過遷移は専用 cron Worker で実行
観察可能性	各状態の在庫数 (現在のレコード数) を KPI として monitoring:thresholds:<kpi_id> で監視

12.4.8 3.8 状態遷移共通エラーコード

エラー ID	HTTP	説明
E-OP-STATE-001	409	INVALID_STATE_TRANSITION(許可されない from → to ペア)
E-OP-STATE-002	409	VERSION_CONFLICT(楽観ロック競合)
E-OP-STATE-003	410	RESOURCE_ARCHIVED(archived / dlq_archived 状態は操作不可)
E-OP-STATE-004	423	RESOURCE_LOCKED(executing 等で並行更新拒否)

各機能固有のエラー ID(E-OP-ANN-* / E-OP-WEBHOOK-* 等) は各 DD を参照。

12.4.9 3.9 状態テーブル一覧

テーブル	状態列	状態数	主管 DD
webhook_events	state	12	DD04
operator_approvals	state	7	DD02
announcement_drafts	state	8	DD06
pii_false_positive_reports	state	6	DD10
webhook_payload_diffs	state	4	DD04
restorations	state	3(executing / succeeded / failed)	DD05
accounts(参照)	contract_status	4	メイン主管
operator_accounts	lifecycle_state	4(invited / active / disabled / revoked)	DD01

すべての完全な DDL + CHECK 制約は [基本設計 / テーブル設計](#) を正本とする。

12.4.10 3.10 状態遷移サマリ

```
stateDiagram-v2
    state webhook_events {
        [*] --> received
```

```

received --> verifying_signature
verifying_signature --> rejected: HMAC NG
verifying_signature --> checking_idempotency: HMAC OK
checking_idempotency --> processing: 未処理
checking_idempotency --> duplicate_skipped_hash_match: 既処理 + 一致
checking_idempotency --> duplicate_diff_detected_high_alert: 既処理 + 不一致
processing --> succeeded: IF#10 200
processing --> failed: 5xx/Timeout
failed --> dlq_retrying: DLQAutoBackoff
dlq_retrying --> succeeded
dlq_retrying --> dlq_manual_replay: 1h 経過
dlq_manual_replay --> succeeded: 手動リプレイ
dlq_manual_replay --> dlq_archived: 30 日経過
}

```

stateDiagram-v2

```

state operator_approvals {
    [*] --> requested: POST /approvals
    requested --> reviewing: start-review
    requested --> withdrawn: withdraw
    requested --> expired: 72h
    reviewing --> approved: approve
    reviewing --> rejected: reject
    reviewing --> expired: 72h
    approved --> executed: execute
    approved --> expired: 72h
}

```

stateDiagram-v2

```

state announcement_drafts {
    [*] --> draft: POST /announcements
    draft --> preview
    preview --> scheduled: schedule
    scheduled --> sending: cron tick
    scheduled --> cancelled: cancel
    sending --> sent: IF#7 200
    sending --> failed: 一時失敗
    failed --> sending: 自動 B0
    failed --> dlq: 3 回失敗
}

```

stateDiagram-v2

```

state pii_false_positive_reports {
    [*] --> reported
    reported --> under_review: SCR-098 開始
    under_review --> ruled_false_positive
    under_review --> ruled_correct_detection
    ruled_false_positive --> rule_updated: KV 更新
    ruled_correct_detection --> archived: 90 日
    rule_updated --> archived: 即時
}

```

stateDiagram-v2

```

state webhook_payload_diffs {
    [*] --> detected
    detected --> reviewed: start-review
    reviewed --> reprocessed_manually: reprocess
    reviewed --> dismissed_no_action: dismiss
}

```

12.5 4. 関連設計

種別	参照先
要件	../01_ 要件定義/index.md
基本設計	../02_ 基本設計/index.md
テーブル設計 (正本)	../02_ 基本設計/03_ テーブル設計.md
メインテーブル設計	../01_ メインシステム/02_ 基本設計/03_ テーブル設計.md
関連 DD	DD02_4-eyes 承認フロー.md / DD04_Stripe_Webhook 一次受信.md / DD06_ お知らせ承認・配信.md / DD10_PII 偽陽性報告.md
運用設計	../04_ 運用設計/index.md
将来対応	../05_future/index.md

12.6 5. テスト観点

12.6.1 5.1 ユニットテスト

- ・各状態の許可遷移ペア網羅 (マトリクステーブル)
- ・拒否ペア (**draft** → **sent** 等) で 409 返却
- ・楽観ロック競合 (同一 from で 2 並列 UPDATE → 1 件成功 / 1 件 409)
- ・バッチ駆動遷移 (72h / 90 日 / 30 日 / 1h) の境界

12.6.2 5.2 結合テスト (Miniflare)

- ・各状態機械の完全パス (happy path)
- ・失敗パス (**rejected** / **failed** / **dlq** / **archived** 全件)
- ・バッチ実行で **expired** / **archived** 遷移
- ・状態遷移時の **audit_logs** 必須記録 (対応 action コード)

12.6.3 5.3 E2E テスト (Playwright)

各 DD の E2E シナリオ (**e2e-scr090-001** ~ **e2e-scr099-001**、**e2e-4eyes-***-001 等) で全状態を踏破。

12.6.4 5.4 状態遷移マトリクス検証 CI

- ・本書の遷移表と実装 (state machine モジュール) の整合性を **03_script/check-state-machine.sh** で検証 (TODO: スクリプト整備)
- ・各 state テーブルの CHECK 制約と本書記載の値リストの一致

12.7 6. 未確定事項・確認事項

確認事項 ID	確認内容	優先度	ステータス
-	v1.0 リリース時点で全項目確定済み	低	確認済

\newpage

13 DD12: 運営者用ログ・テスト (運営者システム)

13.1 0. 文書情報

項目	内容
文書名	DD12: 運営者用ログ・テスト (運営者システム)
詳細設計 ID	DD12
対象システム	FAQ AI ウィジェット SaaS / 運営者システム
関連機能 ID	TH-11, NFR-103, NFR-105, NFR-106, AC-034, AC-036, AC-037, AC-038, AC-040, AC-041, AC-042, AC-044, AC-046
作成日	2026-05-17
版数	v1.0

項目	内容
ステータス	承認済

13.2 1. 対象範囲

種別	ID	名称
機能	TH-11	エラーログ詳細 (構造化ログ)
テスト	ユニット / 結合 / E2E / Webhook 統合 / 負荷 / ペネトレ	全テストレベル
受入条件	AC-034~046	本書側主管 AC + メイン共同 AC

13.3 2. 収録ロジック・対応章

元章	元タイトル	概要
§ 16	エラーログ・構造化ログ(★TH-11)	エラー ID 体系 + 構造化ログスキーマ (個別設計書参照)
§ 17 全文	テスト戦略・受入条件マッピング	テスト種別 × 責務 + AC マッピング + 負荷シナリオ + 品質回帰データセット
付録 K	構造化ログ JSON Schema	実装ファイルとの一致要求

13.4 3. 詳細設計本文

13.4.1 3.1 構造化ログ JSON Schema(★TH-11)

すべての Worker は標準出力に JSON 1 行形式で構造化ログを出力する。Logpush / Logflare 等の集約基盤に送出され、Trace ID で横串集計可能。

```
{
  "level": "debug | info | warn | error | critical",
  "timestamp": "2026-05-12T10:00:00Z",
  "worker": "AdminConsoleWorker",
  "request_id": "<cf-ray>",
  "trace_id": "01J...",
  "operator_id": "01J... | null",
  "owner_account_id": "01J... | null",
  "action": "<action_code> | null",
  "endpoint": "/admin/api/v1/...",
  "method": "GET | POST | PUT | DELETE",
  "status_code": 200,
  "duration_ms": 123,
  "error_code": "E-OP-... | null",
  "error_message": "...",
  "stack_trace": "..."
}
```

必須フィールド: level、timestamp、worker、request_id。任意フィールド: trace_id、operator_id、owner_account_id、action、endpoint、method、status_code、duration_ms、error_code、error_message、stack_trace。

実装ファイル admin/src/shared/logger.ts の型定義と一致させること。

13.4.2 3.2 機密項目マスキング

ログ出力前に必ず次のフィールドをマスクする:

フィールド	マスキング
Authorization ヘッダ	完全削除
X-CSRF-Token	最初の 4 文字 + ***
パスワード / MFA コード / リカバリーコード	完全削除
メール本文 PII / 電話番号 / クレカ	DD10 と同方式でマスク

フィールド	マスキング
Stripe customer.id	末尾 4 文字 + *** (法令対応で完全削除は避ける)
IP アドレス	/24(IPv4) // 48(IPv6) 単位で切り捨て

13.4.3 3.3 エラー ID 体系

エラー ID 体系の正本は [基本設計 / エラー設計](#)。E-OP-* 87 件、エラー分類 13 種、異常系共通方針、critical レベルエラーは当該ドキュメントを参照。

主な接頭辞:

接頭辞	範囲	DD 参照
E-OP-AUTH-*	認証 / 認可	DD01
E-OP-4EYES-*	4-eyes	DD02
E-OP-AUDIT-*	監査ログ	DD03
E-OP-WEBHOOK-*	Webhook	DD04
E-OP-RESTORE-*	削除データ復元	DD05
E-OP-ANN-*	お知らせ	DD06
E-OP-PII-*	PII 報告	DD10
E-OP-STATE-*	状態遷移共通	DD11

13.4.4 3.4 RFC 7807 problem+json 形式

エラーレスポンスは RFC 7807 に準拠:

```
{
  "type": "https://admin.open-faq.example.com/errors/E-OP-AUTH-005",
  "title": "Re-authentication required",
  "status": 403,
  "code": "E-OP-AUTH-005",
  "detail": "Critical operation requires re-authentication within 5 minutes",
  "trace_id": "01J...",
  "instance": "/admin/api/v1/owners/01J.../suspend"
}
```

type フィールドは code フィールドと同一のエラー ID を URL 末尾に持ち、運営者がドキュメントへ到達可能にする。

13.4.5 3.5 テスト種別 × 責務 (基本設計 §14.1 準拠)

種別	範囲	ツール	実施タイミング	責務
ユニット	Workers ハンドラ・ドメインロジック・SLA 計測・ハッシュチェーン計算	Vitest	コミット毎	本書
結合	D1/R2/KV を含む、運営者認証・4-eyes 承認・Webhook 受信	Miniflare + Vitest	PR 毎	本書
E2E(運営画面)	SCR-090~099 主要フロー、4-eyes 承認、削除データ復元、お知らせ配信	Playwright	nightly + リリース前	本書
Webhook 統合	Stripe 署名検証、event_id 冪等、ペイロード差分検出、DLQ 自動 BO、手動リプレイ	カスタム + Stripe Test Mode	リリース前 + 月次	本書
AI 品質回帰	FAQ × 想定質問ペア (MVP 50)	カスタムパッチ	モデル更新時 + プロンプト変更時	データセット = 本書、実行 = メイン CI
プロンプト注入回帰	20 ケース以上	カスタム	モデル更新時 + 四半期	テストセット = 本書、実行 = メイン CI
ペネトレーション	全エンドポイント (運営者プレーン重点)	外部委託	年 1 回 + 重大機能変更時	共同 (本書側で実施記録 5 年保持)
オーナー境界によるデータ分離	クロス契約アクセス試行 (自動 50 + 手動 5)	カスタム	月次自動 + 年次手動	メイン主管 + 本書側で運営者横断クエリ境界

種別	範囲	ツール	実施タイミング	責務
負荷試験	監査ログ検索(100万件以下)/ エクスポート(10万行/60s)/ 4-eyes 承認 API	k6	月次 + 主要リリース前	本書
4-eyes E2E	申請 → 承認 → 実行 → 失敗ロールバック	Playwright	nightly	本書

カバレッジ目標 (MVP リリース判定で達成必須):

種別	目標値	補足
ユニット statements	☑ 80%	SLA 計測 / ハッシュチェーン計算は ☑ 90%
ユニット branches	☑ 75%	状態遷移ガード系は ☑ 90%
結合 (Miniflare)	API エンドポイント網羅率 100%	認可境界・状態遷移 1 件以上
E2E(Playwright)	SCR-090~099 各画面で主要フロー 1 件以上 + 4-eyes 完全フロー	—
AC マッピング	§ 3.8 受入条件すべてに対応するテスト ID	AC 識別子をテスト記述に埋め込み

負荷試験シナリオ詳細 (本書主管):

シナリオ	構成	目的
(A1) 監査ログ検索負荷	audit_logs 100 万件、SCR-096 検索 50 RPS	p95 ☑ 1000ms 検証
(A2) 監査ログエクスポート	10 万行/ファイル、5 並列実行	☑ 60s + メモリ上限遵守
(A3) 4-eyes 承認バースト	同時申請 50 件 / 承認 50 件	楽観ロック・状態遷移整合性
(A4) Stripe Webhook 受信	100 event/分 + 10% で差分検出	冪等性 + DLQ 挙動

合格基準: 全シナリオで基本設計 § 13.1 目標値 + http_req_failed < 1%。

ペネトレーション実施記録テンプレート (5 年保持):

- ・ 保管場所: docs/security/pentest/YYYY-MM-DD.md(TBD: 初回実施時に確定)
- ・ 記載項目: scope / 実施者 / 日付 / findings(Critical/High/Medium/Low)/ retest 結果 / 残課題チケット ID
- ・ 整合: GitHub Issue に findings を同期 (Critical/High はセキュリティ SLA に対応)

13.4.6 3.6 SCR × FR × AC × テストレベル早見表

SCR	主 FR	AC	ユニット	結合	E2E
SCR-090	FR-200, FR-223	-	検索条件バリデーション	API 経由表示	一覧表示 + 詳細遷移
SCR-091	FR-201~211	-	(a)~(g) 各ステップ	トランザクション + ロールバック	復元フロー全体 + 423 ケース
SCR-092	FR-055, FR-061~066	AC-034	階層解決	KV PUT + メイン同期	4-eyes ハードゲート + 段階ロールアウト
SCR-093	FR-121, FR-128	AC-040, AC-044	バリデーション	KV PUT	上書き + IF #5 同期
SCR-094	FR-149, FR-188-189	-	サニタイズ	1 分 cron 配信	予約 + 取消 + テスト送信 + 訂正告知
SCR-096	FR-229, FR-230, FR-232	AC-038	ハッシュチェーン計算	カーソル検索	エクスポート + HMAC 検証
SCR-097	FR-302, NFR-808	AC-041	状態遷移	DLQ 自動 BO + 手動リプレイ	30 日ウィンドウ
SCR-098	FR-060, FR-064	AC-036	ルール更新	KV 即時反映	3 営業日タイマー
SCR-099	FR-302 異常系	AC-041	差分検出アルゴリズム	同 event_id 再送	dismiss / reprocess

13.4.7 3.7 受入条件マッピング (本書側主管)

AC	主管	検証手段
AC-036(PII 3 層 + 報告フロー)	本書	単体 + 報告 E2E + 都度運用
AC-037(運営者 MFA + 再認証)	本書	認証テスト
AC-038(運営者操作監査 + 通知)	本書	E2E + 月次監査
AC-041(Webhook 冪等性 + 差分検出)	本書	統合テスト (差分検出ケース含) + リプレイ運用
AC-042(月次請求 cron 冪等)	本書	統合テスト
AC-046(p95 4 週間達成)	メイン主管 / 本書観測	Analytics Engine ダッシュボード

13.4.8 3.8 品質回帰データセット

データセット	件数 (MVP)	管理	実行
AI 品質回帰 (質問 → FAQ → 回答)	50	本書 SCR-096 経由	メイン CI
プロンプト注入	20+(役割再定義 / タグ脱出 / コード注入 / 言語切替 / 指示上書き / ロール変更 / SP 開示 / FAQ 外事実 / 機密抽出)	本書	メイン CI
PII 第 1 層	陽性 100 + 陰性 100 / 各国内パターン	本書	本書 CI
HTML サニタイズ攻撃	50 ペイロード	本書	本書 CI

13.4.9 3.9 負荷試験 (顧管側、メイン §17.5 と対称形)

メイン §17.5 で定義した 5 シナリオ (S1~S5) に加え、本書側 (運営者画面) 固有のシナリオを以下に定義する。

シナリオ	構成	目的	合格基準
(A1) 監査ログ大量検索	200 契約 × 5 年分監査ログ (約 550 万行) を GET /audit-logs で (action, occurred_at) 範囲検索 100 並列	SCR-096 検索性能 / インデックス選択性	p95 ☒ 1000ms
(A2) 監査ログ大量エクスポート	10 万行エクスポートを 5 並列で同時起動	R2 chunk PUT のスループット、メモリ上限 128MB の遵守	全ジョブ完了 ☒ 90s、メモリ上限 ヒット率 0%
(A3) Webhook 集中受信	Stripe Webhook 1000 件/分 × 5 分間	署名検証 + 冪等性ストア (KV) のスループット	p95 ☒ 500ms、web-hook_events.state 整合性 100%
(A4) 4-eyes 同時申請	100 申請を 10 秒間に集中投入、別運営者から 100 並列承認	operator_approvals テーブルの楽観ロック、同時実行制御	レース条件で重複承認 = 0 件、申請 → 承認 → 実行が単調進行
(A5) DLQ 大量リプレイ	1000 件の dlq_manual_replay を同時起動	サーキットブレーカ・指数 BO の挙動	サーキットブレーカ open 率が想定通り、5xx → 自動 BO で全件最終完了

実行: k6 + Miniflare 組合せ。月次 + 主要リリース前。初回実施日 + k6 シナリオファイル所在は SRE が運用設計に記載する (TBD)。

13.4.10 3.10 トレース ID 採番

採番場所	形式
エッジ (Cloudflare)	<cf-ray>(自動付与、request_id フィールド)
Worker 内	ULID 26 文字を新規発行、trace_id フィールド
クライアント送出	X-Trace-Id ヘッダ (任意、Worker は受け取った値を継承)
連携 IF #1~#12	mTLS + JWT に trace_id クレーム含有

trace_id は構造化ログ・audit_logs.trace_id 列・エラーレスポンス trace_id フィールドのすべてで一致する。

13.4.11 3.11 ログ集約 + 検索

項目	仕様
集約基盤	Cloudflare Logpush → 外部ストア (MVP では R2 + 必要時に DuckDB クエリ)
保持	エラーログ 90 日、トレースログ 30 日 (R2 lifecycle rule)
検索	運営者は SCR-096 経由でエラーログをフィルタ可能 (action_code = error.log で監査記録)
アラート	level=critical で運営者 critical alert(オーナー + users:manage 全員、強制送信)

13.4.12 3.12 テストインフラ

項目	仕様
ユニット	Vitest(vitest.config.ts、CI: GitHub Actions)
結合	Miniflare(vitest --pool=workers)
E2E	Playwright(playwright.config.ts、CI: nightly + リリース前)
Webhook 統合	Stripe Test Mode + カスタムフィクチャ
負荷	k6 + Miniflare(tests/load/*.js)
カバレッジレポート	Vitest coverage + CodeCov 連携

13.4.13 3.13 リリース判定チェックリスト (MVP)

- ☐ ユニット statements ☒ 80%、branches ☒ 75%
- ☐ 結合 API エンドポイント網羅率 100%
- ☐ E2E: SCR-090~099 主要フロー + 4-eyes 完全フロー (5 シナリオ)
- ☐ Webhook 統合: 署名検証 / 幂等 / 差分 / DLQ / リプレイ
- ☐ AI 品質回帰: 50 件 + プロンプト注入 20 件
- ☐ PII 第 1 層: 陽性 100 + 陰性 100
- ☐ HTML サニタイズ: 50 ペイロード
- ☐ 負荷試験: A1~A5 全合格
- ☐ ペネトレ: Critical/High 0 件 (年 1 回 + 重大機能変更時)
- ☐ AC-036 / AC-037 / AC-038 / AC-041 / AC-042 すべて緑

13.5 4. 関連設計

種別	参照先
要件	../01_要件定義/index.md
基本設計	../02_基本設計/index.md
エラー設計 (正本)	../02_基本設計/05_エラー設計.md
トレーサビリティマトリクス	../02_基本設計/07_トレーサビリティマトリクス.md
関連 DD	DD01 ~ DD11 全 11 ファイル
運用設計	../04_運用設計/index.md
将来対応	../05_future/index.md

13.6 5. テスト観点

(本書自体がテスト戦略を定義する文書のため、テスト観点はメタとして以下を含む。)

13.6.1 5.1 構造化ログのスキーマ整合性

- admin/src/shared/logger.ts の TypeScript 型定義と JSON Schema の一致を CI で検証
- 機密項目マスキング関数のユニットテスト (各種 PII / 認証情報)
- ログ集約後の検索 SLO(p95 ☒ 5s 以内に取得可能)

13.6.2 5.2 テストインフラ自体の検証

- Miniflare 環境での D1 / KV / R2 のローカル動作確認
- k6 シナリオの本番モック性 (統計分布の妥当性)
- Playwright の並列実行とレース条件耐性

13.6.3 5.3 受入条件カバレッジ計算

- AC ID をテスト記述に埋め込み、03_script/check-ac-coverage.sh(TODO: 整備中) で網羅率を計算
- カバレッジ < 95% で CI fail

13.7 6. 未確定事項・確認事項

確認事項 ID	確認内容	優先度	ステータス
-	v1.0 リリース時点で全項目確定済み	低	確認済
TBD-DD12-001	k6 シナリオファイルの初回実施日 (SRE 主管、運用設計に記載予定)	中	未確定
TBD-DD12-002	ペネトレーション実施記録の正式パス (docs/security/pentest/YYYY-MM-DD.md) の運用開始	中	未確定

\newpage